

Wydział Informatyki

Piotr Słupski

Zaawansowana aplikacja webowa do organizacji zadań

Praca: inżynierska
Kierunek: Informatyka
Specjalność: Programowanie
Nr albumu: 7481

Praca wykonana pod kierunkiem:
dr Leszek Grocholski

Wrocław 2026

Spis treści

Wstęp	3
1. Analiza wymagań	5
1.1 Omówienie podobnych aplikacji	5
1.2 Historie użytkownika	8
1.3 Kanban	8
1.4 Najważniejsze funkcje aplikacji	10
2. Frontend	11
2.1 Komponenty w React	12
2.2 React Hooks	15
2.3 Tailwind CSS	17
2.4 TypeScript	18
3. Backend	19
3.1 Next.js	19
3.2 Routing	20
3.3 Generowanie zawartości strony	21
3.4 API w Next.js	21
3.5 Autoryzacja i autentykacja	22
4. Bazy danych	24
4.1 MySQL	25
4.2 MongoDB	26
5. Architektura aplikacji	28
5.1 Omówienie warstw aplikacji	28
5.2 Diagramy architektoniczne	30
5.3 Struktura bazy danych aplikacji	32
6. Projekt interfejsu użytkownika	37
6.1 Scenariusze przypadków użycia	37
6.2 Widoki użytkownika aplikacji	43

7. Implementacja kodu po stronie frontendu	51
7.1 Implementacja widoku kanban	51
7.2 Implementacja sekcji przypiętych elementów dashboardu	54
7.3 Implementacja formularza taska	57
7.4 Implementacja formularza projektu	59
8. Implementacja kodu po stronie backendu	63
8.1 Implementacja handlera API checklist	63
8.2 Wspólne mechanizmy obsługi żądań API	66
8.3 Implementacja API odpowiedzialnego za autoryzację	69
9. Testy jednostkowe	71
9.1 Testy funkcjonalności tasków	71
9.2 Testy endpointów API tasków	73
Podsumowanie i zakończenie	76
Bibliografia	77
Spis rysunków	79
Spis tabel	81

Wstęp

Tematem niniejszej pracy inżynierskiej jest opracowanie wraz z implementacją aplikacji internetowej, służącej do organizacji zadań. Zaprojektowane narzędzie skierowane jest do użytkowników indywidualnych, potrzebujących pomocy w sprawnym planowaniu zarówno obowiązków dnia codziennego jak i bardziej złożonych zadań.

W praktyce codziennej organizacja zadań rzadko ogranicza się do jednej prostej listy. Użytkownik musi pamiętać o drobnych sprawach, terminach, notatkach oraz większych projektach, które składają się z wielu etapów. Z tego powodu aplikacja dostępna w przeglądarce powinna nie tylko zapisywać informacje, lecz także pomagać szybko odnaleźć to, co w danym momencie wymaga uwagi.

Inspiracją dla tematu były popularne aplikacje do zarządzania projektami, w których duży nacisk położono na współpracę zespołową. W przypadku pojedynczego użytkownika część takich funkcji staje się jednak zbędna: rozbudowane uprawnienia, widoki zespołów czy mechanizmy raportowania, potrafią odciągać uwagę, a nawet dezorientować niektórych użytkowników. Dlatego w projekcie przyjęto założenie, że najważniejsze rozwiązania znane z innych, podobnych aplikacji, powinny zostać uproszczone i przystosowane do pracy indywidualnej.

Strona internetowa powinna umożliwiać użytkownikom organizowanie pracy w sposób przejrzysty oraz intuicyjny, z wykorzystaniem funkcji takich jak na przykład przypinanie wybranych treści do widoku strony głównej. Istotnym założeniem projektu jest również zapewnienie bezpieczeństwa danych poprzez wdrożenie mechanizmów logowania, tworzenia konta użytkownika przez zewnętrznego dostawcę oraz autoryzacji dostępu do zasobów aplikacji.

Do celów szczegółowych pracy należy zaprojektowanie graficznego wyglądu interfejsu użytkownika, opracowanie logiki aplikacji zarówno po stronie klienckiej, jak i serwerowej, zaprojektowanie struktury bazy danych oraz implementacja mechanizmów obsługi kont użytkowników. Projekt zakłada wykorzystanie współczesnych technologii internetowych, umożliwiających budowę aplikacji działającej w architekturze klient-serwer, z rozdzieleniem warstwy prezentacji, logiki biznesowej oraz przechowywania danych.

Zakres realizowanych prac obejmuje analizę dostępnych na rynku rozwiązań, opis odpowiednich narzędzi programistycznych, przygotowanie widoków interfejsu użytkownika oraz przedstawienie implementacji kodu aplikacji. Zakres pracy nie obejmuje publikacji aplikacji w domenie publicznej ani jej komercyjnego wdrożenia.

Rezultatem pracy jest działająca aplikacja internetowa, w której użytkownik może tworzyć taski, checklisty, notatki i projekty, łączyć je ze sobą oraz wyróżniać najważniejsze elementy na stronie głównej.

Rozdział pierwszy przedstawia analizę wymagań projektowanej aplikacji. Omówiono w nim potrzeby użytkownika indywidualnego, porównano podobne narzędzia do organizacji zadań oraz wskazano wymagania funkcjonalne i нефункционалне systemu. W rozdziale opisano również historie użytkownika oraz podstawy wizualnej organizacji zadań w projektach.

Rozdział drugi dotyczy technologii warstwy interfejsu użytkownika, zastosowanych podczas tworzenia aplikacji. Przedstawiono w nim rolę podstawowych języków używanych do budowy interfejsu, a także opisano wykorzystanie biblioteki komponentowej, mechanizmów pracy ze stanem, narzędzia do stylowania oraz statycznego typowania. Rozdział ten wyjaśnia, w jaki sposób wskazane technologie wspierają budowę interaktywnego i responsywnego interfejsu użytkownika.

Rozdział trzeci opisuje technologie warstwy serwerowej oraz mechanizmy odpowiedzialne za działanie aplikacji po stronie serwera. Szczególną uwagę poświęcono zastosowanemu narzędziu projektowemu, sposobowi obsługi adresów aplikacji, generowaniu zawartości strony, komunikacji między częściami systemu oraz procesom autoryzacji i autentykacji użytkowników.

Rozdział czwarty przedstawia zagadnienia związane z przechowywaniem danych. Omówiono w nim podstawowe cechy baz danych oraz uzasadniono wybór dokumentowego rozwiązania dopasowanego do charakteru danych przetwarzanych przez aplikację.

Rozdział piąty zawiera opis architektury systemu. Przedstawiono w nim podział aplikacji na warstwy, przepływ danych, najważniejsze elementy struktury projektu oraz aktualny opis bazy danych zgodny ze stanem implementacji aplikacji.

Rozdział szósty prezentuje projekt interfejsu użytkownika oraz scenariusze przypadków użycia. Opisano w nim sposób korzystania z głównych funkcji systemu, takich jak obsługa konta użytkownika, praca z notatkami, elementami kontrolnymi, zadaniami i projektami oraz wykorzystanie widoku tablicowego.

Rozdział siódmy opisuje wybrane elementy implementacji kodu warstwy interfejsu użytkownika. Skupiono się w nim na wspólnych komponentach interfejsu, realizacji widoku tablicy projektu, sekcji przypiętych elementów dashboardu oraz formularzy odpowiedzialnych za tworzenie i edycję tasków oraz projektów.

Rozdział ósmy przedstawia wybrane elementy implementacji części serwerowej aplikacji. Opisano w nim handler API odpowiedzialny za pobieranie, tworzenie, aktualizowanie i archiwizowanie checklist, wspólne mechanizmy obsługi żądań API oraz konfigurację autoryzacji opartą na Auth.js, zewnętrznych dostawcach logowania i sesji użytkownika.

Rozdział dziewiąty dotyczy testów jednostkowych. Wyjaśniono w nim rolę testów w projekcie oraz opisano przygotowane testy walidacji danych tasków i testy endpointów API odpowiedzialnych za obsługę tasków. Wskazano również, jaki zakres działania aplikacji został objęty sprawdzeniem.

1. Analiza wymagań

Analizę wymagań rozpoczęto od określenia sytuacji, w których użytkownik korzysta z aplikacji samodzielnie, bez zespołu projektowego i bez formalnego procesu zarządzania pracą. W takim przypadku najważniejsze są: prosty zapis obowiązków, szybki powrót do aktualnie wykonywanych zadań oraz możliwość oddzielenia drobnych spraw od większych projektów. Narzędzia takie jak Trello, Asana czy ClickUp pokazują różne sposoby organizacji pracy, lecz wiele ich funkcji powstało z myślą o większej ilości użytkowników, raportowaniu i koordynacji zespołu.

Użytkownik indywidualny nie potrzebuje licznych nakładek i dodatkowych funkcjonalności aplikacji, usprawniających pracę zespołową, natomiast ceni sobie prostotę. Aplikacja dla niego skierowana powinna przede wszystkim zapewniać responsywny, intuicyjny, czytelny i niezawodny interfejs, umożliwiający bezproblemowe nawigowanie po stronie. Narzędzie takie powinno także umożliwiać organizację zadań w różnych formach, jak na przykład checklista¹ do sprawdzania zakupów, lub tablica z taskami² w celu organizacji skomplikowanego projektu. Aplikacja powinna również oferować prosty i motywujący do działania interfejs graficzny.

Z punktu widzenia projektowanej aplikacji istotny problem stanowi rozproszenie informacji. Użytkownik może prowadzić osobną listę codziennych zadań, oddzielne notatki i osobny plan większego projektu, przez co część danych traci kontekst. Dlatego projekt skupia się również na możliwości przypinania i wyświetlania różnych elementów aplikacji w jednym miejscu.

1.1 Omówienie podobnych aplikacji

Asana³, ClickUp⁴ oraz Trello⁵ to przykładowe, popularne narzędzia do zarządzania projektami. Z perspektywy pojedynczego użytkownika ich przydatność zależy od złożoności potrzeb oraz preferowanego stylu pracy. **Trello** uchodzi za proste i intuicyjne narzędzie z przej-

¹ Checklista - uporządkowany zestaw elementów do wykonania, w którym poszczególne pozycje można oznaczać jako ukończone. Jako element organizacji zadań występuje między innymi w aplikacji Trello. Źródło: Atlassian, *Trello*, strona internetowa aplikacji, [b.r.], <https://trello.com> (dostęp: 17.07.2026).

² Task - z jęz. ang. zadanie; w niniejszej pracy termin oznacza element aplikacji tworzony przez użytkownika w celu opisanie konkretnej czynności do wykonania. Jako element organizacji pracy występuje między innymi w aplikacji Trello. Źródło: Ibidem.

³ Asana Inc., *Asana*, strona internetowa aplikacji, [b.r.], <https://asana.com> (dostęp: 17.07.2026).

⁴ ClickUp, *ClickUp*, strona internetowa aplikacji, [b.r.], <https://clickup.com> (dostęp: 17.07.2026).

⁵ Atlassian, *Trello*, strona internetowa aplikacji, [b.r.], <https://trello.com> (dostęp: 17.07.2026).

rzystym interfejsem. **Asana** jest natomiast bardziej rozbudowana i wykorzystywana przede wszystkim do nadzoru nad złożonymi projektami zespołowymi. Z kolei **ClickUp** oferuje szeroki zakres funkcji, co może być atrakcyjne dla wymagających użytkowników indywidualnych, jednak duża liczba możliwości czyni tę aplikację skomplikowaną w obsłudze.

Trello wyróżnia się **prostotą i intuicyjnością** – jest łatwe do opanowania dla początkujących, dzięki czemu idealnie nadaje się do podstawowego porządkowania. Jego jedyna forma prezentacji - kanban⁶ sprzyja szybkiemu organizowaniu prostych projektów. Jednocześnie Trello pozostawia dużo przestrzeni użytkownikom bardziej wymagającym, poprzez możliwość integracji przeróżnych narzędzi zewnętrznych, tak zwanych ”power-upów”.

Asana, zaprojektowana została z myślą o pracy zespołowej, dostarczając bardziej ustrukturyzowane podejście do zarządzania zadaniami i projektem. W darmowej wersji obcięte zostały niektóre funkcjonalności, jednak nie posiada ona limitów na ilość zadań, czy projektów. Wiele istotnych funkcji oferowanych przez tę aplikację nie jest przydatne dla użytkownika pojedynczego. Plusem Asany jest stosunkowa prostota w obsłudze oraz czytelność interfejsu, w zestawieniu z ilością oferowanych funkcji.

ClickUp spośród analizowanej trójki dostarcza bardzo szeroki zestaw funkcji. Zaawansowane widoki, zależności między zadaniami, śledzenie czasu pracy, raporty oraz liczne integracje sprawiają, że jest to aplikacja rozbudowana i elastyczna. Taka obfitość narzędzi wpływa jednak na złożoność interfejsu ClickUp. Korzystając z niego, użytkownik musi poświęcić więcej uwagi na odnalezienie potrzebnej funkcji, a nauka wielu funkcjonalności aplikacji może wymagać dodatkowego czasu. Rozpatrując narzędzie w kontekście użytkownika indywidualnego, ClickUp może być atrakcyjniejszy od Asany ze względu na zakres dostępnych funkcji, natomiast wielu użytkownikom może bardziej odpowiadać Trello ze względu na prostotę.

Poniżej znajduje się porównanie najważniejszych wybranych funkcjonalności oferowanych przez trzy wcześniej wspomniane aplikacje. Warto zaznaczyć, że niektóre płatne funkcjonalności, a w szczególności limity zależą również od planu, który możemy wybrać. W każdej z wymienionych aplikacji istnieje kilka możliwości subskrypcji, kierowanych do różnych grup docelowych.

⁶ Kanban - metoda wizualnego zarządzania pracą, w której zadania przedstawiane są na tablicy podzielonej na etapy realizacji. Źródło: M. O. Ahmad, D. Dennehy, K. Conboy, M. Oivo, *Kanban in Software Engineering: A Systematic Mapping Study*, "Journal of Systems and Software" 2018, Vol. 137, s. 96–113, <https://doi.org/10.1016/j.jss.2017.11.045> (dostęp: 31.07.2026).

	 ASANA	 TRELLO	 CLICKUP
GRUPA DOCELOWA	DUŻE ZESPOŁY KORPORACJE	ZESPOŁY UŻYTKOWNICY INDYWIDUALNI	ZESPOŁY UŻYTKOWNICY INDYWIDUALNI
INTERFEJS	Skomplikowany, jednak bardziej czytelny niż ten w ClickUp.	Najprostszy i najbardziej intuicyjny. Kiedy ma się załączoną tablicę z projektem, nie są wyświetlane zakładki z dodatkowymi funkcjami aplikacji.	Mocno skomplikowany. Ciężko się w nim na pierwszy rzut oka odnaleźć. Posiada dużą ilość pasków z narzędziami i nawigacją.
MOŻLIWOŚĆ INTEGRACJI Z INNYMI NARZĘDZIAM (SLACK, GOOGLE DRIVE)			
WIDOKI PROJEKTÓW	- lista - tablica kanban - kalendarz - oś czasu (wersja płatna) - widoku Gantt'a do planowania projektu na wykresie czasowym (wersja płatna)	- karty, znajdujące się na listach w formie tablicy (kanban). - oś czasu (wersja płatna) - kalendarz (wersja płatna) - tabelaryczne zestawienie zadań (wersja płatna)	- lista - tablica kanban - kalendarz - oś czasu - tabela - widok Gantt'a - wiele innych
AUTOMATYZACJA	posiada narzędzie umożliwiające dodawanie reguł automatyzacji ale tylko w wersji płatnej aplikacji	- umożliwia tworzenie prostych reguł, przenoszących karty między listami - zapewnia automatyczne ustawianie terminów - limit do 250 komend automatyzacji miesięcznie	- limit do 100 automatyzacji na miesiąc w planie darmowym - plan płatny podnosi ten limit do 1000 miesięcznie
ZADANIA POWTARZALNE	Obsługuje w pełni zadania cykliczne, powtarzające się w określonych terminach	Wymagana jest do tego ustawienie reguły automatyzującej lub skorzystania ze specjalnej nakładki zapewnianej przez integracje z innymi aplikacjami	Obsługuje w pełni zadania cykliczne, powtarzające się w określonych terminach
FORMULARZE	Posiada możliwość tworzenia formularzy do zbierania informacji niezbędnych do utworzenia tasków. Ułatwia to pracę w dużych grupach.	Nie posiada wbudowanego narzędzia, zapewniającego tę funkcjonalność, ale można w tym celu korzystać z narzędzi zewnętrznych.	Podobnie jak Asana zapewnia narzędzie do tworzenia formularzy.
RAPORTY	Zapewnia pełne wsparcie generowania raportów oraz statystyk projektowych. Dodatkowo narzędzie do raportowania, w porównaniu do innych aplikacji jest łatwo dostępne.	Generowanie raportów umożliwiają integracje z innymi programami (tzw. Power-Up).	Umożliwia tworzenie zaawansowanych raportów oraz śledzenia statystyk, dzięki licznym widokom, które można ustawić na dashboard projektu.
DODATKOWE INFORMACJE	Płatna wersja umożliwia budowanie workflow, czyli tworzenie powtarzalnych procesów wraz z ustrukturyzowanymi sekwencjami zadań, które pomagają zespołom.	-	Aplikacja zapewnia w swoich narzędziach takie rozwiązania, które świetnie obsługują szybkie powiększanie się zespołu o kolejne osoby. Dzięki temu jest wysoce zalecana dla startupów, czy solowych projektów

Rys. 1. Tabela, porównująca Trello, Asanę oraz ClickUp.

Ostateczny wybór zależy od indywidualnych preferencji i potrzeb. Dla typowego użytkownika indywidualnego z prostą listą zadań lub małym projektem Trello może okazać się najwygodniejsze, podczas gdy **Asana** sprawdzi się tam, gdzie potrzebne jest bardziej formalne planowanie projektów. **ClickUp** będzie użyteczny w przypadku potrzeby wszechstronnego i elastycznego narzędzia do uporządkowania wielu różnorodnych zadań.

1.2 Historie użytkownika

Historie użytkownika wykorzystano jako krótkie opisy sytuacji, w których aplikacja ma pomóc w wykonaniu konkretnej czynności. Zamiast zaczynać od listy ekranów lub struktur danych, przyjęto perspektywę osoby, która chce coś szybko zapisać, uporządkować albo odnaleźć. Dzięki temu wymagania łatwiej powiązać z faktycznymi działaniami w interfejsie.

Każda historia wskazuje rolę użytkownika, jego potrzebę oraz powód wykonania danej czynności. Taki zapis pomógł wybrać funkcje najważniejsze dla pierwszej wersji systemu: przypinanie elementów, obsługę tasków, checklist, notatek i projektów oraz przełączanie sposobu prezentacji zadań w projekcie.

Poniżej zebrano pięć historii, które wyznaczają główne ścieżki pracy użytkownika w aplikacji.

1. Jako użytkownik indywidualny, chcę mieć możliwość przypięcia niektórych elementów do strony głównej w celu szybszego dostępu.
2. Jako pracownik biurowy, potrzebuję utworzyć kilka zadań wraz z opisami, określonymi priorytetami oraz czasem ich realizacji, w celu efektywniejszej pracy.
3. Jako głowa rodziny, potrzebuję utworzyć checklistę zakupów, aby w sklepie niczego nie zapomnieć.
4. Jako pracownik nadzorujący dostawy w restauracji, potrzebuję utworzyć notatkę z informacją odnośnie przebiegu dostawy oraz ilości palet zwróconych przez restaurację.
5. Jako student potrzebuję utworzyć projekt, zawierający datę realizacji, opis oraz listę tasków z możliwością nadawania im różnych statusów i wyświetlania w formie tablicy, by nie pogubić się podczas pisania pracy dyplomowej.

Na podstawie powyższych historii użytkownika opracowano szczegółowe scenariusze przypadków użycia, opisujące sposób interakcji użytkownika z systemem w konkretnych sytuacjach. Znajdują się one w szóstym rozdziale pracy.

1.3 Kanban

Kanban jest metodą organizacji pracy opartą na wizualnym przedstawianiu zadań oraz etapów ich realizacji. W aplikacjach do zarządzania zadaniami przyjmuje najczęściej postać ta-

blicy podzielonej na kolumny, które odpowiadają kolejnym stanom pracy. Zadania są przedstawiane jako karty, a ich przenoszenie pomiędzy kolumnami pozwala szybko ocenić, które elementy są dopiero planowane, które są aktualnie wykonywane, a które zostały już zakończone.

W literaturze dotyczącej inżynierii oprogramowania kanban jest opisywany jako podejście wspierające ciągły przepływ pracy oraz stopniowe dostarczanie wartości użytkownikowi. Ahmad, Dennehy, Conboy i Oivo wskazują, że badania nad kanbanem w oprogramowaniu koncentrują się między innymi na widoczności pracy, ograniczaniu pracy w toku oraz praktykach usprawniających zarządzanie zadaniami⁷.

Zaletą tablicy kanban jest jej prostota. Użytkownik nie musi analizować długiej listy zadań ani otwierać każdego elementu osobno, aby sprawdzić postęp pracy. Wystarczy spojrzeć na układ kart w kolumnach. Dzięki temu metoda dobrze sprawdza się zarówno w pracy zespołowej, jak i w pracy indywidualnej, na przykład podczas planowania projektu, nauki, pisanie pracy dyplomowej lub organizowania obowiązków domowych.

Przykładowa tablica kanban może wyglądać następująco:

Tabela 1. Przykładowa tablica kanban

Backlog	Do zrobienia	W trakcie	Testowanie	Zrobione
Pomysły na nowe funkcje	Przygotowanie formularza zadania	Implementacja widoku projektu	Sprawdzenie formularza	Konfiguracja logowania
Analiza podobnych aplikacji	Dodanie filtrowania zadań	Połączenie z bazą danych	Test działania checklist	Utworzenie modelu użytkownika

Kolumna *Backlog* przechowuje pomysły lub zadania, które nie zostały jeszcze wybrane do realizacji. Kolumna *Do zrobienia* zawiera elementy zaplanowane jako najbliższe prace. Kolumna *W trakcie* pokazuje zadania aktualnie wykonywane, natomiast *Testowanie* grupuje elementy wymagające sprawdzenia przed uznaniem ich za ukończone. Ostatnia kolumna, *Zrobione*, zawiera zadania zakończone.

W projektowanej aplikacji tablica kanban została wykorzystana w widoku projektu. Projekt zawiera własne kolumny, zawierające statusy realizacji zadań, a powiązane z nim zadania są przenoszone pomiędzy nimi. Takie rozwiązanie ułatwia kontrolowanie postępu pracy i pozwala użytkownikowi szybko zobaczyć, które zadania wymagają jeszcze uwagi.

⁷ M. O. Ahmad, D. Dennehy, K. Conboy, M. Oivo, *Kanban in Software Engineering: A Systematic Mapping Study*, "Journal of Systems and Software" 2018, Vol. 137, s. 96–113, <https://doi.org/10.1016/j.jss.2017.11.045> (dostęp: 31.07.2026).

1.4 Najważniejsze funkcje aplikacji

W tej części zebrano funkcje, będące dostępne w projekcie aplikacji, oraz cechy jakościowe wpływające na wygodę korzystania z systemu. Wymagania te wynikają bezpośrednio z wcześniej opisanych historii użytkownika.

Podstawową funkcjonalnością systemu jest umożliwienie użytkownikowi kompleksowego zarządzania elementami organizacyjnymi, którymi są checklisty, notatki, taski oraz projekty. Aplikacja wspiera pełen zestaw operacji CRUD⁸, co pozwala na swobodne dodawanie elementów, ich edycję oraz usuwanie w zależności od potrzeb użytkownika.

Istotnym wymaganiem systemu jest możliwość prezentacji projektów w formie tablicy kanban, z własnymi kolumnami statusów oraz możliwością przenoszenia zadań pomiędzy tymi kolumnami.

Dodatkowo aplikacja udostępnia panel strony głównej, określany jako dashboard, na którym wyświetlane są przypięte przez użytkownika dowolne elementy, takie jak na przykład checklisty. Funkcja ta umożliwia szybki dostęp do najważniejszych informacji bez konieczności przeszukiwania całej aplikacji. System pozwala także wzbogacać zadania i projekty o dodatkowe dane, takie jak opis, tagi tematyczne, poziom priorytetu czy przewidywany czas realizacji. Dzięki temu użytkownik może lepiej organizować pracę, filtrować elementy oraz ustalać kolejność wykonywania obowiązków.

Oprócz funkcjonalności biznesowych istotne znaczenie mają również aspekty jakościowe systemu. Aplikacja umożliwia personalizację poprzez zmianę motywów kolorystycznych oraz przełączanie interfejsu na tryb nocny, co zwiększa komfort użytkowania w różnych warunkach oświetleniowych. Interfejs powinien być minimalistyczny, intuicyjny oraz estetyczny, aby zapewnić łatwość obsługi nawet dla nowych użytkowników.

W zakresie bezpieczeństwa przewidziano integrację z zewnętrznymi dostawcami autentykacji, takimi jak Google i GitHub, co pozwala na logowanie bez przechowywania haseł w aplikacji. System powinien również zapewniać krótkie czasy ładowania dla kluczowych operacji oraz sprawne odświeżanie danych po zapisaniu zmian przez użytkownika.

Architektura aplikacji umożliwia późniejsze dodawanie kolejnych typów elementów i filtrów bez przebudowy całego systemu. W projekcie przewidziano także testy sprawdzające najważniejsze reguły związane z obsługą tasków, aby zmiany w kodzie nie naruszały podstawowych scenariuszy działania.

⁸ CRUD (z jęz. ang. Create, Read, Update, Delete) - zestaw podstawowych operacji wykonywanych na danych w systemach informatycznych, obejmujący ich tworzenie, odczyt, modyfikację oraz usuwanie.

2. Frontend¹

Frontend to część strony internetowej lub aplikacji, z którą użytkownik bezpośrednio wchodzi w interakcję. Obejmuje on wszystkie akcje, wykonujące się po stronie przeglądarki. Na stronie internetowej do frontendu zalicza się:

1. Układ strony - poszczególne elementy, jak sekcje, nagłówki, przyciski;
2. Wygląd strony - czcionki, kolory, animacje;
3. Interakcje - wypełnianie formularzy, obsługa klikania elementów strony, menu nawigacyjne;
4. Responsywność - automatyczne dostosowywanie się aplikacji do różnych wielkości ekranu.

Fundamentalne narzędzia używane do tworzenia frontendu to HTML², CSS³ oraz JavaScript⁴. HTML to język znaczników, odpowiadający za strukturę strony internetowej. Określa on semantyczny układ treści, takich jak nagłówki, akapity, listy, obrazy, czy formularze. HTML stanowi szkielet warstwy frontendowej aplikacji webowej, będąc interpretowanym przez przeglądarkę w celu prezentacji treści użytkownikowi. Poprawne i semantyczne rozmieszczenie znaczników HTML ma kluczowe znaczenie dla dostępności cyfrowej, ponieważ umożliwia prawidłową interpretację treści przez czytniki ekranu wykorzystywane przez osoby z niepełnosprawnościami wzroku.

CSS jest językiem służącym do definiowania warstwy prezentacji stron internetowych. Odpowiada za wygląd i układ elementów HTML, w tym kolory, czcionki, odstępy, pozycjonowanie oraz animacje. Dzięki określonemu mechanizmowi dziedziczenia, CSS umożliwia spójne oraz elastyczne zarządzanie stylem w obrębie całej aplikacji webowej, a jego poprawne zastosowanie wspiera czytelność interfejsu oraz dostępność treści dla użytkowników.

JavaScript natomiast jest językiem programowania, umożliwiającym nadanie stronie interaktywności i dynamicznego charakteru. Pozwala na reagowanie na działania użytkownika, modyfikowanie treści oraz struktury dokumentu HTML w czasie rzeczywistym, a także

¹ Frontend - termin pochodzący z jęz. ang., będący złożeniem słów “front” (przód, część widoczna) oraz “end” (część, warstwa), odnoszący się do tej części aplikacji, która jest bezpośrednio dostępna dla użytkownika.

² HTML - z jęz. ang. Hypertext Markup Language, czyli “hipertekstowy język znaczników”. Źródło: J. Dean, *Web Programming with HTML5, CSS, and JavaScript*, Jones & Bartlett Learning, Burlington 2017, s. 7–15.

³ CSS - z jęz. ang. Cascading Style Sheets, czyli “kaskadowe arkusze stylów”. Źródło: Ibidem, s. 73–75.

⁴ Źródło: Ibidem, s. 311–313.

komunikację z serwerem bez konieczności przeładowywania strony. JavaScript stanowi kluczowy element nowoczesnych aplikacji webowych, integrując warstwę prezentacji z logiką działania interfejsu użytkownika.

Początkowo HTML, CSS i JavaScript wystarczały do budowy prostych stron, w których najważniejsza była prezentacja treści, jednak wraz ze wzrostem złożoności współczesnych interfejsów użytkownika, wynikła potrzeba rozbudowy i rozszerzania podstawowych technologii frontendowych. W aplikacjach, takich jak system do organizacji zadań, sam układ dokumentu nie jest jednak wystarczający, ponieważ interfejs musi reagować na zmiany stanu, formularze, filtry i dane pobierane z serwera. Dlatego w projektach stron internetowych zaczęto korzystać z bibliotek, które porządkują komponenty i ułatwiają ponowne użycie kodu.

Biblioteki programistyczne to zbiory gotowych funkcji, klas oraz komponentów, które rozszerzają możliwości języków programowania i ułatwiają realizację określonych zadań. Ich celem jest ponowne wykorzystanie sprawdzonych rozwiązań oraz ograniczenie konieczności samodzielnego implementowania powtarzalnych mechanizmów. Pozwalają one na skrócenie czasu tworzenia oprogramowania oraz zwiększenie czytelności i niezawodności kodu. W kontekście technologii frontendowych biblioteki wspierają między innymi budowę interfejsu użytkownika, obsługę zdarzeń oraz zarządzanie danymi po stronie klienta.

Przykładem jest tutaj React⁵, czyli biblioteka języka JavaScript. Zgodnie z dokumentacją biblioteki React, dostarcza ona narzędzia pozwalające między innymi tworzyć komponenty nadające się do ponownego użycia w różnych sekcjach aplikacji. W ten sposób można przygotować na przykład przycisk lub nagłówek bez potrzeby wielokrotnego przepisywania tego samego kodu. Takie podejście oszczędza czas oraz poprawia przejrzystość kodu. Z tego powodu w projekcie zdecydowano się na wykorzystanie tej biblioteki.

Obok bibliotek, rozwinęło się dużo narzędzi programistycznych zwanych frameworkami⁶. Różnica pomiędzy biblioteką a frameworkiem dotyczy przede wszystkim stopnia kontroli nad przepływem aplikacji. Biblioteka dostarcza zestaw narzędzi i funkcji, które są wywoływane bezpośrednio przez programistę w wybranym miejscu. Framework natomiast narzuca określoną strukturę projektu oraz sposób działania aplikacji, przejmując kontrolę nad jej przepływem i wymuszając pewne ramy, których programista musi przestrzegać.

2.1 Komponenty w React

Komponenty w React stanowią podstawowy element budowy interfejsu użytkownika i umożliwiają podział aplikacji na mniejsze, niezależne moduły. Takie podejście redukuje powielanie kodu oraz ułatwia jego testowanie i utrzymanie. Modułowa struktura aplikacji opartej na React zwiększa również skalowalność projektu. Istotnym ułatwieniem w tworzeniu komponentów jest składnia JavaScript Extension, czyli JSX.

⁵ *React: The Library for Web and Native Interfaces*, dokumentacja biblioteki React, [b.r.], <https://react.dev> (dostęp: 28.01.2026).

⁶ Framework - z jęz. ang., jest złożeniem słów "frame" (rama) oraz "work" (praca, struktura). Dosłownie oznacza "ramę konstrukcyjną" lub "szkielet".

HTML jest językiem deklaratywnym służącym do opisu struktury dokumentu, natomiast JavaScript odpowiada za logikę i manipulację tą strukturą przez wywoływane funkcje. W klasycznym podejściu HTML i JavaScript są rozdzielone - pierwszy język definiuje widok, a drugi modyfikuje go poprzez interakcję z DOM⁷. Aby zmodyfikować DOM przy użyciu czystego JavaScriptu, należy skorzystać z wielu funkcji odpowiedzialnych za tworzenie, wyszukiwanie i aktualizowanie elementów dokumentu. JSX pozwala uprościć ten proces i opisywać strukturę interfejsu w sposób deklaratywny wewnątrz plików JavaScript. Znacznie ułatwia to definiowanie, utrzymanie oraz ponowne wykorzystanie komponentów. Dodatkowo składnia JSX jest podczas procesu budowania aplikacji transpilowana do czystego JavaScriptu, dzięki czemu React może efektywnie zarządzać komponentami, ich stanem oraz ponownym renderowaniem. Poniższy przykład wyjaśnia, jak w React działa JavaScript Extension.

```
function Button() {  
  const btn = document.createElement("button");  
  btn.textContent = "Kliknij";  
  btn.onclick = () => alert("Kliknięto");  
  return btn;  
}
```

Rys. 2. *Fragment kodu przedstawiający definiowanie komponentu "Button" w czystym JavaScript, bez nakładki JSX*

Jak widać na wyżej zamieszczonej grafice, w czystym JavaScriptcie komponent można zbudować wyłącznie poprzez imperatywne tworzenie elementów. Takie podejście jest mało czytelne, trudne w utrzymaniu i nie skaluje się dobrze w przypadku złożonych interfejsów.

```
function Button() {  
  return <button onClick={() => alert("Kliknięto")}>Kliknij</button>;  
}
```

Rys. 3. *Fragment kodu przedstawiający deklaratywne definiowanie komponentu "Button" dzięki JSX*

Wyżej przedstawiono, jak w sposób deklaratywny dodaje się elementy HTML dzięki bibliotece React. W React wyróżnia się dwa podstawowe typy komponentów - klasowe oraz

⁷ DOM - z jęz. ang. Document Object Model, to hierarchiczna reprezentacja struktury dokumentu HTML w postaci drzewa obiektów. Dean opisuje DOM jako model wszystkich części dokumentu strony w formie węzłów drzewa. Źródło: J. Dean, *Web Programming with HTML5, CSS, and JavaScript*, Jones & Bartlett Learning, Burlington 2017, s. 322.

funkcyjne, różniące się zarówno składnią, jak i sposobem zarządzania logiką aplikacji. Komponenty klasowe opierają się na klasach wprowadzonych do JavaScript wraz z aktualizacją ES6⁸. Dziedziczą one po klasie “React.Component”, a zarządzanie stanem oraz cyklem życia komponentu odbywa się za pomocą metod takich jak “constructor”, “componentDidMount”, czy “componentDidUpdate”. Taki model prowadzi często do bardziej rozbudowanego i mniej czytelnego kodu, zwłaszcza w przypadku złożonych komponentów. Poniżej widnieje przykład komponentu klasowego o nazwie “Button”. Komponent ten renderuje przycisk, którego treść oraz obsługa zdarzenia kliknięcia są przekazywane z zewnątrz za pomocą parametru “props”, co jest standardem zarówno w komponentach klasowych, jak i funkcyjnych.

```
import React from "react";

class Button extends React.Component {
  render() {
    return (
      <button onClick={this.props.onClick}>
        {this.props.title}
      </button>
    );
  }
}

export default Button;
```

Rys. 4. *Fragment kodu przedstawiający klasowy komponent “Button”*

Jeśli chodzi o komponenty funkcyjne, to są one zwykłymi funkcjami JavaScript, przyjmującymi dane wejściowe za pomocą “props” i zwracają strukturę interfejsu użytkownika. Początkowo były one ograniczone do komponentów prezentacyjnych, jednak wprowadzenie mechanizmu React Hooks rozszerzyło ich zastosowanie do pełnoprawnych elementów logiki aplikacji. Dzięki prostszej składni i lepszej modularności komponenty funkcyjne są obecnie preferowanym i rekomendowanym podejściem w nowoczesnych aplikacjach. Poniżej zamieszczono przykład tego samego komponentu “Button”, lecz w formie funkcji.

⁸ ES6 - skrót od “ECMAScript 2015”, jest to szósta edycja standardu języka JavaScript, która znacząco rozszerzyła możliwości języka.

```
import React from "react";

function Button({ onClick, title }) {
  return (
    <button onClick={onClick}>
      {title}
    </button>
  );
}

export default Button;
```

Rys. 5. Fragment kodu przedstawiający funkcyjny komponent "Button"

W projektowanej aplikacji podejście komponentowe zastosowano między innymi w formularzach, kartach elementów, widoku dashboardu oraz tablicy kanban. Dzięki temu powtarzalne fragmenty interfejsu, takie jak przyciski, pola formularzy, karty tasków i sekcje listujące dane, mogły zostać wydzielone do osobnych komponentów i użyte w kilku miejscach aplikacji.

2.2 React Hooks

Mechanizm hooków⁹ umożliwia przechowywanie danych pomiędzy kolejnymi wywołaniami funkcji komponentu, mimo że sama funkcja jest wykonywana od nowa przy każdym renderowaniu. Każdy hook jest wywoływany bezpośrednio w ciele funkcji komponentu, dzięki czemu React może jednoznacznie przypisać jego działanie do danego komponentu oraz zachować odpowiednie wartości pomiędzy kolejnymi wywołaniami tej funkcji. Kolejność wywołań hooków ma kluczowe znaczenie, ponieważ na jej podstawie React identyfikuje, które dane należą do danego hooka.

W odróżnieniu od zwykłych zmiennych lokalnych, które są tracone po zakończeniu wykonania funkcji, wartości zarządzane przez hooki są przechowywane poza samą funkcją, lecz udostępniane jej ponownie przy każdej aktualizacji interfejsu użytkownika. Dzięki temu komponent funkcyjny może być ponownie wykonywany, zachowując ciągłość danych oraz logiki aplikacji.

W JavaScriptcie zasięg funkcji określa obszar kodu, w którym dana zmienna lub funkcja jest dostępna. W przypadku komponentów funkcyjnych oznacza to, że wszystkie zmienne, hooki oraz funkcje pomocnicze zadeklarowane wewnątrz komponentu są dostępne wyłącznie w obrębie jego ciała. Zapewnia to, że stan oraz logika komponentu pozostają hermetyczne i nie kolidują z innymi komponentami, co sprzyja modularności, przewidywalności oraz bezpieczeństwu struktury aplikacji.

⁹ *Built-in React Hooks*, dokumentacja hooków biblioteki React, [b.r.], <https://react.dev/reference/react/hooks> (dostęp: 16.08.2026).

Zarządzanie stanem aplikacji w komponencie polega na przechowywaniu oraz aktualizowaniu danych, które mają bezpośredni wpływ na wyświetlany interfejs użytkownika. Stan reprezentuje bieżące parametry komponentu, takie jak wartości formularza, licznik, widoczność elementów, czy dane pobrane z serwera. Zmiana stanu powoduje ponowną aktualizację widoku komponentu w celu odzwierciedlenia nowych danych.

W komponentach funkcyjnych, stan jest obsługiwany za pomocą specjalnej funkcji, składającej się z aktualnej wartości stanu oraz metody służącej do jej zmiany. Funkcja ta nie modyfikuje stanu bezpośrednio, lecz informuje Reacta o konieczności jego aktualizacji.

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Licznik: {count}
    </button>
  );
}

export default Counter;
```

Rys. 6. Przykład funkcji zarządzającej stanem komponentu "Counter"

W powyższym przykładzie:

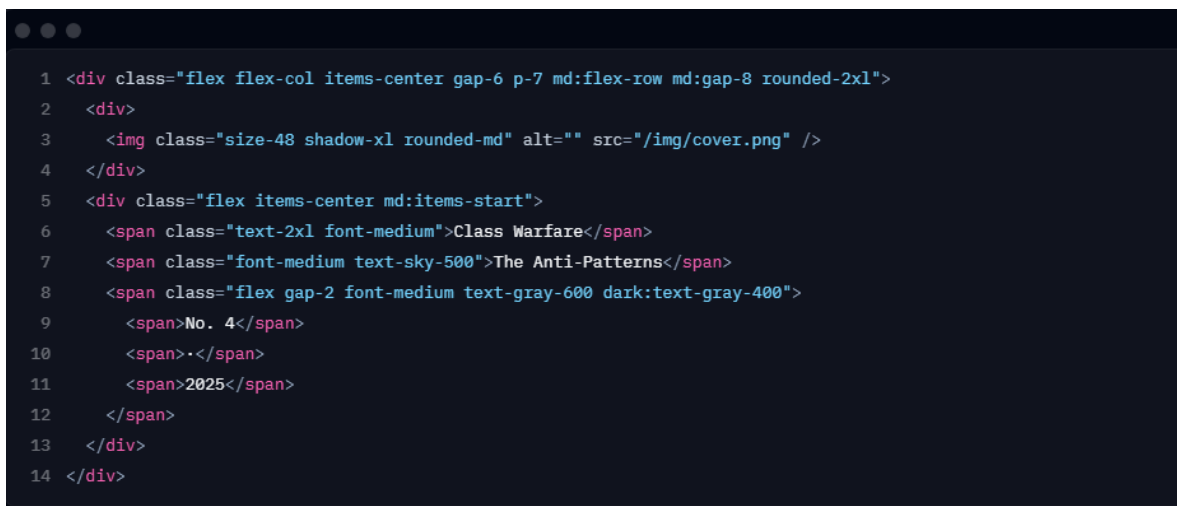
- zmienna "count" przechowuje aktualny stan licznika,
- funkcja "setCount" odpowiada za aktualizację stanu,
- wywołanie funkcji "setCount" powoduje zmianę danych oraz odświeżenie widoku komponentu na stronie

Takie podejście zapewnia enkapsulację danych wewnątrz komponentu oraz precyzyjne sterowanie jego zachowaniem. Dzięki hookom zarządzanie stanem staje się czytelne i ściśle powiązane z zakresem funkcji komponentu, co ułatwia rozwój i utrzymanie aplikacji.

W aplikacji hooki wykorzystano przede wszystkim w komponentach interaktywnych. "useState" przechowuje stan formularzy, filtrów, sortowania oraz przeciąganych tasków, "useMemo" służy do wyliczania przefiltrowanych list i kolumn kanban, a "useEffect" synchronizuje stan komponentu z danymi przekazanymi z serwera.

2.3 Tailwind CSS

Tailwind CSS¹⁰ jest frameworkiem służącym do stylowania interfejsów użytkownika za pomocą gotowych klas CSS, używanych bezpośrednio w znacznikach HTML lub komponentach React. Tailwind CSS umożliwia szybkie budowanie spójnych i responsywnych układów bez konieczności pisania osobnych arkuszy stylów dla każdego komponentu. Technologia ta jest stosowana w celu zwiększenia produktywności, ograniczenia duplikacji kodu CSS oraz łatwiejszego skalowania warstwy prezentacyjnej aplikacji.



```
1 <div class="flex flex-col items-center gap-6 p-7 md:flex-row md:gap-8 rounded-2xl">
2   <div>
3     
4   </div>
5   <div class="flex items-center md:items-start">
6     <span class="text-2xl font-medium">Class Warfare</span>
7     <span class="font-medium text-sky-500">The Anti-Patterns</span>
8     <span class="flex gap-2 font-medium text-gray-600 dark:text-gray-400">
9       <span>No. 4</span>
10      <span>•</span>
11      <span>2025</span>
12    </span>
13  </div>
14 </div>
```

Rys. 7. Przykładowe użycie Tailwind CSS, pochodzące z oficjalnej strony frameworka
(dostęp: 28.01.2026)

Jak widać na obrazie, całe stylowanie realizowane jest poprzez tak zwane klasy narzędziowe (po angielsku utility classes) Tailwinda, bez użycia osobnych plików CSS. Dla przykładu klasa “flex” nadana znacznikowi “div” jest odpowiednikiem zapisu “display: flex;” w CSS. W projekcie aplikacji Tailwind CSS wykorzystano, ponieważ ogranicza potrzebę definiowania własnych klas CSS oraz umożliwia szybkie tworzenie responsywnych i spójnych komponentów. Eliminuje także potrzebę przechodzenia do osobnych plików stylów w celu sprawdzenia struktury wizualnej znacznika lub komponentu. W projekcie Tailwind został użyty między innymi do przygotowania układów formularzy, kart elementów, bocznej nawigacji, widoku dashboardu oraz tablicy kanban. Klasy narzędziowe pozwoliły zachować spójne odstępy, kolory, responsywność i warianty trybu ciemnego bez tworzenia osobnych plików stylów dla każdego widoku.

¹⁰ Tailwind Labs, *Tailwind CSS*, dokumentacja frameworka CSS, [b.r.], <https://tailwindcss.com> (dostęp: 28.01.2026).

2.4 TypeScript

Kolejną technologią zastosowaną w projekcie jest TypeScript¹¹, czyli nadzbiór języka JavaScript. Rozszerza on JavaScript o system typów, interfejsy oraz mechanizmy kontroli poprawności kodu na etapie kompilacji. Kod TypeScript nie jest bezpośrednio interpretowany przez przeglądarkę, lecz przed uruchomieniem musi zostać transpilowany do standardowego JavaScriptu, zachowując pełną kompatybilność ze środowiskiem wykonawczym.

W projekcie TypeScript został wykorzystany przede wszystkim po to, aby jawnie opisać dane przekazywane między komponentami, formularzami i endpointami API. Statyczne typowanie pomaga wykrywać część pomyłek jeszcze podczas pracy nad kodem, na przykład niezgodny typ pola, brak wymaganej właściwości albo błędny argument funkcji. Ma to znaczenie szczególnie w miejscach, w których ten sam obiekt taska lub projektu przechodzi przez formularz, walidację, handler API i model bazy danych.

W samym JavaScriptcie wiele takich błędów ujawnia się dopiero po uruchomieniu aplikacji. Przykładem może być literówka w nazwie pola obiektu albo przekazanie wartości liczbowej tam, gdzie komponent oczekuje tekstu. TypeScript nie zastępuje testów ani walidacji danych wejściowych, ale stanowi dodatkową warstwę kontroli podczas implementacji. Dzięki temu zmiana struktury danych, np. dodanie pola do projektu lub taska, szybciej informuje o miejscach wymagających aktualizacji.

Jak dokładnie działa TypeScript, można wytłumaczyć na następującym przykładzie: tworzony jest komponent odpowiedzialny za wyświetlanie na stronie danych użytkownika, takich jak imię, adres e-mail oraz wiek. W TypeScriptcie definiowany jest typ lub interfejs, który jednoznacznie określa, jaką strukturę muszą mieć parametry przyjmowane przez komponent. Na przykład imię i adres e-mail muszą być zmiennymi typu "string", a wiek - "number". Następnie komponent lub funkcja przyjmująca dane użytkownika deklaruje, że oczekuje dokładnie takiego typu danych. Dzięki temu TypeScript już na etapie pisania kodu sprawdza, czy przekazywane wartości są poprawne. Jeżeli programista spróbuje przekazać zmienną typu "number" zamiast "string" lub pominie wymagane pole, błąd zostanie wykryty jeszcze przed uruchomieniem aplikacji. W trakcie dalszego rozwoju projektu, gdy struktura danych użytkownika ulegnie zmianie, TypeScript automatycznie wskaże wszystkie miejsca w aplikacji, które wymagają aktualizacji. Środowiska programistyczne posiadają możliwość integracji z TypeScriptem poprzez wyświetlanie na bieżąco powiadomień i ostrzeżeń o potencjalnych błędach wykrywanych przez ten język.

W projekcie TypeScript został użyty do opisu typów danych przekazywanych do komponentów, formularzy, funkcji pomocniczych oraz endpointów API. Dzięki temu struktury takie jak task, checklista, projekt, pin czy element formularza mają jasno określone pola, co zmniejsza ryzyko pomyłek podczas rozwijania aplikacji.

¹¹ Microsoft, *TypeScript Documentation*, dokumentacja języka TypeScript, [b.r.], <https://www.typescriptlang.org/docs/> (dostęp: 25.07.2026).

3. Backend¹

Backend to warstwa aplikacji działająca po stronie serwera i odpowiedzialna za przetwarzanie danych oraz komunikację z bazą danych lub innymi systemami. Backend nie jest bezpośrednio widoczny dla użytkownika końcowego. Obsługuje m.in. autoryzację, walidację danych, realizację reguł biznesowych oraz udostępnianie danych frontendowi za pomocą API².

API natomiast to zestaw reguł, metod i struktur danych umożliwiających komunikację pomiędzy różnymi systemami informatycznymi lub ich częściami. Określa, w jaki sposób aplikacje mogą wymieniać dane oraz wywoływać określone funkcje, bez konieczności znajomości wewnętrznej implementacji drugiej strony. API pełni rolę pośrednika pomiędzy frontendem a backendem. Frontend wysyła żądania lub zapytania, natomiast backend przetwarza je, zwracając później odpowiedź.

3.1 Next.js

Next.js³ jest frameworkiem opartym na bibliotece React, przeznaczonym do tworzenia pełnostosowych aplikacji webowych. Interfejs użytkownika budowany jest przy użyciu komponentów React, natomiast Next.js dostarcza dodatkowe funkcje oraz mechanizmy optymalizacyjne. Framework automatycznie konfiguruje narzędzia niskiego poziomu, takie jak bundlery oraz kompilatory, dzięki czemu programista może skupić się na tworzeniu aplikacji i szybkim wdrażaniu jej do użytku.

Next.js rozszerza możliwości React o funkcje takie jak renderowanie zawartości po stronie serwera, routing oparty na strukturze plików, optymalizację obrazów oraz wygodne tworzenie endpointów API. Umożliwia również budowę aplikacji przyjaznych dla wyszukiwarek internetowych, ponieważ część treści może zostać przygotowana przed wysłaniem strony do przeglądarki. Dzięki temu framework łączy w jednym projekcie warstwę interfejsu użytkownika i wybrane mechanizmy backendowe.

W projekcie zdecydowano się na wykorzystanie Next.js, ponieważ framework ogranicza potrzebę utrzymywania osobnego projektu backendowego i ułatwia rozwijanie aplikacji jako jednej spójnej całości. W decyzji wykorzystania Nexta istotna była także możliwość

¹ Backend - termin pochodzący z jęz. ang., będący złożeniem słów “back” (tył, zaplecze) oraz “end” (część, warstwa), odnoszący się do części aplikacji niedostępnej dla użytkownika.

² API - skrót z jęz. ang., oznaczający Application Programming Interface.

³ Vercel, *Next.js Docs*, dokumentacja frameworka Next.js, [b.r.], <https://nextjs.org/docs> (dostęp: 29.01.2026).

prostego połączenia z biblioteką `Auth.js`⁴, która pozwala zabezpieczać widoki i endpointy bez tworzenia całego mechanizmu logowania od podstaw.

Dodatkową zaletą `Next.js` jest wsparcie dla SEO⁵, w tym lepszej indeksowalności strony dzięki renderowaniu treści po stronie serwera oraz możliwości zarządzania metadany dokumentu. Duże znaczenie ma również opcja wygodnego rozwijania widoków użytkownika i kodu serwerowego w tym samym środowisku technologicznym.

3.2 Routing

Routing w `Next.js` stanowi mechanizm odpowiedzialny za mapowanie adresów URL na konkretne widoki aplikacji. W przeciwieństwie do klasycznych aplikacji `React`, w których konfiguracja tras wymaga dodatkowych bibliotek i ręcznego definiowania reguł nawigacji, `Next.js` wykorzystuje routing oparty na strukturze plików - z jęz. ang. file-based routing. Ścieżki URL są automatycznie generowane na podstawie organizacji katalogów i plików w projekcie.

Podstawową zasadą działania tego mechanizmu jest przypisanie każdemu plikowi w katalogu `“pages”` lub `“app”` odpowiadającej mu trasy. Przykładowo plik `“about.js”` generuje ścieżkę `“/about”`, natomiast plik umieszczony w podkatalogu `“blog/post.js”` odpowiada przeglądarkowemu adresowi `“/blog/post”`. Takie rozwiązanie upraszcza konfigurację projektu, zwiększa czytelność struktury aplikacji oraz ogranicza konieczność tworzenia dodatkowych plików konfiguracyjnych.

`Next.js` obsługuje również routing dynamiczny, który umożliwia tworzenie stron o zmiennych parametrach, np. dla wpisów blogowych czy profili użytkowników. Realizowane jest to poprzez specjalną składnię nazw plików, co pozwala na generowanie wielu widoków na podstawie jednego szablonu komponentu. Aby utworzyć dynamiczną ścieżkę, zmieniającą się w zależności od parametru `“id”` użytkownika, należy w katalogu `“users”` utworzyć plik o nazwie `“[id].js”`. Wewnątrz komponentu parametr `“id”` można odczytać ze specjalnego obiektu routingu. Przykładowo, pojedynczy komponent może generować widok profilu na podstawie identyfikatora przekazanego w adresie URL. Takie podejście pozwala na tworzenie wielu stron przy użyciu jednego szablonu, co zwiększa modularność oraz skalowalność aplikacji. Dodatkowo framework wspiera nawigację po stronie klienta, realizowaną za pomocą komponentu `“Link”`, dzięki czemu przejścia pomiędzy stronami odbywają się bez pełnego przeładowania dokumentu. Ten aspekt znacznie poprawia płynność działania interfejsu użytkownika.

W projektowanej aplikacji mechanizm routingu pozwolił powiązać strukturę katalogów z modułami widocznymi dla użytkownika, takimi jak dashboard, taski, checklisty, notatki i projekty. Dzięki temu ścieżki widoków oraz endpointów API są łatwiejsze do odnale-

⁴ `Auth.js`, *Auth.js*, dokumentacja biblioteki do obsługi uwierzytelniania, [b.r.], <https://authjs.dev> (dostęp: 17.07.2026).

⁵ SEO - z jęz. ang. Search Engine Optimization, czyli optymalizacja strony pod kątem wyszukiwarek internetowych. Dean wskazuje, że określony sposób budowy strony może wpływać na to, czy treść zostanie uwzględniona przez mechanizmy indeksujące wyszukiwarek. Źródło: J. Dean, *Web Programming with HTML5, CSS, and JavaScript*, Jones & Bartlett Learning, Burlington 2017, s. 305.

zienia w kodzie.

3.3 Generowanie zawartości strony

Domyślnie Next.js używa serwerowych komponentów React. Wykonują się one najpierw po stronie serwera, a następnie ich wynik wysyłany jest do klienta. Komponenty te zapewniają wsparcie dla obiektów typu Promise w JavaScriptcie, umożliwiając wykonywanie operacji asynchronicznych bez konieczności używania dodatkowych bibliotek czy hooków React. Pobieranie danych odbywa się asynchronicznie, a ponieważ komponenty serwerowe działają po stronie serwera, możliwe jest bezpośrednie wykonywanie zapytań do bazy danych bez konieczności korzystania z dodatkowej warstwy API. Pozwala to ograniczyć ilość kodu, który trzeba pisać i utrzymywać.

Pobrane dane można generować i przetwarzać na dwa różne sposoby. Renderowanie statyczne polega na generowaniu treści na etapie budowania aplikacji lub podczas jej ponownej walidacji, a następnie dostarczaniu gotowych stron użytkownikom, co zwiększa wydajność i zmniejsza obciążenie serwera oraz sprzyja optymalizacji SEO. Renderowanie dynamiczne natomiast, odbywa się przy każdym żądaniu użytkownika, umożliwiając prezentację danych aktualnych w czasie rzeczywistym. Pierwsze podejście sprawdza się w przypadku stron o stałej zawartości, natomiast drugie jest preferowane w aplikacjach, wymagających częstych aktualizacji danych, takich jak panele administracyjne czy profile użytkowników.

Buforowanie danych polega na tymczasowym przechowywaniu wyników wcześniej wykonanych operacji, takich jak zapytania do bazy danych lub odpowiedzi z API. W aplikacjach z danymi użytkownika mechanizm ten trzeba stosować ostrożnie, ponieważ część widoków, na przykład dashboard lub lista tasków, powinna po zapisie szybko pokazywać aktualny stan.

Next.js wspiera ograniczanie powtarzania tych samych zapytań w obrębie renderowania. Ma to znaczenie wtedy, gdy kilka części widoku korzysta z podobnych danych i nie powinno wymuszać osobnych, identycznych operacji. W projekcie ważniejsze od agresywnego buforowania było jednak zachowanie poprawnego odświeżania po utworzeniu lub edycji elementu.

3.4 API w Next.js

Ścieżki API w Next.js to mechanizm, umożliwiający definiowanie endpointów⁶ API bezpośrednio w projekcie Next.js. Punkty końcowe można tworzyć poprzez umieszczenie plików w katalogu “pages/api”, a każda taka funkcja obsługuje żądania HTTP i zwraca odpowiedź

⁶ Endpoint - z jęz. ang. punkt końcowy; w kontekście API oznacza konkretny adres lub ścieżkę, pod którą aplikacja udostępnia określoną operację albo zasób.

w modelu “request/response”⁷.

W nowszym podejściu odpowiednikiem ścieżek API są tzw. “Route Handlers”⁸, definiowane w katalogu “app” i pliku “route.js”. W tym modelu obsługa metod HTTP realizowana jest przez eksport funkcji takich jak “GET”, “POST”, “PUT” czy “DELETE”, co pozwala tworzyć endpointy w sposób bardziej zbliżony do standardowych interfejsów typu “request/response”. W implementowanej aplikacji wykorzystano właśnie ten model, a poszczególne endpointy zapisano w plikach “route.ts” wewnątrz katalogu “app”.

Zastosowanie API Routes obejmuje m.in. obsługę formularzy, operacje CRUD, integracje z usługami zewnętrznymi, czy realizację logiki po stronie serwera, która nie powinna być wykonywana w przeglądarce. Istotną zaletą jest fakt, że kod endpointów nie trafia do klienta, dzięki czemu możliwe jest bezpieczniejsze przetwarzanie danych oraz użycie zmiennych środowiskowych po stronie serwera.

W projektowanej aplikacji endpointy API pełnią rolę kontrolowanej bramy pomiędzy interfejsem użytkownika a bazą danych. To po ich stronie możliwe jest sprawdzenie sesji użytkownika, walidacja przesłanych danych, odrzucenie niepoprawnego żądania oraz wykonanie zapisu lub odczytu w bazie. Dzięki temu frontend nie musi znać szczegółów działania modeli danych, a użytkownik nie otrzymuje bezpośredniego dostępu do warstwy przechowywania informacji.

Takie podejście ułatwia utrzymanie spójności logiki biznesowej. Jeżeli kilka widoków aplikacji korzysta z tej samej operacji, na przykład tworzenia zadania albo aktualizowania checklisty, reguły tej operacji mogą zostać zapisane w jednym handlerze API. Zmniejsza to ryzyko rozbieżności między widokami oraz upraszcza późniejsze testowanie zachowania systemu.

3.5 Autoryzacja i autentykacja

W Next.js autentykację i autoryzację można wdrożyć poprzez integrację z dostawcą OAuth⁹ przy użyciu biblioteki Auth.js. Rozwiązanie to pozwala skonfigurować logowanie przez zewnętrznych dostawców, takich jak Google czy GitHub. Biblioteka automatycznie obsługuje sesje użytkownika, przekazywanie tokenów oraz proces powrotu użytkownika do aplikacji po zakończonym logowaniu.

Auth.js działa jako warstwa pośrednia pomiędzy aplikacją Next.js a zewnętrznym

⁷ Interfejs typu request/response – model komunikacji pomiędzy klientem a serwerem, w którym klient wysyła żądanie (request) zawierające określone dane lub operację, a serwer przetwarza je i zwraca odpowiedź (response) z wynikiem działania, danymi lub informacją o błędzie. Dean opisuje analogiczny przepływ przy przetwarzaniu formularzy po stronie serwera: dane formularza są przesyłane do serwera, serwer wykonuje obliczenia i odsyła odpowiedź do klienta. Źródło: J. Dean, *Web Programming with HTML5, CSS, and JavaScript*, Jones & Bartlett Learning, Burlington 2017, s. 324.

⁸ Vercel, *Route Handlers*, dokumentacja Next.js dotycząca obsługi endpointów, [b.r.], <https://nextjs.org/docs/app/getting-started/route-handlers> (dostęp: 16.08.2026).

⁹ OAuth - z jęz. ang. Open Authorization, to standard protokołu autoryzacji umożliwiający aplikacjom uzyskanie dostępu do zasobów użytkownika w innym systemie bez konieczności przekazywania jego hasła. Zamiast tego wykorzystywane są specjalne tokeny dostępu, które określają zakres przyznanych uprawnień.

dostawcą tożsamości. Po stronie aplikacji programista definiuje dostawców logowania, strategię sesji, adapter bazy danych oraz callbacki, które pozwalają dopisać do sesji informacje potrzebne w dalszej części systemu¹⁰. Dzięki temu mechanizm logowania nie musi być implementowany ręcznie od podstaw, a aplikacja może korzystać z ustandaryzowanego przepływu obejmującego przekierowanie użytkownika, obsługę odpowiedzi zwrotnej i utworzenie sesji.

Autentykacja polega na potwierdzeniu tożsamości użytkownika, natomiast autoryzacja określa, do jakich zasobów i operacji użytkownik ma dostęp po zalogowaniu. W aplikacji do zarządzania zadaniami rozróżnienie to ma duże znaczenie, ponieważ samo zalogowanie użytkownika nie powinno jeszcze oznaczać dostępu do cudzych projektów, checklist, notatek lub tasków.

Po zalogowaniu aplikacja może kontrolować dostęp do zasobów na podstawie informacji z sesji, np. identyfikatora użytkownika, ról lub uprawnień. W praktyce każdy endpoint obsługujący dane powinien sprawdzać, czy żądanie pochodzi od zalogowanego użytkownika oraz czy wskazany dokument należy właśnie do niego. Pozwala to ograniczyć dostęp do wybranych stron i endpointów API oraz zmniejsza ryzyko nieautoryzowanej modyfikacji danych.

¹⁰ Auth.js, *Auth.js*, op. cit.

4. Bazy danych

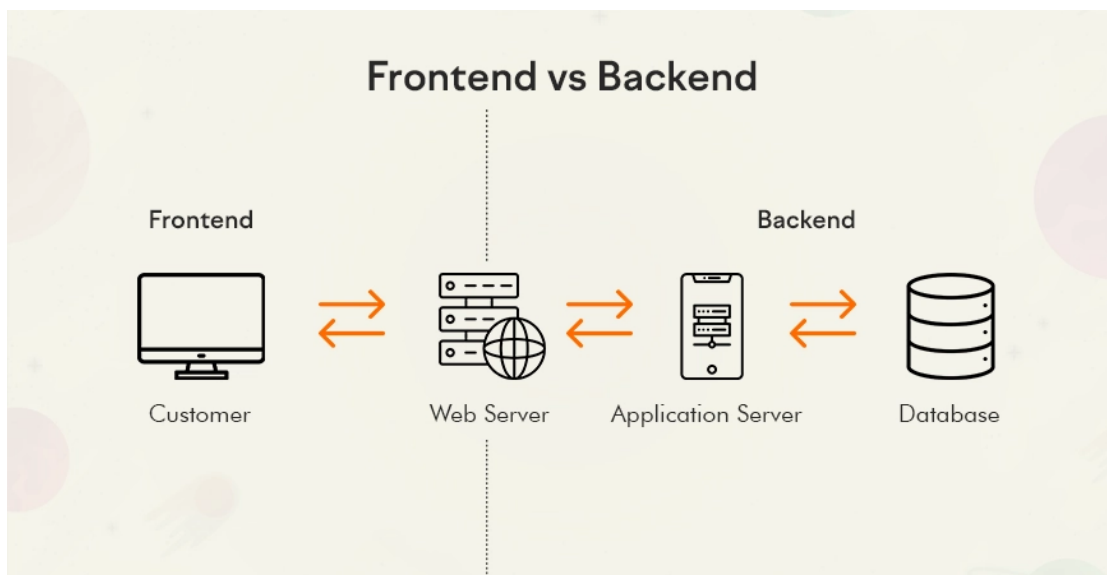
Baza danych to system służący do trwałego przechowywania, organizowania oraz zarządzania danymi, wykorzystywanymi przez aplikację. Umożliwia ona zapisywanie informacji w ustrukturyzowanej formie, ich wyszukiwanie, modyfikację oraz usuwanie, zapewniając jednocześnie spójność, integralność i bezpieczeństwo przechowywanych informacji. Najczęściej stosuje się bazy relacyjne lub nierelacyjne, w zależności od charakteru przetwarzanych danych.

Relacyjne bazy danych to systemy, przechowujące dane w postaci tabel, składających się z wierszy i kolumn, powiązanych ze sobą za pomocą relacji. Wykorzystują one ustrukturyzowany schemat danych oraz język zapytań SQL¹, co zapewnia wysoką spójność i integralność informacji. Sprawdzają się szczególnie w aplikacjach, które wymagają precyzyjnych zależności między danymi.

Bezrelacyjne bazy danych, zwane bazami noSQL, przechowują dane w bardziej elastycznej formie. Mogą to na przykład być dokumenty, pary klucz–wartość, grafy lub kolekcje. Takie bazy nie wymagają sztywno zdefiniowanego schematu, co ułatwia skalowanie oraz pracę z dużymi i zróżnicowanymi zbiorami danych.

Poniższy rysunek ogólnie pokazuje udział frontendu, backendu i bazy danych w procesie pobierania oraz prezentowania informacji użytkownikowi. Baza danych uzupełnia frontend i backend, ponieważ przechowuje stan aplikacji pomiędzy kolejnymi działaniami użytkownika. Frontend odpowiada za pokazanie wyniku w czytelnej formie, backend kontroluje żądanie i przygotowuje operację, natomiast baza zwraca dane, które następnie mogą zostać przekształcone w widok strony. Dzięki temu przedstawiony na rysunku przepływ nie kończy się na samej komunikacji klienta z serwerem, lecz obejmuje również trwałe źródło informacji potrzebnych do zbudowania odpowiedzi.

¹ SQL - z jęz. ang. Structured Query Language, co oznacza ustrukturyzowany język zapytań.



Rys. 8. Przedstawienie procesu, umożliwiającego wyświetlenie danych użytkownikowi aplikacji internetowej. Źródło:

<https://d1zruf9db62p8s.cloudfront.net/2025/08/Frontend-vs-Backend.webp> (dostęp: 02.02.2026).

4.1 MySQL

MySQL² należy do najczęściej wykorzystywanych technologii bazodanowych w aplikacjach internetowych ze względu na wysoką wydajność, stabilność, łatwość konfiguracji oraz szerokie wsparcie społeczności i narzędzi programistycznych. Ten typ bazy danych wykorzystuje język SQL do definiowania struktury danych, wykonywania zapytań oraz zarządzania integralnością informacji.

Dane w MySQL przechowywane są w tabelach złożonych z wierszy – tak zwanych rekordów i kolumn, zwanych atrybutami. Relacje pomiędzy tabelami realizowane są za pomocą kluczy głównych i obcych, co pozwala zachować spójność danych oraz minimalizować redundancję. Klucz główny to kolumna lub zestaw kolumn, których wartość jednoznacznie identyfikuje każdy rekord w tabeli. Wartości klucza głównego muszą być unikalne i nie mogą przyjmować wartości pustych (null). Natomiast klucz obcy to kolumna lub zestaw kolumn w tabeli, odwołujący się do klucza głównego w innej tabeli. Służy on do tworzenia powiązań między danymi, np. przypisania zamówienia do konkretnego użytkownika. Mechanizm kluczy obcych wymusza spójność relacyjną, zapobiegając wprowadzaniu odwołań do nieistniejących rekordów. MySQL obsługuje transakcje, indeksowanie oraz mechanizmy buforowania, co zapewnia bezpieczne i szybkie operacje odczytu oraz zapisu.

Backend aplikacji komunikuje się z bazą poprzez zapytania, pobierając lub modyfikując dane, a następnie przekazuje je do warstwy frontendowej za pośrednictwem interfejsu.

² Oracle, *MySQL Documentation*, dokumentacja systemu zarządzania bazą danych MySQL, [b.r.], <https://dev.mysql.com/doc/> (dostęp: 25.07.2026).

sów API. Takie podejście oddziela logikę biznesową od warstwy prezentacji oraz zwiększa bezpieczeństwo systemu. Do kluczowych funkcjonalności MySQL należą obsługa zapytań, takich jak “SELECT”, “INSERT”, “UPDATE”, “DELETE”, realizujących operacje na danych. Oprócz tego język ten dostarcza mechanizmy programistyczne takie jak transakcje - odpowiedzialne za wykonywanie kilku różnych operacji na bazie danych jednocześnie, układając je w jedną logiczną całość. Popularne jest też tworzenie widoków, czyli wirtualnych tabel, upraszczających złożone zapytania lub tworzenie procedur składowanych i wyzwalaczy, służących do automatyzowania operacji po stronie bazy danych. MySQL jednak przede wszystkim umożliwia sprawne zarządzanie użytkownikami, uprawnieniami oraz szyfrowaniem połączeń. Język ten dobrze pasuje do systemów, w których struktura danych jest z góry określona, a relacje między tabelami powinny być ściśle kontrolowane przez bazę. Przykładem mogą być aplikacje administracyjne, systemy rezerwacji lub sklepy internetowe, gdzie wiele operacji opiera się na jasno zdefiniowanych zależnościach między rekordami.

MySQL został opisany jako punkt odniesienia dla technologii finalnie wykorzystanej w projekcie, czyli bezrelacyjny typ bazy danych MongoDB. Porównanie obu rozwiązań pozwala wskazać różnicę między relacyjnym a dokumentowym modelem bazodanowym. W przypadku projektowanej aplikacji większe znaczenie miała elastyczność struktury danych, dlatego wybrano MongoDB, natomiast MySQL pozostaje przykładem rozwiązania właściwego dla systemów o bardziej stałych i silnie powiązanych relacjach.

4.2 MongoDB

MongoDB³ jest nierelacyjnym systemem zarządzania bazą danych, którego podstawową różnicą względem relacyjnych rozwiązań, jest sposób przechowywania informacji. Technologia ta wykorzystuje model dokumentowy, w którym dane zapisywane są w elastycznych strukturach przypominających format JSON⁴. Oznacza to brak sztywno zdefiniowanego schematu tabel oraz większą swobodę w organizacji informacji. W MongoDB powiązane informacje mogą być zapisywane w obrębie jednego dokumentu albo łączone przez identyfikatory innych dokumentów. W projekcie wykorzystano oba podejścia: część danych, na przykład elementy checklisty, jest zagnieżdżona, natomiast relacje między taskami, projektami, notatkami i przypięciami są realizowane przez identyfikatory.

Jako że system MongoDB zapisuje dane w kolekcjach składających się z dokumentów, każdy taki dokument może posiadać inną strukturę oraz zestaw pól, co pozwala na łatwe rozszerzanie modelu danych bez konieczności modyfikowania schematu całej bazy. Natomiast komunikacja z bazą odbywa się poprzez zapytania realizowane w formie obiektów lub metod API, zamiast klasycznych zapytań w języku SQL.

W projekcie aplikacji stworzonym na potrzeby niniejszej pracy inżynierskiej, klu-

³ MongoDB, Inc., *Welcome to the MongoDB Docs*, dokumentacja systemu MongoDB, [b.r.], <https://www.mongodb.com/docs/> (dostęp: 02.02.2026).

⁴ JSON - z jęz. ang. JavaScript Object Notation, tekstowy format zapisu i wymiany danych oparty na strukturze obiektów oraz tablic, wykorzystywany w komunikacji między aplikacjami.

czowe znaczenie ma łatwość rozbudowy funkcjonalności oraz możliwość obsługi dynamicznie zmieniających się struktur informacji. Z tego względu zdecydowano się na zastosowanie bazy dokumentowej MongoDB zamiast klasycznej relacyjnej bazy danych. W systemie zarządzania zadaniami pojedyncze obiekty, takie jak projekt lub zadanie, mogą posiadać zróżnicowane właściwości. Model dokumentowy MongoDB pozwala przechowywać takie dane bez konieczności tworzenia wielu tabel i relacji. Ułatwia to modyfikację struktury danych w trakcie rozwoju aplikacji, bez potrzeby przeprowadzania kosztownych migracji schematu. Korzystne dla rozwoju aplikacji jest także to, że dane przechowywane w Mongo nie mają wyłącznie tabelarycznego charakteru. Projekt może posiadać własne kolumny kanban, taski mogą zawierać różne zestawy powiązań, a checklista składa się z tablicy elementów. Model dokumentowy ułatwia zapis takich struktur bez projektowania wielu tabel pomocniczych. Wybrana technologia dobrze współpracuje również z aplikacją pisaną w TypeScriptie oraz Next.js, ponieważ dokumenty mogą być obsługiwane w kodzie jako obiekty. W projekcie dodatkową warstwą porządkującą strukturę danych są modele Mongoose⁵, opisujące pola dokumentów i pozwalające zachować kontrolę nad zapisem mimo elastycznego charakteru bazy.

⁵ Mongoose, *Schemas*, dokumentacja biblioteki Mongoose dotycząca schematów danych, [b.r.], <https://mongoosejs.com/docs/guide.html> (dostęp: 16.08.2026).

5. Architektura aplikacji

W aplikacji zaimplementowana została architektura klient-serwer, będąca jednym z podstawowych modeli projektowania stron internetowych. System podzielony jest w niej na dwie warstwy - klienta oraz serwera. Klient odpowiada za interakcję z użytkownikiem i prezentację danych, natomiast serwer realizuje logikę biznesową, przetwarzanie danych oraz komunikację z bazą danych.

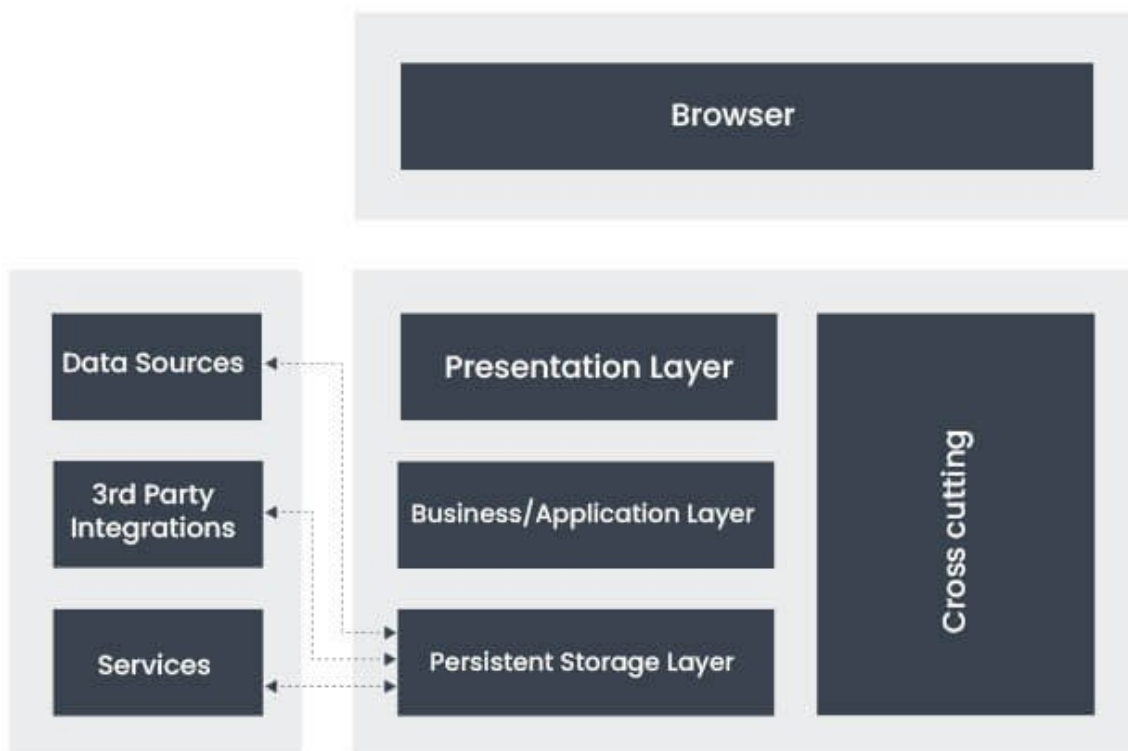
Komunikacja odbywa się w schemacie “żądanie/odpowiedź”. Klient wysyła do serwera zapytanie, np. o pobranie danych lub zapis informacji, a serwer przetwarza je i zwraca odpowiedź w postaci danych lub komunikatu o wyniku operacji. Takie podejście pozwala oddzielić warstwę prezentacji od warstwy przetwarzania danych, co zwiększa przejrzystość projektu oraz ułatwia jego rozwój.

W projekcie podział klient-serwer ma przede wszystkim znaczenie bezpieczeństwa i porządku odpowiedzialności. Przeglądarka prezentuje widoki oraz zbiera dane z formularzy, natomiast operacje zapisu i odczytu przechodzą przez warstwę serwerową. To tam sprawdzana jest sesja użytkownika, wykonywana jest walidacja i dopiero potem następuje komunikacja z bazą danych. Użytkownik nie otrzymuje więc bezpośredniego dostępu do kolekcji bazy ani do mechanizmów odpowiedzialnych za kontrolę właściciela danych.

Takie rozdzielenie ułatwia również rozwijanie projektu, ponieważ zmiana wyglądu formularza nie musi oznaczać zmiany modelu danych, a dopisanie kolejnego endpointu nie wymaga przebudowy całego interfejsu.

5.1 Omówienie warstw aplikacji

Aplikację internetową można zasadniczo podzielić na trzy warstwy: prezentacji (frontend), logiki aplikacji (backend) oraz danych. Na poniżej przedstawionym schemacie zaprezentowano klasyczną, trójwarstwową architekturę aplikacji internetowej. Poszczególne warstwy odpowiadają za różne obszary odpowiedzialności systemu, co zwiększa modularność, bezpieczeństwo oraz łatwość utrzymania oprogramowania.



Rys. 9. Diagram, przedstawiający warstwy aplikacji internetowej. Źródło: Albiorix Technology, <https://www.albiorixtech.com/wp-content/uploads/2025/12/web-application-architecture-layers.jpg> (dostęp: 03.02.2026).

Przeglądarka (*browser*) to środowisko uruchomieniowe po stronie klienta, w którym działa aplikacja frontendowa. Użytkownik korzysta z systemu poprzez przeglądarkę, która renderuje interfejs i wysyła żądania do serwera. Ponadto umożliwia wykonywanie kodu JavaScriptu, obsługuje interakcje z systemem, wyświetla interfejs użytkownika oraz może komunikować się z backendem za pomocą protokołów HTTP lub HTTPS¹.

Warstwa prezentacji odpowiada za interfejs użytkownika (UI) oraz sposób prezentacji danych. Wyświetla widoki formularzy, zbiera dane wejściowe, które potem mogą zostać przesłane w postaci zapytań do bazy danych oraz zajmuje się podstawową walidacją formularzy. Odpowiada również za wysyłanie zapytań do backendu.

Warstwa logiki biznesowej stanowi rdzeń aplikacji, odpowiadając za przetwarzanie danych. Realizuje operacje CRUD, dbając też o szczegółową walidację danych. Realizuje autentykację i autoryzację użytkowników.

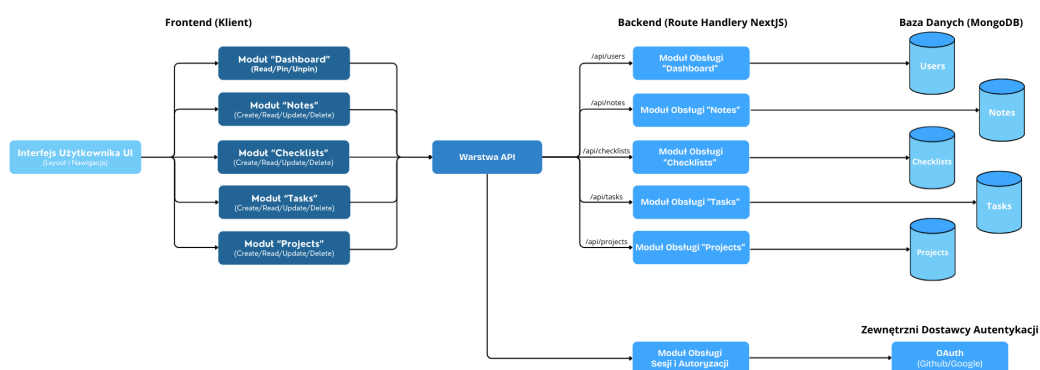
Za magazynowanie danych w sposób trwały odpowiada warstwa przechowywania danych (na rysunku opisana jako *persistent storage layer*), będąca zazwyczaj bazą danych. Ta część aplikacji odczytuje, przechowuje i zapewnia integralność danych.

¹ HTTP (HyperText Transfer Protocol) – protokół komunikacyjny warstwy aplikacji wykorzystywany do przesyłania danych pomiędzy klientem a serwerem w sieci WWW. Jego rozszerzona, bardziej bezpieczna wersja, wykorzystująca mechanizmy szyfrowania to protokół HTTPS (HyperText Transfer Protocol Secure). Źródło: J. Dean, *Web Programming with HTML5, CSS, and JavaScript*, Jones & Bartlett Learning, Burlington 2017, s. 6, 221.

Źródła zewnętrzne z kolei to systemy i usługi, z którymi aplikacja się integruje. Mogą to być zewnętrzne bazy lub pliki z danymi (*data sources*), integracje z systemami firm trzecich (*3rd party integrations*) lub dodatkowe mikroserwisy oraz API (na schemacie opisane jako *services*). Mechanizmy przekrojowe (na schemacie widniejące jako *cross-cutting*) to elementy działające na wszystkich warstwach. Nie należą one do konkretnej warstwy, lecz wpływają na cały system. Mogą obejmować na przykład mechanizmy bezpieczeństwa, logowania zdarzeń, obsługę błędów lub monitorowanie wydajności aplikacji.

5.2 Diagramy architektoniczne

Diagram komponentów przygotowano po to, aby pokazać najważniejsze części systemu i kierunek komunikacji między nimi. Schemat obejmuje interfejs użytkownika, warstwę API, logikę serwerową, bazę danych oraz zewnętrznych dostawców autentykacji. Strzałki wskazują, które elementy przekazują sobie dane podczas typowej operacji wykonywanej przez użytkownika.



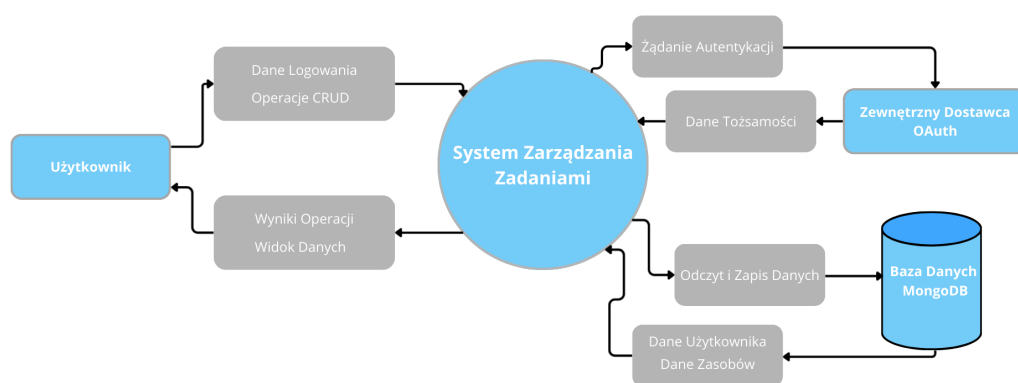
Rys. 10. Prosty diagram komponentów internetowej aplikacji do zarządzania zadaniami.

Przepływ danych można rozpisać na poszczególne kroki:

1. Użytkownik wykonuje akcję w interfejsie.
2. Frontend wysyła żądanie do API.
3. Backend przetwarza logikę, sprawdzając również uprawnienia.
4. Dane są pobierane lub zapisywane w bazie.
5. Odpowiedź wraca do interfejsu użytkownika (UI).

Najważniejszą ideą architektoniczną jest przede wszystkim separacja warstw, zapewniająca oprócz uporządkowanej struktury, bezpieczeństwo przez brak bezpośredniego dostępu klienta do bazy danych. Oprócz niej takie rozwiązanie gwarantuje modularność, wynikającą z osobnych serwisów dla każdego typu zasobu oraz ułatwia skalowalność aplikacji.

Kolejne diagramy, powstałe na etapie projektowania architektury, przygotowano z wykorzystaniem informacji z artykułu Canva: DFD². Pokazują one, jak dane przemieszczają się w systemie między źródłami, procesami oraz magazynami danych. Są to tak zwane diagramy przepływu danych.



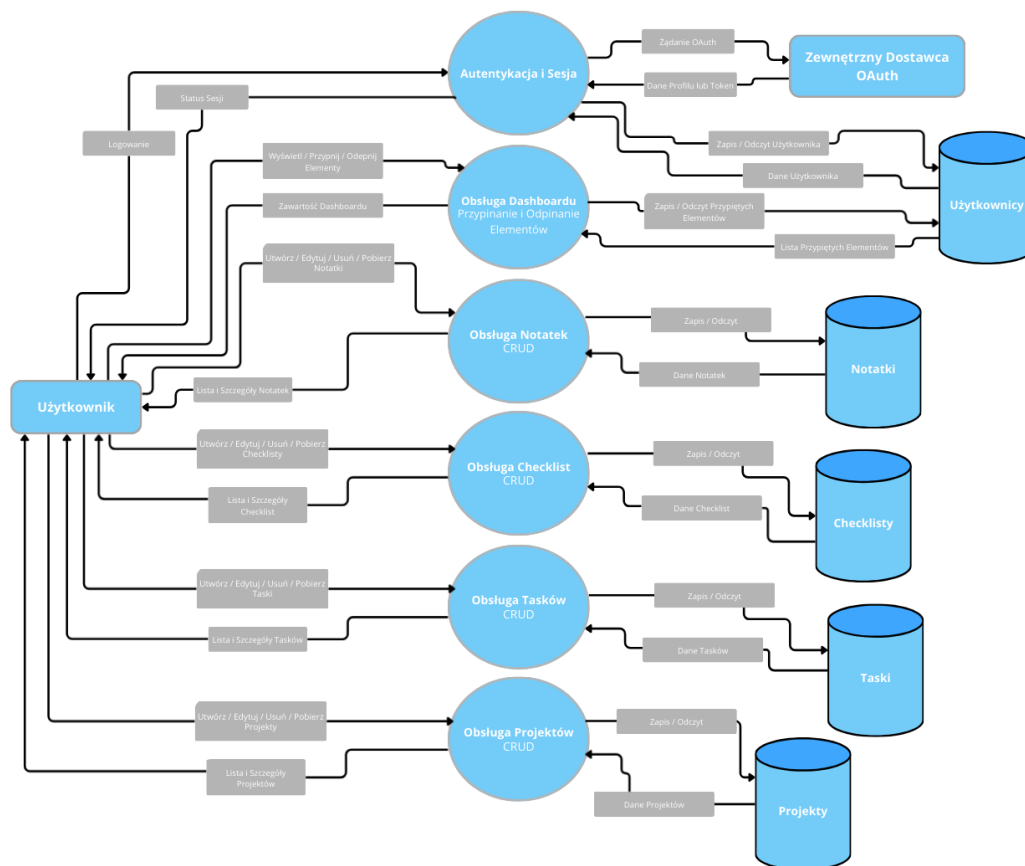
Rys. 11. Diagram przepływu danych poziomu 0, przedstawiający system jako jeden proces wraz z relacjami z otoczeniem.

Diagram przepływu danych poziomu 0 przedstawia aplikację do zarządzania zadaniami jako pojedynczy, logiczny proces, przetwarzający dane pomiędzy użytkownikiem, zewnętrznym dostawcą autentykacji oraz bazą danych. Użytkownik inicjuje operacje takie jak logowanie oraz zarządzanie zadaniami, projektami, notatkami i checklistami, natomiast system przetwarza te żądania, komunikuje się z bazą danych oraz zwraca odpowiednie wyniki. Diagram ten obrazuje granice systemu oraz jego relacje z otoczeniem, pomijając szczegóły wewnętrznej struktury aplikacji, które przedstawiane są na kolejnym rysunku.

Diagram przepływu danych poziomu 1 przedstawia szczegółową dekompozycję systemu na główne procesy funkcjonalne oraz pokazuje sposób wymiany informacji pomiędzy użytkownikiem, modułami aplikacji, bazą danych i zewnętrznym dostawcą autentykacji. Użytkownik inicjuje operacje takie jak logowanie, przeglądanie oraz zarządzanie zadaniami, projektami, notatkami i checklistami, a odpowiednie procesy systemowe realizują logikę biznesową i wykonują operacje zapisu lub odczytu danych w dedykowanych kolekcjach bazy

² Canva, *Creating a Data Flow Diagram: How-tos, Templates, and Tips*, artykuł dotyczący tworzenia diagramów przepływu danych, [b.r.], <https://www.canva.com/online-whiteboard/data-flow-diagrams/> (dostęp: 03.02.2026).

danych. Dodatkowo wydzielony został proces autentykacji i zarządzania sesją, który komunikuje się z dostawcą OAuth w celu weryfikacji tożsamości użytkownika. Diagram ilustruje rozdzielenie odpowiedzialności pomiędzy warstwą logiki aplikacji i warstwą przechowywania danych.



Rys. 12. Diagram przepływu danych poziomy 1

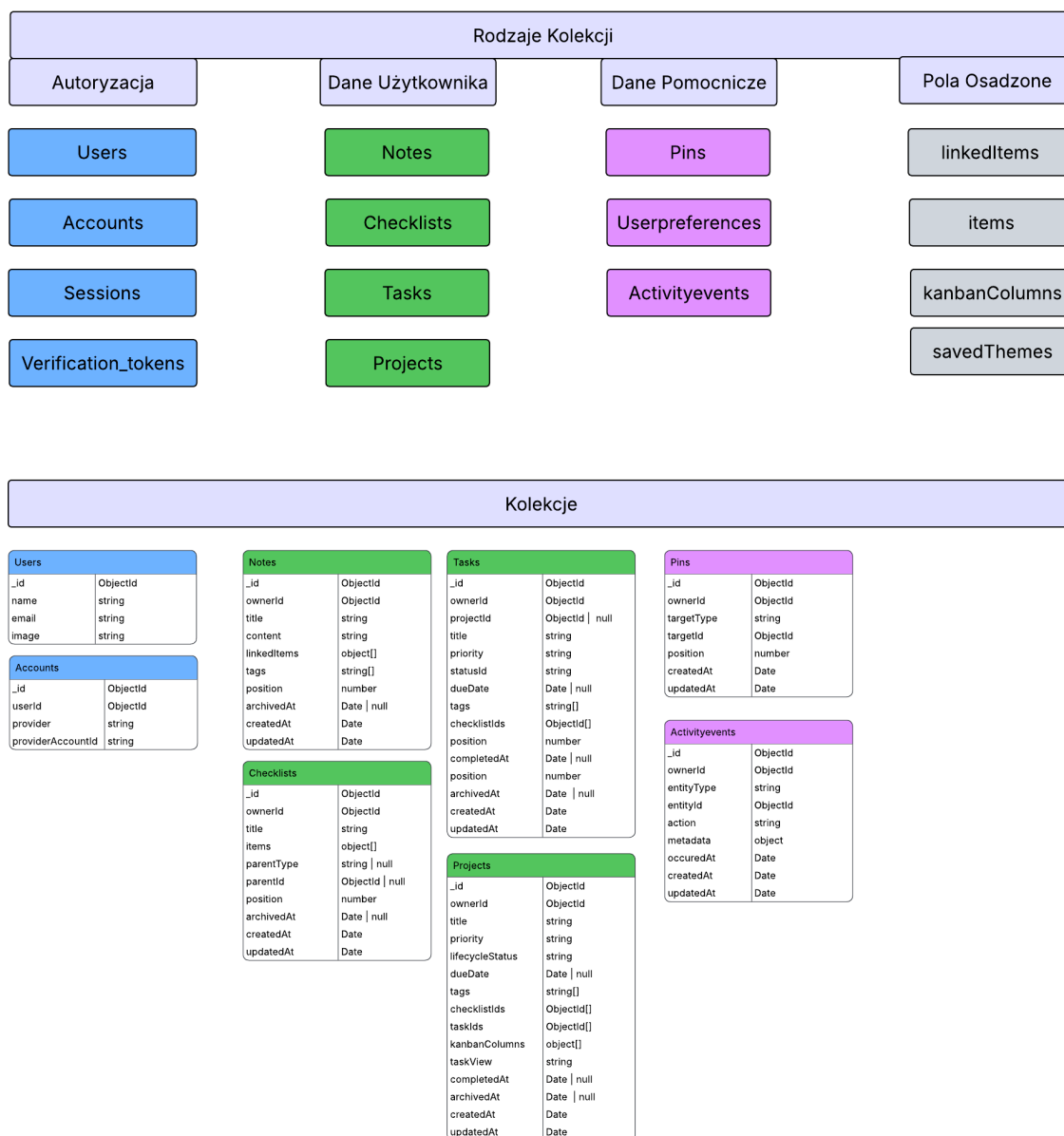
5.3 Struktura bazy danych aplikacji

W aplikacji zastosowano bazę danych MongoDB oraz bibliotekę Mongoose, która została wykorzystana do opisanie struktury dokumentów domenowych. Połączenie z bazą jest realizowane przez moduł odpowiedzialny za utrzymywanie współdzielonego połączenia Mongoose, natomiast mechanizm logowania OAuth korzysta dodatkowo z adaptera MongoDB dla NextAuth³. Dzięki temu dane logowania oraz dane aplikacyjne są przechowywane w tej samej bazie, ale obsługiwane przez odrębne warstwy aplikacji.

Podstawową zasadą modelu danych jest przypisanie dokumentów do konkretnego użytkownika. Notatki, checklisty, zadania, projekty, przypięcia oraz zdarzenia aktywności zawierają pole "ownerId", które wskazuje właściciela dokumentu. Preferencje interfejsu użyt-

³ NextAuth to biblioteka wykorzystywana w aplikacjach Next.js do obsługi logowania, sesji użytkownika oraz integracji z zewnętrznymi dostawcami tożsamości. Projekt NextAuth.js jest obecnie rozwijany w ramach szerszego ekosystemu Auth.js. Źródło: Auth.js, *Auth.js Core Reference*, dokumentacja callbacków biblioteki Auth.js, [b.r.], <https://authjs.dev/reference/core#callbacks> (dostęp: 16.08.2026).

kownika korzystają z pola “userId”. Każda operacja pobierania, aktualizacji lub archiwizacji danych wykonywana jest z uwzględnieniem identyfikatora aktualnie zalogowanego użytkownika, co zabezpiecza aplikację przed dostępem do cudzych zasobów.



Rys. 13. Schemat struktury bazy danych aplikacji.

Na rysunku powyżej przedstawiono graficzny schemat struktury bazy danych aplikacji. Schemat wykorzystuje notację dokumentową zgodną ze sposobem modelowania danych w MongoDB i Mongoose, dlatego nie należy interpretować go jako klasycznego relacyjnego diagramu ERD opartego na tabelach, kluczach głównych i kluczach obcych. Diagram został podzielony na cztery grupy kolekcji: autoryzacyjne, przechowujące dane użytkownika, pomocnicze oraz pola osadzone. Taki podział pokazuje, że dane odpowiedzialne za logowanie użytkownika są odseparowane od danych domenowych aplikacji, takich jak zadania, projekty, checklisty i notatki.

W schemacie zastosowano oznaczenie “ObjectId”, które oznacza specjalny typ identyfikatora używany w MongoDB. Każdy dokument zapisany w kolekcji otrzymuje pole “_id” tego typu, a inne pola, takie jak “ownerId”, “projectId”, “targetId” albo “userId”, mogą przechowywać identyfikator innego dokumentu. Dzięki temu aplikacja może wskazywać powiązania pomiędzy dokumentami bez stosowania klasycznych kluczy obcych znanych z relacyjnych baz danych.

Typ “Date” zapisany jest wielką literą, ponieważ w kontekście MongoDB, Mongoose oraz JavaScriptu jest to nazwa specjalnego typu obiektowego reprezentującego datę i czas. Zapis ma więc charakter techniczny i odpowiada nazwie typu, a nie zwykłemu słowu opisowemu. Z tego samego powodu na diagramie widoczne są zapisy takie jak “Date | null”, które oznaczają, że pole może zawierać datę albo wartość pustą. Przykładem jest “archivedAt”, które pozostaje puste dla aktywnego dokumentu, a po archiwizacji otrzymuje datę wykonania tej operacji.

Zapisy z nawiasami kwadratowymi, takie jak “string[]”, “ObjectId[]” albo “object[]”, oznaczają tablice wartości danego typu. Przykładowo “string[]” oznacza tablicę tekstów, czyli wiele wartości typu “string”. Pole “tags” może więc przechowywać kilka tagów o typie danych string, pole “checkboxListIds” może przechowywać wiele identyfikatorów checkbox, a pole “kanbanColumns” zawiera tablicę obiektów opisujących kolumny tablicy kanban.

Warstwa autoryzacji tworzy cztery kolekcje. Są to “users”, “accounts”, “sessions” oraz “verification_tokens”. Najważniejsze z punktu widzenia działania aplikacji są kolekcje “users” i “accounts”, ponieważ przechowują użytkowników oraz powiązania z zewnętrznymi dostawcami logowania, takimi jak Google i GitHub. Sesja użytkownika jest obsługiwana z użyciem strategii JWT⁴, jednak identyfikator użytkownika pochodzący z NextAuth stanowi podstawę powiązań z dokumentami domenowymi.

Kolekcja “accounts” pełni funkcję technicznego powiązania pomiędzy użytkownikiem zapisanym w kolekcji “users” a kontem pochodzącym od zewnętrznego dostawcy logowania. Pole “userId” wskazuje dokument użytkownika, natomiast pola “provider” i “providerAccountId” określają, z jakiego dostawcy OAuth pochodzi dane konto oraz jaki identyfikator użytkownik posiada u tego dostawcy. Dzięki temu jedna osoba może być poprawnie rozpoznawana przez system po zalogowaniu z użyciem zewnętrznej usługi.

Główne dane aplikacji zostały podzielone na kolekcje “notes”, “checkboxlists”, “tasks”, “projects”, “pins”, “userpreferences” oraz “activityevents”. Kolekcja “notes” przechowuje notatki użytkownika, zawierające tytuł, treść, tagi, pozycję sortowania oraz listę powiązanych elementów. Powiązania notatki są przechowywane w tablicy “linkedItems”, gdzie każdy wpis składa się z typu obiektu oraz jego identyfikatora. Dzięki temu notatka może odnosić się do innej notatki, checkboxlisty, zadania albo projektu.

Checkboxlisty są przechowywane w kolekcji “checkboxlists”. Każda checkboxlista posiada tytuł, pozycję, opcjonalne powiązanie z rodzicem oraz tablicę elementów. Elementy checkboxlisty zostały osadzone bezpośrednio w dokumencie checkboxlisty, ponieważ są odczytywane i aktu-

⁴ JWT - z jęz. ang. JSON Web Token, czyli tekstowy token służący do bezpiecznego przekazywania informacji między systemami w postaci podpisanego obiektu JSON.

alizowane razem z nią. Pojedynczy element zawiera nazwę, informację o ukończeniu, datę ukończenia oraz pozycję. Checklista może funkcjonować samodzielnie albo być powiązana z zadaniem lub projektem przez pola “parentType” i “parentId”.

Zadania są zapisywane w kolekcji “tasks”. Dokument zadania zawiera tytuł, opis, priorytet, status, termin wykonania, tagi, pozycję, datę ukończenia oraz datę archiwizacji. Zadanie może być samodzielnym elementem listy zadań albo częścią projektu. W drugim przypadku pole “projectId” wskazuje projekt, do którego należy zadanie. Status zadania jest przechowywany w polu “statusId”. W widoku tablicy kanban odpowiada on identyfikatorowi jednej z kolumn projektu. Zadanie może mieć również powiązane checklisty, których identyfikatory są przechowywane w tablicy “checklistIds”.

Projekty są przechowywane w kolekcji “projects”. Dokument projektu zawiera tytuł, opis, priorytet, status cyklu życia, termin, tagi, pozycję, listę zadań, checklisty oraz konfigurację widoku zadań. W projekcie przechowywana jest tablica “kanbanColumns”, opisująca kolumny tablicy kanban. Każda kolumna ma identyfikator, nazwę, kolor, pozycję oraz informację, czy jest to stan zakończony. Domyślny zestaw kolumn obejmuje “backlog”, “todo”, “in_progress”, “testing” oraz “done”. Pole “taskView” określa, czy zadania projektu są prezentowane jako lista, czy jako tablica kanban.

Relacja pomiędzy projektami i zadaniami jest utrzymywana dwukierunkowo na poziomie aplikacji. Główne powiązanie znajduje się w zadaniu, w polu “projectId”. Projekt przechowuje dodatkowo tablicę “taskIds”, która zawiera identyfikatory powiązanych zadań. Podczas tworzenia zadania z przypisanym projektem backend sprawdza, czy projekt istnieje, nie jest zarchiwizowany i należy do aktualnego użytkownika. Następnie identyfikator zadania jest dopisywany do projektu. Przy zmianie projektu lub archiwizacji zadania identyfikator jest usuwany ze starego projektu.

Kolekcja “pins” przechowuje elementy przypięte na ekranie głównym aplikacji. Dokument przypięcia zawiera typ wskazywanego obiektu, jego identyfikator oraz pozycję na dashboardzie. Dzięki parze pól “targetType” i “targetId” możliwe jest przypięcie notatki, checklisty, zadania albo projektu bez tworzenia osobnych kolekcji dla każdego typu przypięcia. W modelu zastosowano unikalny indeks obejmujący właściciela, typ celu i identyfikator celu, co zapobiega wielokrotnemu przypięciu tego samego elementu przez jednego użytkownika.

Preferencje interfejsu zapisano w kolekcji “userpreferences”. Dokument preferencji zawiera tryb kolorystyczny, układ dashboardu, zestaw kolorów używanych w aplikacji oraz zapisane własne motywy. Pole “userId” jest unikalne i pozwala każdemu użytkownikowi posiadać tylko jeden dokument preferencji. Takie rozwiązanie upraszcza pobieranie ustawień podczas renderowania interfejsu.

Kolekcja “activityevents” przechowuje zdarzenia aktywności. Każde zdarzenie zawiera typ encji, identyfikator dokumentu, rodzaj akcji oraz czas wystąpienia. Rejestrowane są między innymi operacje utworzenia, aktualizacji, usunięcia, przeniesienia oraz przypięcia elementu. Endpoint odpowiedzialny za komunikację w czasie rzeczywistym odczytuje te zdarzenia i przekazuje je klientowi, co pozwala odświeżać widoki po zmianach danych.

Najważniejsze relacje logiczne w bazie danych można przedstawić następująco:

- użytkownik jest właścicielem notatek, checklist, zadań, projektów, przypięć i zdarzeń aktywności,
- użytkownik posiada jeden dokument preferencji interfejsu,
- zadanie może należeć do projektu przez pole “projectId”,
- projekt przechowuje pomocniczą listę zadań w tablicy “taskIds”,
- zadanie i projekt mogą mieć powiązane checklists,
- notatka może wskazywać inne elementy aplikacji przez tablicę “linkedItems”,
- przypięcie wskazuje dowolny obsługiwany typ elementu przez pola “targetType” i “targetId”.

W bazie zastosowano indeksy wspierające najczęstsze operacje aplikacji. Większość kolekcji posiada indeksowanie po polu właściciela, co przyspiesza pobieranie danych aktualnego użytkownika. Dodatkowe indeksy obejmują między innymi pozycję sortowania, tagi, termin wykonania, priorytet, status zadania oraz pola wykorzystywane do przypięć. Dzięki temu aplikacja może sprawnie filtrować i sortować elementy dashboardu oraz szybko sprawdzać, czy któryś z nich nie został już przypięty.

Usuwanie większości elementów domenowych zostało zrealizowane jako archiwizacja. Zadania, projekty i checklists otrzymują wartość w polu “archivedAt”, a zapytania listujące pobierają wyłącznie dokumenty aktywne. W przypadku projektu podczas archiwizacji ustawiany jest również status cyklu życia “archived”. Takie podejście pozwala zachować spójność powiązań i historię zmian bez fizycznego usuwania dokumentów.

6. Projekt interfejsu użytkownika

Projekt interfejsu użytkownika podporządkowano prostemu przepływowi pracy. Najpierw rozpisano scenariusze przypadków użycia, a dopiero później przygotowano widoki, które odpowiadają tym czynnościom. Główną uwagę podczas tworzenia interfejsu graficznego skupiono na umożliwieniu użytkownikowi szybkiego przejścia od strony głównej do taska, checklisty, notatki albo projektu.

6.1 Scenariusze przypadków użycia

1) Przypięcie elementu do strony głównej.

Aktor: Użytkownik indywidualny.

Cel: Szybki dostęp do śledzonych tasków, notatek, checklist i projektów z poziomu strony głównej.

Warunki wstępne: Użytkownik jest zalogowany; element istnieje.

Scenariusz podstawowy:

1. Użytkownik otwiera jedną z list elementów, na przykład tasków, checklist lub projektów.
2. Użytkownik wybiera konkretny element.
3. Użytkownik widzi, że element nie jest przypięty, ponieważ ikona pinezki jest pusta w środku.
4. Użytkownik klika ikonę i dodaje element do przypiętych.
5. System oznacza element jako „przypięty” poprzez zmianę ikony pinezki na wypełnioną.
6. System wyświetla element w sekcji “przypięte” na stronie głównej.

Scenariusze alternatywne:

1. **Element już jest przypięty.**
 - 3a. System informuje użytkownika, że element jest już przypięty, poprzez wypełnioną ikonę pinezki.
 - 4a. Użytkownik może kliknąć ikonę w celu odpięcia elementu od ekranu głównego.

Warunki końcowe: Element jest widoczny w sekcji „przypięte” na stronie głównej.

2) Utworzenie nowego taska z parametrami.

Aktor: Pracownik biurowy.

Cel: Uporządkowanie pracy poprzez utworzenie taska z właściwościami.

Warunki wstępne: Użytkownik jest zalogowany.

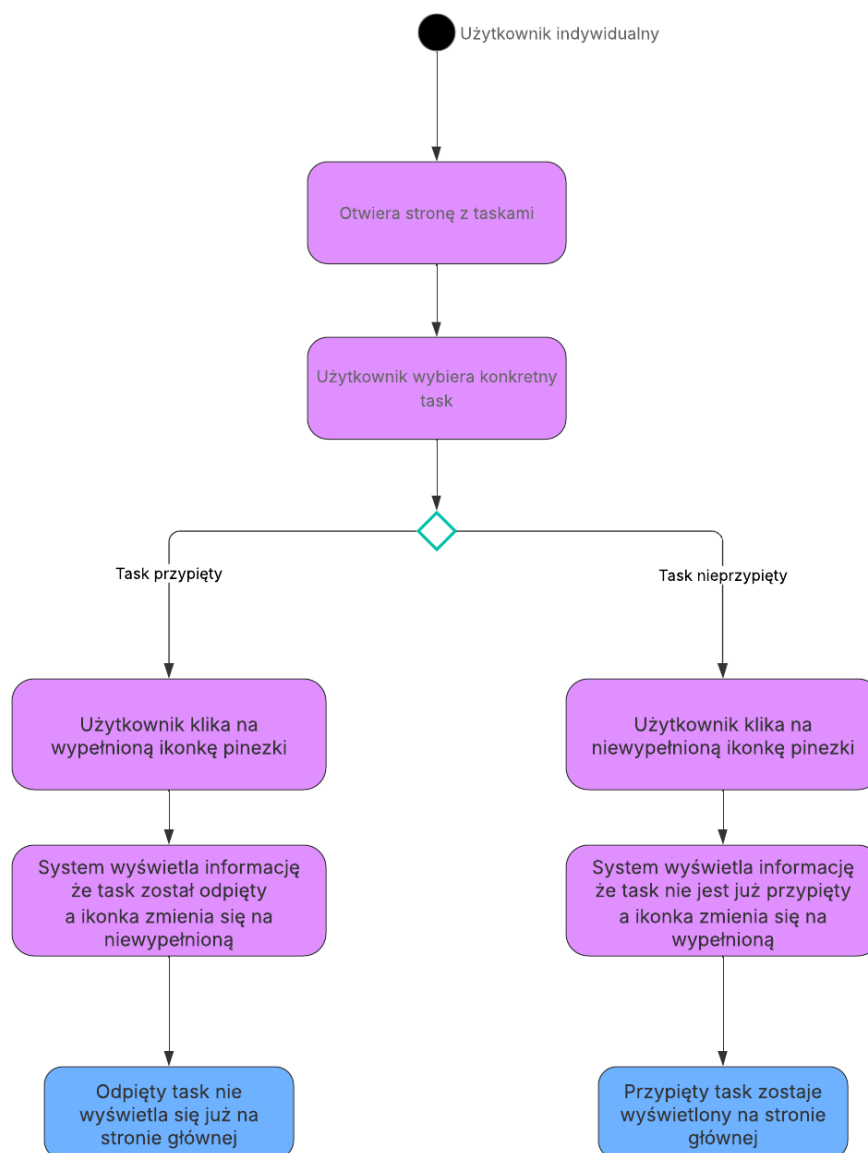
Scenariusz podstawowy:

1. Użytkownik wybiera z paska bocznego zakładkę “zadania”.
2. Użytkownik zostaje przekierowany na stronę z taskami.
3. Użytkownik wybiera ikonę z plusem, znajdującą się obok istniejących tasków.
4. System otwiera formularz tworzenia taska.
5. Użytkownik wpisuje nazwę taska.
6. Użytkownik uzupełnia opis taska.
7. Użytkownik wybiera priorytet z rozwijanej listy wyboru.
8. Użytkownik ustawia datę realizacji taska.
9. Użytkownik zapisuje task.
10. System tworzy task i wyświetla go na liście tasków.

Scenariusze alternatywne:

1. **Brak opisu.**
 - 6a. Użytkownik pomija opis; system zapisuje task bez opisu.
2. **Brak wymaganych danych (np. nazwy).**
 - 5a. Użytkownik pomija nazwę taska.
 - 9a. System blokuje zapis i wskazuje brakujące pola.
3. **Niepoprawna data realizacji.**
 - 8a. Użytkownik ustawia datę z przeszłości.
 - 9a. System blokuje zapis i wskazuje błędną datę.

Warunki końcowe: Dodany task wraz z opisem, priorytetem i datą realizacji pojawia się na liście tasków.



Rys. 14. *Diagram UML, wizualizujący scenariusz przypięcia taska*

Na diagramie zaprezentowano sekwencję działań wykonywanych podczas przypinania taska do strony głównej aplikacji. Widoczny jest podział odpowiedzialności pomiędzy użytkownika, interfejs aplikacji oraz system zapisujący zmianę. Po wybraniu elementu użytkownik otrzymuje informację o aktualnym stanie przypięcia, a następnie może zmienić ten stan za pomocą ikony pinezki.

3) Utworzenie checklisty zakupów z możliwością odhaczania pozycji.

Aktor: Głowa rodziny.

Cel: Nie zapomnieć produktów w sklepie.

Warunki wstępne: Użytkownik jest zalogowany.

Scenariusz podstawowy:

1. Użytkownik przechodzi do strony “szybkie notatki”.

2. Użytkownik wybiera ikonę z plusem, znajdującą się obok istniejących elementów.
3. Użytkownik wybiera, czy chce dodać notatkę, czy checklistę.
4. System otwiera formularz tworzenia checklisty.
5. Użytkownik ustawia nazwę checklisty.
6. Użytkownik dodaje pozycje w checkliście (np. mleko, chleb, warzywa).
7. System zapisuje checklistę i wyświetla ją użytkownikowi.
8. W sklepie użytkownik otwiera checklistę.
9. Użytkownik odhacza kolejne pozycje.
10. System zapisuje, które przedmioty w checkliście zostały odhaczone.

Scenariusze alternatywne:

1. **Dodanie pozycji w trakcie zakupów.**
 - 7a. Użytkownik dodaje nową pozycję.
 - 8a. System aktualizuje checklistę.

Warunki końcowe: Checklist zakupów istnieje, a pozycje mogą być odhaczane i zapisywane.

4) Tworzenie notatki.

Aktor: Pracownik nadzorujący dostawy w restauracji.

Cel: Utworzenie notatki zawierającej informacje o przebiegu dostawy oraz liczbie palet zwróconych przez restaurację.

Warunki wstępne: Użytkownik jest zalogowany w systemie; użytkownik ma dostęp do sekcji notatek.

Scenariusz podstawowy:

1. Użytkownik przechodzi do strony z notatkami.
2. System wyświetla listę istniejących notatek.
3. Użytkownik wybiera przycisk dodania nowej notatki.
4. System otwiera formularz tworzenia notatki.
5. Użytkownik wpisuje tytuł notatki, np. „Dostawa - restauracja”.
6. Użytkownik wpisuje treść notatki, zawierającą informacje o przebiegu dostawy.
7. Użytkownik dopisuje informację o liczbie palet zwróconych przez restaurację.

8. Użytkownik opcjonalnie dodaje tagi porządkujące notatkę, np. „dostawa” lub „restauracja”.
9. Użytkownik zapisuje notatkę.
10. System tworzy notatkę i zapisuje ją w bazie danych.
11. System wyświetla utworzoną notatkę na liście notatek.

Scenariusze alternatywne:

1. **Brak treści notatki.**

- 6a. Użytkownik pomija wpisanie treści notatki.
- 9a. System zapisuje notatkę z samym tytułem.

2. **Brak wymaganej nazwy notatki.**

- 5a. Użytkownik nie wpisuje tytułu notatki.
- 9a. System blokuje zapis i informuje użytkownika o konieczności podania tytułu.

3. **Dodanie powiązania z innym elementem.**

- 8a. Użytkownik wybiera element aplikacji, z którym notatka ma zostać powiązana, np. task, projekt lub checklista.
- 9a. System zapisuje notatkę wraz z informacją o powiązanym elemencie.

4. **Anulowanie tworzenia notatki.**

- 4a. System wyświetla formularz tworzenia notatki.
- 5a. Użytkownik rezygnuje z dodawania notatki i wraca do listy notatek.
- 6a. System nie zapisuje żadnych danych i ponownie wyświetla listę notatek.

Warunki końcowe: Notatka zostaje utworzona i jest widoczna na liście notatek. Użytkownik może później otworzyć jej podgląd, edytować treść, dodać tagi lub powiązać ją z innymi elementami aplikacji.

5) Utworzenie projektu i zarządzanie taskami w widoku tablicy.

Aktor: Student.

Cel: Organizacja pisania pracy dyplomowej poprzez projekt i powiązane taski.

Warunki wstępne: Użytkownik jest zalogowany.

Scenariusz podstawowy:

1. Użytkownik przechodzi do strony z projektami.
2. Użytkownik wybiera ikonę z plusem, znajdującą się obok istniejących projektów.
3. System otwiera formularz tworzenia projektu.
4. Użytkownik ustawia nazwę projektu.

5. Użytkownik ustawia datę realizacji projektu.
6. Użytkownik dodaje opis projektu.
7. Użytkownik chce dodać taski do projektu.
8. System otwiera użytkownikowi formularz dodawania tasków.
9. Użytkownik wybiera, czy dodać istniejący już task, czy stworzyć nowy.
10. Użytkownik dodaje kilka tasków do projektu.
11. Użytkownik ustawia statusy poszczególnym taskom.
12. Użytkownik zapisuje projekt.
13. Użytkownik uruchamia stronę projektu i przegląda jego opis.
14. Użytkownik przełącza widok tasków z listy na tablicę.
15. System wyświetla kolumny statusów oraz karty tasków.
16. Użytkownik przeciąga karty między kolumnami, zmieniając w ten sposób ich status.
17. System zapisuje zmiany statusów.

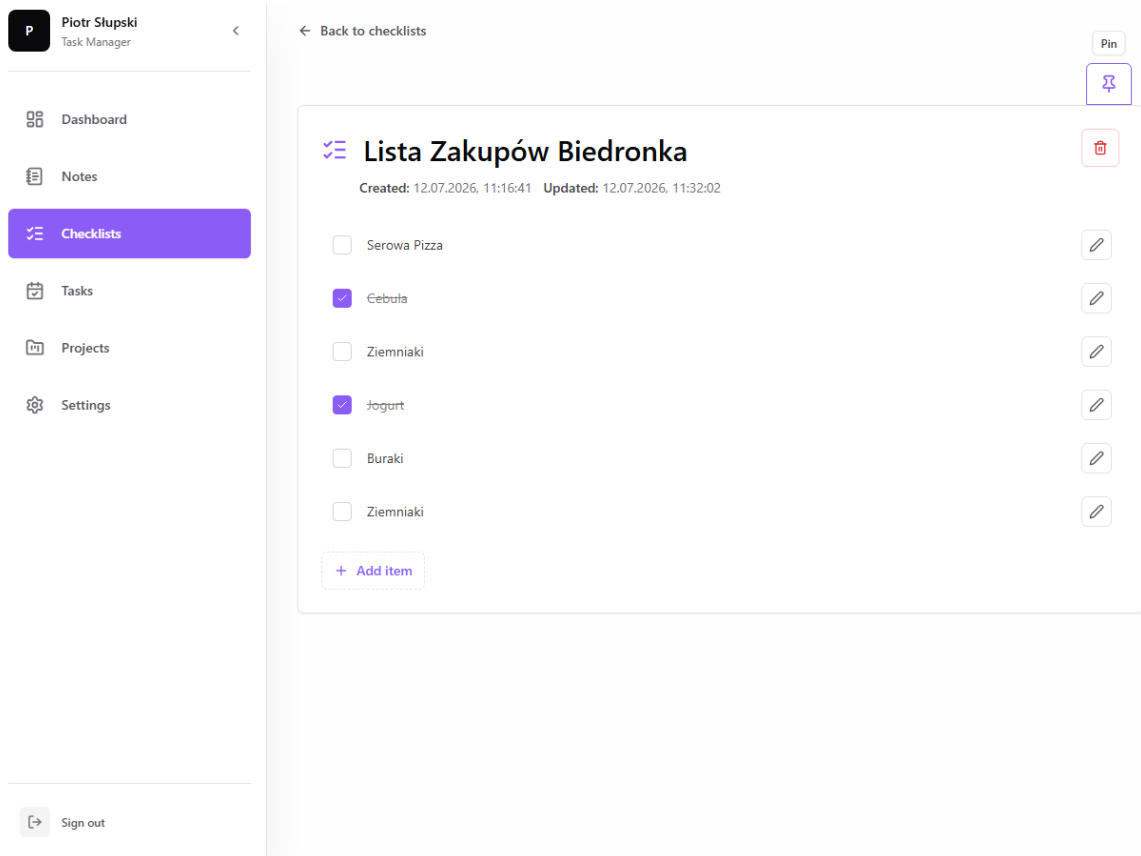
Scenariusze alternatywne:

1. **Brak daty realizacji.**
 - 6a. Użytkownik pomija wypełnianie daty realizacji.
 - 12a. System zapisuje projekt bez daty realizacji.
2. **Brak wymaganych danych (np. nazwy).**
 - 4a. Użytkownik pomija nazwę projektu.
 - 12a. System blokuje zapis i wskazuje brakujące pola.
3. **Niepoprawna data realizacji.**
 - 5a. Użytkownik ustawia datę z przeszłości.
 - 12a. System blokuje zapis i wskazuje błędną datę.

Warunki końcowe: Projekt istnieje z opisem, datą i innymi wypełnionymi parametrami, a system może wyświetlać taski zarówno w formie listy, jak i tablicy.

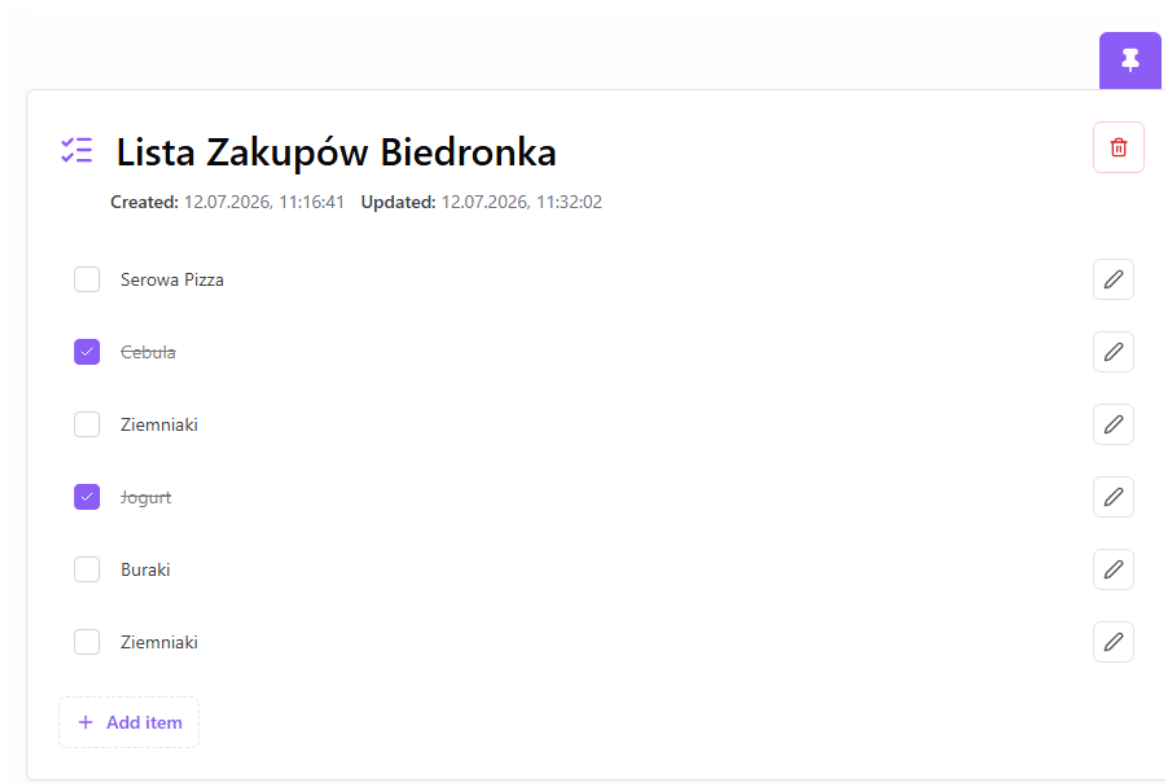
6.2 Widoki użytkownika aplikacji

W podrozdziale pokazano widoki odpowiadające wybranym scenariuszom przypadków użycia. Są to ekrany związane z przypięciem checklisty do strony głównej, utworzeniem taska oraz przygotowaniem projektu, którego zadania można oglądać jako tablicę kanban albo listę. Przy opisie zwrócono uwagę przede wszystkim na rozmieszczenie elementów, czytelność stanu interfejsu i informacje, które użytkownik otrzymuje bez konieczności przechodzenia do innych miejsc aplikacji.



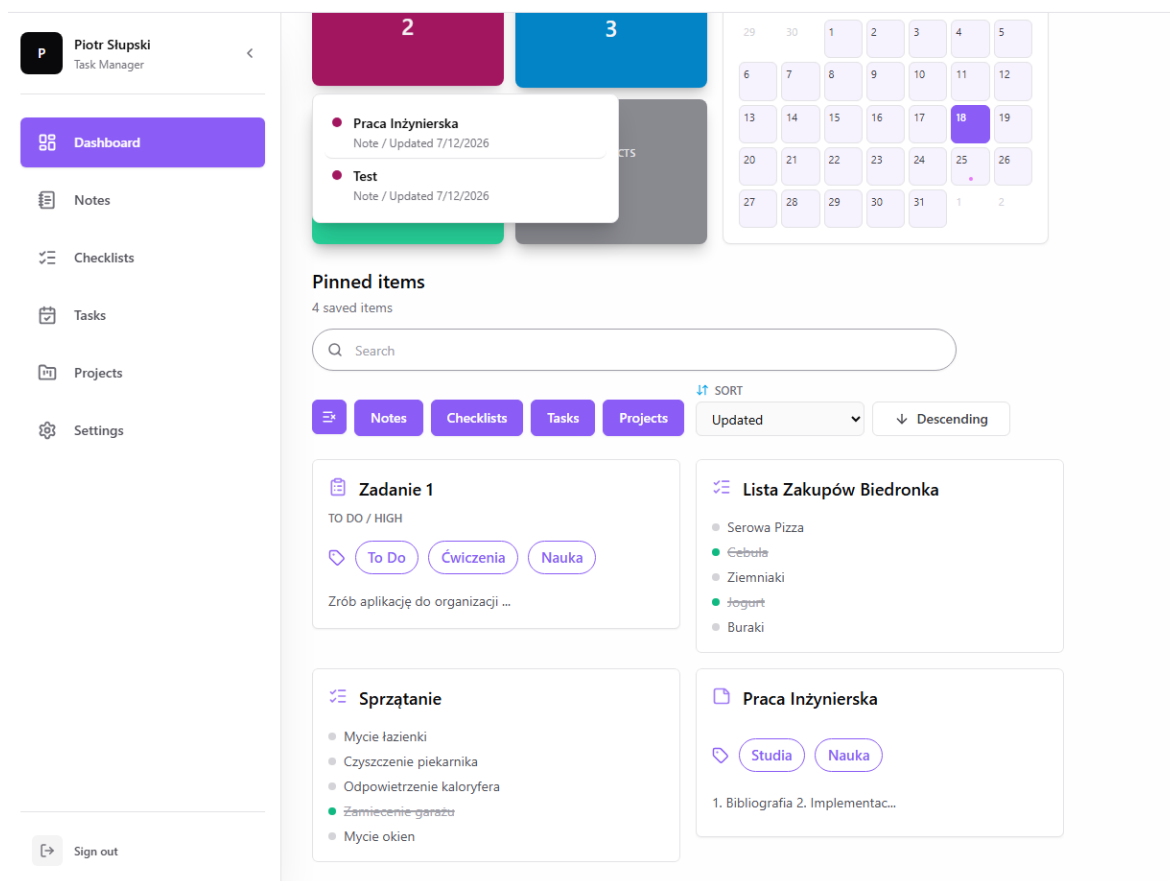
Rys. 15. Widok, przedstawiający checklistę, z możliwością przypięcia jej do ekranu głównego.

Pierwszy ekran scenariusza pokazuje szczegóły checklisty. Najważniejsze miejsce zajmuje panel z tytułem, datami utworzenia i aktualizacji oraz pozycjami do odhaczenia. Po prawej stronie umieszczono przyciski akcji, w tym ikonę pinezki służącą do przypięcia checklisty do ekranu głównego. Wykonane elementy są wyróżnione kolorem oraz przekreśleniem, dlatego postęp można ocenić bez dodatkowego otwierania szczegółów.



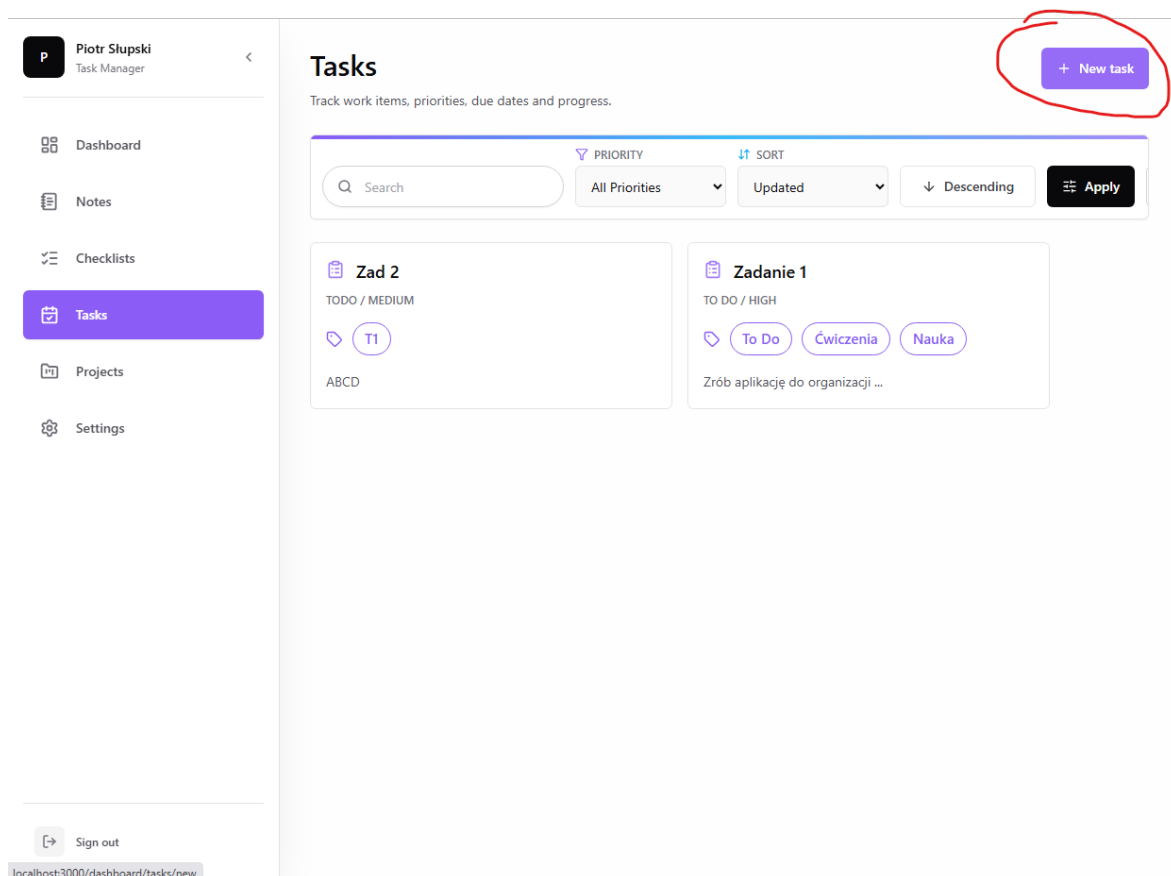
Rys. 16. Widok przedstawiający zachowanie przycisku “Przypnij” po przypięciu checklisty do ekranu głównego.

Po wykonaniu akcji przypięcia zmienia się wygląd przycisku widocznego przy checklistie. Ikona pinezki zostaje wypełniona kolorem, dzięki czemu interfejs natychmiast informuje, że checklista jest już przypięta. Takie rozwiązanie ogranicza potrzebę dodatkowych komunikatów tekstowych, ponieważ sam stan przycisku staje się informacją zwrotną.



Rys. 17. Widok przedstawiający elementy przypięte do ekranu głównego.

Sekcja przypiętych elementów na dashboardzie została zorganizowana w formie kart. Elementy przedstawiane są użytkownikowi z ikonami typów, tytułami, tagami oraz skróconym podglądem zawartości. Nad kartami znajduje się menu wyszukiwania, filtrowania i sortowania, dzięki czemu użytkownik może szybko odnaleźć przypięty element bez przechodzenia do osobnych modułów aplikacji.



Rys. 18. Widok, przedstawiający stronę z taskami użytkownika.

Główna strona tasków użytkownika wykorzystuje stały układ z boczną nawigacją i obszarem roboczym. Lewy panel nawigacyjny wskazuje aktywną sekcję aplikacji, a w części roboczej widoczne są nagłówki, opis modułu, pasek filtrowania i sortowania oraz karty tasków. Przycisk utworzenia nowego taska został wyróżniony kolorem akcentu i umieszczony w prawym górnym rogu, co ułatwia rozpoczęcie scenariusza dodawania zadania.

Piotr Siłowski
Task Manager

Dashboard
Notes
Checklists
Tasks
Projects
Settings

← Back to tasks

New task

Create a task with priority, status and optional project links.

Title
Testowe Zadanie

Description
Zadanie testowe

Priority
Low

Status
▼ Do zrobienia

Project
No Project

Due date
▼ 15.07.2026

Tags
Testy Aplikacji × + Add tag

Checklists + Add checklist

☐ Lista Zakupów Biedronka ☐ Sprzątanie

☐ Test 2

Linked notes

☐ Praca Inżynierska ☐ Test

Create task

Sign out

Rys. 19. Widok, przedstawiający formularz kreacji taska.

Formularz tworzenia taska uporządkowano według ważności uzupełnianych informacji. Najwyżej umieszczono pola tytułu i opisu, ponieważ są one najważniejsze dla identyfikacji zadania. Niżej znajdują się parametry organizacyjne, takie jak priorytet, status, projekt, termin realizacji, tagi, checklisty oraz powiązane notatki. Formularz zachowuje jednolity układ pól i odstępów, co zwiększa czytelność nawet przy większej liczbie ustawień.

Piotr Słupski
Task Manager

Dashboard

Notes

Checklists

Tasks

Projects

Settings

Sign out

Back to projects

New project

Create a project with priority, lifecycle status and linked checklists.

Title

Description

Priority: Medium Status: Active

Due date: dd-mm-yyyy

Tags: Tag 1 + Add tag

Project checklists: + Add checklist

- ☐ Lista Zakupów Biedronka
- ☐ Sprzątanie
- ☐ Test 2

Linked notes:

- ☐ Praca Inżynierska
- ☐ Test

Project tasks: + Add task

Rys. 20. Widok przedstawiający pierwszą część formularza kreacji projektu.

Pierwsza część formularza projektu skupia się na podstawowych danych opisowych. Widoczne są pola takie jak tytuł, opis, priorytet, status, termin realizacji oraz tagi. W dalszej części ekranu znajdują się również powiązania z checklistami i notatkami, co pokazuje, że projekt może grupować różne typy informacji potrzebnych do organizacji pracy.

Project tasks: + Add task

Add tasks here to create them together with this project.

Kanban columns: + Add column

Column title	Color	Done	Up	Down	Delete
Backlog		☐	↑	↓	🗑️
To do		☐	↑	↓	🗑️
In progress		☐	↑	↓	🗑️
Testing		☐	↑	↓	🗑️
Done		☒	↑	↓	🗑️

Create project

Projects

Settings

Sign out

Rys. 21. Widok przedstawiający drugą część formularza kreacji projektu.

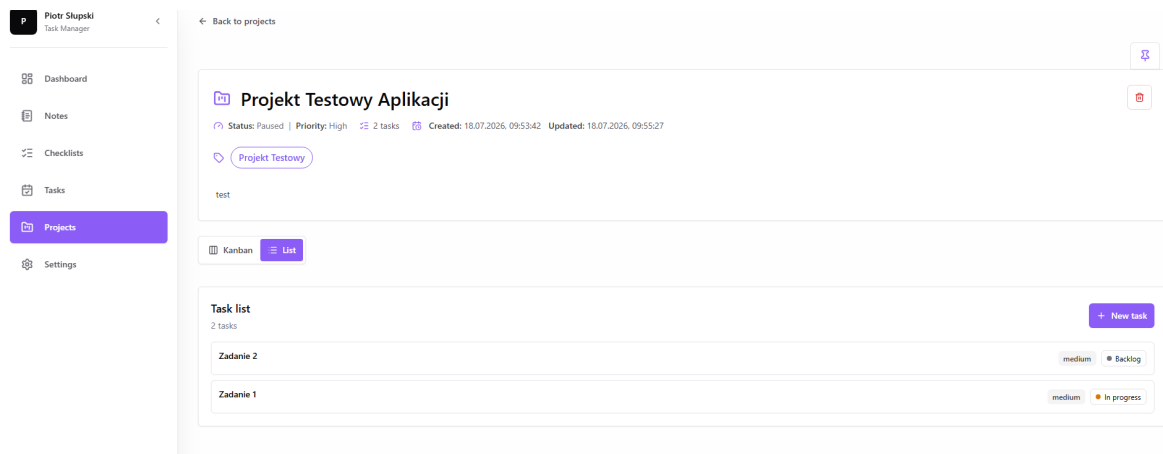
W drugiej części formularza użytkownik może skonfigurować kolumny tablicy kanban. Każda kolumna jest przedstawiona jako osobny wiersz z nazwą, kolorem, oznaczeniem stanu zakończonego oraz przyciskami zmiany kolejności i usunięcia. Dzięki temu struktura przyszłej tablicy projektu jest widoczna jeszcze przed zapisaniem projektu.

Rys. 22. Widok, przedstawiający opcję dodawania tasków do projektu.

Sekcja dodawania tasków pozwala uzupełnić taski bezpośrednio podczas tworzenia projektu. Poszczególne taski zostały ułożone w wierszach, a każdy z nich posiada pola tytułu, priorytetu, statusu i terminu realizacji. Taki układ pozwala użytkownikowi szybko porównać kilka tasków i przypisać im początkowe statusy jeszcze przed utworzeniem projektu.

Rys. 23. Widok, przedstawiający tablicę kanban projektu.

Widok tablicy kanban prezentuje projekt jako zestaw kolumn odpowiadających statusom zadań. Zadania są rozmieszczone w kolumnach, a kolorowe oznaczenia nagłówków pomagają odróżnić poszczególne etapy pracy. Karty tasków zawierają najważniejsze informacje, takie jak tytuł i priorytet, a przyciski przejścia do poprzedniej lub następnej kolumny wspierają zmianę statusu zadania.



Rys. 24. Widok, przedstawiający taski projektów w formie listy.

Alternatywny widok tasków projektu przyjmuje formę listy. W tym układzie zadania są pokazane jako poziome wiersze, dzięki czemu użytkownik może szybko przejrzeć ich nazwy, priorytety oraz aktualne statusy. Przełącznik pomiędzy widokiem kanban i listą znajduje się nad obszarem tasków, co pozwala dobrać sposób prezentacji danych do aktualnego trybu pracy.

7. Implementacja kodu po stronie frontendu

Rozdział opisuje te fragmenty frontendu, które najmocniej wpływają na codzienną pracę z aplikacją: widok kanban w projekcie, przypięte elementy dashboardu oraz formularze tasków i projektów. Nie omawiano każdego komponentu osobno, lecz wybrano miejsca, w których najlepiej widać sposób łączenia stanu interfejsu, danych z backendu i reakcji na działania użytkownika. Przed opisem konkretnych widoków warto jednak wskazać, że warstwa frontendowa została oparta na komponentach używanych w wielu modułach aplikacji. Dotyczy to przede wszystkim formularzy, kart elementów, sekcji listujących dane i przycisków akcji. Dzięki temu użytkownik spotyka podobny układ pól i podobne zachowanie kontrolek niezależnie od tego, czy tworzy task, projekt, notatkę czy checklistę.

Przykładem takiego rozwiązania jest komponent “FormShell”, wykorzystywany jako wspólna struktura formularzy tasków i projektów. Odpowiada on za układ nagłówka, obszaru pól, komunikatów oraz przycisków zapisu, dzięki czemu poszczególne formularze mogą skupiać się na własnych danych i logice wysyłania żądania. Podobną rolę pełni komponent “ObjectCard”, który pozwala prezentować notatki, checklisty, taski i projekty w ujednoliconej formie karty z ikoną, tytułem, opisem oraz metadanymi.

Wspólne komponenty pojawiają się także przy tagach i wyborze powiązanych elementów. Edytor tagów działa tak samo w formularzu taska i projektu, a komponenty wyboru checklist oraz notatek porządkują sposób łączenia danych. W rezultacie zmiana wyglądu lub zachowania jednego powtarzalnego elementu może zostać wykonana w jednym miejscu, zamiast w kilku osobnych formularzach.

7.1 Implementacja widoku kanban

Widok kanban zaimplementowano w komponencie “ProjectKanbanBoard”, zapisanym w pliku “project-kanban-board.tsx”. Komponent odpowiada za ułożenie kolumn statusów, pokazanie kart tasków oraz obsługę przenoszenia ich między etapami pracy. Dane wejściowe obejmują identyfikator projektu, listę kolumn i listę tasków, dlatego komponent nie musi samodzielnie pobierać danych z bazy. Jego rola jest ograniczona do prezentacji i obsługi interakcji na otrzymanym zestawie informacji.

Każda kolumna kanban jest reprezentowana przez obiekt zawierający identyfikator,

nazwę, kolor oraz informację, czy dane zadanie zostało zakończone. Zadanie zawiera między innymi identyfikator, tytuł, opis, priorytet, aktualny status, pozycję, termin oraz tagi. Relacja pomiędzy zadaniem i kolumną jest realizowana przez pole “statusId”. Jeżeli “statusId” zadania jest równe identyfikatorowi kolumny, karta zadania zostaje wyrenderowana właśnie w tej kolumnie.

Lokalny stan tablicy przechowywany jest przy użyciu hooków “useState”. Obejmuje on aktualne taski, kolumny, identyfikator przeciąganego taska, aktywną kolumnę upuszczania oraz dane edytowanej kolumny. Hook “useEffect” synchronizuje ten stan z danymi przekazanymi przez stronę projektu, co jest potrzebne po odświeżeniu danych przez router albo po zapisaniu zmian w bazie.

Lista kolumn powstaje w “useMemo”, ponieważ zależy od konfiguracji projektu i statusów istniejących tasków. Gdy projekt nie ma własnych kolumn, dodawana jest kolumna domyślna “todo”. Jeśli natomiast task wskazuje status, którego nie ma już w konfiguracji projektu, komponent tworzy dodatkową kolumnę techniczną. Dzięki temu task nie znika z widoku tylko dlatego, że dane projektu i dane tasków chwilowo nie są w pełni zgodne.

```
const orderedColumns = useMemo(() => {
  const baseColumns = boardColumns.length
    ? boardColumns
    : [{ id: "todo", title: "To do", color: "#2563eb", isDone: false }];
  const knownColumnIds = new Set(baseColumns.map((column) => column.id));
  const extraColumns = Array.from(
    new Set(
      boardTasks
        .map((task) => task.statusId)
        .filter((statusId) => statusId && !knownColumnIds.has(statusId))
    )
  ).map((statusId) => ({
    id: statusId,
    title: statusId.replace(/_/g, " "),
    color: "#71717a",
    isDone: false
  }));
  return [...baseColumns, ...extraColumns];
}, [boardColumns, boardTasks]);
const columnIds = useMemo(() => orderedColumns.map((column) => column.id), [orderedColumns]);
const taskCount = boardTasks.length;
```

Rys. 25. Fragment kodu przedstawiający wyliczanie kolumn widoku kanban.

Powyższy kod pokazuje zabezpieczenie przed pustą konfiguracją kolumn oraz przed niespójnością między statusem taska a definicją tablicy. Komponent najpierw ustala bazową listę kolumn, a następnie dopisuje kolumny techniczne wynikające z danych tasków. W efekcie widok pozostaje kompletny nawet wtedy, gdy konfiguracja projektu wymaga późniejszego uporządkowania.

Przenoszenie zadania między kolumnami zostało zaimplementowane na dwa sposoby. Pierwszym jest mechanizm “drag and drop”, w którym funkcje “handleDragStart”, “handleDragOver” i “handleDrop” obsługują przeciąganie karty oraz upuszczanie jej w wybranej kolumnie. Drugim sposobem są przyciski “Back” i “Next”, które przenoszą zadanie

do poprzedniej lub następnej kolumny. Dzięki temu widok pozostaje użyteczny również wtedy, gdy użytkownik nie chce korzystać z przeciągania.

Za właściwą zmianę statusu odpowiada funkcja “moveTask”. Po wybraniu nowej kolumny funkcja oblicza następną pozycję zadania w tej kolumnie, aktualizuje lokalny stan tablicy i wysyła żądanie “PATCH” do endpointu “/api/tasks/{taskId}”. W treści żądania przekazywane są pola “statusId”, “projectId” oraz “position”. Aktualizacja lokalnego stanu wykonywana jest przed odpowiedzią serwera, co daje użytkownikowi wrażenie natychmiastowej reakcji interfejsu. Jeżeli backend zwróci błąd, komponent przywraca poprzedni stan zadań i wyświetla komunikat o niepowodzeniu operacji.

```
async function moveTask(task: KanbanTask, nextStatusId: string) {
  if (task.statusId === nextStatusId) {
    return;
  }

  setError(null);
  setMovingTaskId(task.id);
  const nextPosition =
    Math.max(
      -1,
      ...boardTasks
        .filter((currentTask) => currentTask.id !== task.id && currentTask.statusId === nextStatusId)
        .map((currentTask) => currentTask.position ?? 0)
    ) + 1;

  setBoardTasks((currentTasks) =>
    currentTasks.map((currentTask) =>
      currentTask.id === task.id
        ? { ...currentTask, statusId: nextStatusId, position: nextPosition }
        : currentTask
    )
  );

  const response = await fetch(`/api/tasks/${task.id}`, {
    method: "PATCH",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      statusId: nextStatusId,
      projectId,
      position: nextPosition
    })
  });

  if (!response.ok) {
    setError("Could not move the task.");
    setBoardTasks(tasks);
    setMovingTaskId("");
    return;
  }

  setMovingTaskId("");
  router.refresh();
}
```

Rys. 26. Fragment kodu przedstawiający zmianę statusu taska w widoku kanban.

Kod funkcji “moveTask” pokazuje połączenie logiki interfejsu z zapisem danych po stronie serwera. Najpierw wyliczana jest nowa pozycja taska w kolumnie docelowej, potem aktualizowany jest stan lokalny, a następnie wykonywane jest żądanie do API. W razie błędu komponent wycofuje zmianę, dzięki czemu użytkownik nie zostaje z widokiem niespójnym ze stanem zapisanym w bazie danych.

Istotnym elementem implementacji jest również edycja kolumn. Użytkownik może kliknąć nagłówek kolumny, zmienić jej nazwę, kolor oraz oznaczyć ją jako kolumnę końcową. Po zapisaniu zmian, komponent wysyła żądanie “PATCH” do endpointu “/api/projects/{projectId}”, przekazując zaktualizowaną tablicę “kanbanColumns”. Każda

kolumna otrzymuje także pozycję wynikającą z kolejności w tablicy. Po poprawnym zapisie wykonywana jest funkcja `“router.refresh()”`, która odświeża dane projektu.

Widok kanban jest osadzony w komponencie `“ProjectTaskView”`. Ten komponent pozwala przełączać sposób prezentacji zadań między widokiem `“kanban”` a widokiem `“listy”`. Wybrany tryb jest zapisywany w projekcie przez żądanie `“PATCH”` do endpointu projektu, w polu `“taskView”`. Dzięki temu wybór użytkownika nie jest wyłącznie stanem tymczasowym w przeglądarce, ale częścią konfiguracji projektu przechowywaną w bazie danych. Dodatkowo dla tablicy kanban dostępny jest tryb rozszerzony, który wyświetla tablicę na całym ekranie i ułatwia pracę z większą liczbą zadań.

Karty zadań w widoku kanban zawierają najważniejsze informacje potrzebne do szybkiej oceny pracy: tytuł, skrócony opis, priorytet, termin oraz tagi. Priorytety są wyróżnione kolorami, a termin jest prezentowany razem z ikoną kalendarza. Każda karta jest linkiem do szczegółów zadania, dzięki czemu użytkownik może szybko przejść z ogólnego widoku projektu do edycji konkretnego elementu.

7.2 Implementacja sekcji przypiętych elementów dashboardu

Sekcja przypiętych elementów dashboardu została oparta na kolekcji `“pins”`, która przechowuje identyfikator użytkownika, typ przypiętego elementu, identyfikator celu oraz pozycję. Frontend nie przechowuje osobnych list przypiętych notatek, zadań, checklist i projektów. Zamiast tego każdy pin zawiera parę pól `“targetType”` oraz `“targetId”`, która pozwala wskazać dowolny obsługiwany typ elementu.

Pobieranie danych do dashboardu odbywa się w pliku `“dashboard/page.tsx”`. Strona najpierw sprawdza sesję użytkownika, a następnie łączy się z bazą danych. Za pomocą funkcji `“Promise.all”` równolegle pobierane są przypięcia, liczniki głównych typów danych, ostatnio aktualizowane notatki, checklisty, taski i projekty oraz wydarzenia kalendarza wynikające z terminów tasków i projektów. Takie podejście ogranicza czas oczekiwania na dane, ponieważ niezależne zapytania są wykonywane równocześnie.

Dla każdego dokumentu modelu `“Pin”` wykonywana jest funkcja `“findPinnedTarget”`. Funkcja ta sprawdza typ przypiętego elementu i na tej podstawie pobiera właściwy dokument z kolekcji `“notes”`, `“checklists”`, `“tasks”` albo `“projects”`. Każde zapytanie zawiera dwa warunki do spełnienia - element musi posiadać właściwość `“ownerId”` oraz nie być zarchiwizowany, czyli spełniać warunek `“archivedAt: null”`. Dlatego dashboard pokazuje tylko aktywne elementy należące do zalogowanego użytkownika. Jeżeli przypięty dokument nie istnieje lub został zarchiwizowany, nie jest przekazywany do interfejsu.

Po pobraniu celu przypięcia, dane są mapowane do wspólnego formatu `“PinnedItem”`. Obiekt ten zawiera tytuł, opis, typ, status, priorytet, tagi, daty utworzenia i aktualizacji oraz link prowadzący do szczegółów elementu. Dzięki ujednoliceniu danych komponenty frontendowe mogą renderować przypięte notatki, checklisty, zadania i projekty w jednej sekcji,

bez konieczności tworzenia czterech osobnych widoków.

Głównym komponentem dashboardu jest “PinnedBoard”. Renderuje on kafelki metryk, mały kalendarz oraz komponent “PinnedItemsSearch”. Kafelki metryk pokazują liczbę notatek, checklist, zadań i projektów. Po najechnięciu na kafelek wyświetlana jest lista ostatnio aktualizowanych elementów danego typu. Kalendarz jest budowany lokalnie na podstawie zadań i projektów mających termin w bieżącym miesiącu. Funkcja “buildCalendarDays” tworzy siatkę dni, oznacza aktualny dzień oraz przypisuje wydarzenia do odpowiednich dat.

Za właściwą listę przypiętych elementów odpowiada komponent “PinnedItemsSearch”. Komponent ten działa po stronie klienta, ponieważ obsługuje stan wyszukiwarki, filtrów oraz sortowania. Użytkownik może wyszukiwać przypięte elementy po tytule, opisie, typie, statusie i metadanych. Może także filtrować wyniki według typu elementu: notatek, checklist, zadań lub projektów. Lista typów jest przechowywana w stanie “selectedTypes”, a filtrowanie wykonywane jest w funkcji “useMemo”, aby niepotrzebnie nie przeliczać wyników przy każdym renderowaniu.

```
export function PinnedItemsSearch({ pinnedItems }: PinnedItemsSearchProps) {
  const [query, setQuery] = useState("");
  const [selectedTypes, setSelectedTypes] = useState<string[]>(() =>
    typeFilters.map((filter) => filter.value)
  );
  const [sortField, setSortField] = useState("updated");
  const [sortDirection, setSortDirection] = useState<"asc" | "desc">("desc");
  const normalizedQuery = query.trim().toLowerCase();
  const selectedTypeSet = useMemo(() => new Set(selectedTypes), [selectedTypes]);
  const filteredItems = useMemo(() => {
    return pinnedItems.filter((item) =>
      selectedTypeSet.has(item.type) &&
      (!normalizedQuery ||
        [item.title, item.description, item.type, item.meta, item.status]
          .join(" ")
          .toLowerCase()
          .includes(normalizedQuery))
    );
  }, [normalizedQuery, pinnedItems, selectedTypeSet]);
  const sortedItems = useMemo(() => {
    const directionModifier = sortDirection === "asc" ? 1 : -1;

    return [...filteredItems].sort((firstItem, secondItem) => {
      if (sortField === "title") {
        return firstItem.title.localeCompare(secondItem.title) * directionModifier;
      }

      if (sortField === "description") {
        const firstDescription = firstItem.description || firstItem.meta;
        const secondDescription = secondItem.description || secondItem.meta;

        return firstDescription.localeCompare(secondDescription) * directionModifier;
      }

      const firstDate = sortField === "created" ? firstItem.createdAt : firstItem.updatedAt;
      const secondDate = sortField === "created" ? secondItem.createdAt : secondItem.updatedAt;
      const firstTime = firstDate ? new Date(firstDate).getTime() : 0;
      const secondTime = secondDate ? new Date(secondDate).getTime() : 0;

      return (firstTime - secondTime) * directionModifier;
    });
  }, [filteredItems, sortDirection, sortField]);
```

Rys. 27. Fragment kodu przedstawiający filtrowanie i sortowanie przypiętych elementów.

Przedstawiony kod skupia w jednym miejscu stan wyszukiwarki, wybrane filtry typów oraz konfigurację sortowania. Zastosowanie “useMemo” ogranicza ponowne przeliczanie listy do sytuacji, w których zmieniają się dane wejściowe albo ustawienia użytkownika. Dzięki temu komponent pozostaje czytelny, a jednocześnie obsługuje kilka wariantów przeglądania przypiętych elementów.

Sortowanie przypiętych elementów wykorzystuje stan “sortField” i “sortDirection”. Dostępne są kryteria związane z datą aktualizacji, datą utworzenia, tytułem i opisem. Po przefiltrowaniu elementy są kopiowane do nowej tablicy, a dopiero później sortowane, dzięki czemu komponent nie modyfikuje danych wejściowych otrzymanych z dashboardu.

Każdy przypięty element jest renderowany jako karta “ObjectCard”. Ikona karty zależy od typu elementu: notatka, checklista, zadanie lub projekt. Dla zadań i projektów przekazywany jest status oraz priorytet, natomiast dla checklist możliwe jest pokazanie podglądu elementów checklisty. Link karty prowadzi bezpośrednio do szczegółowego widoku danego elementu.

Za przypinanie i odpinanie odpowiada komponent przycisku pinezki, wydzielony w kodzie jako “PinEntityButton”. Komponent otrzymuje typ i identyfikator elementu oraz opcjonalny identyfikator istniejącego przypięcia. Jeżeli element nie jest przypięty, kliknięcie wysyła żądanie “POST” do endpointu “/api/pins”. Jeżeli element jest już przypięty, kliknięcie wysyła żądanie “DELETE” do endpointu “/api/pins/{pinId}”. Po poprawnym zakończeniu operacji komponent aktualizuje lokalny stan, zmienia wygląd ikony pinezki oraz wykonuje funkcję “router.refresh()”, aby odświeżyć dashboard i pozostałe widoki. Po zakończeniu operacji aktualizowany jest też lokalny identyfikator przypięcia, co pozwala natychmiast zmienić wygląd przycisku.

```

Tabnine | Edit | Test | Explain | Document
export function PinEntityButton({
  targetType,
  targetId,
  initialPinId,
  className = ""
}: PinEntityButtonProps) {
  const router = useRouter();
  const [pinId, setPinId] = useState(initialPinId ?? "");
  const [error, setError] = useState<string | null>(null);
  const [isSubmitting, setIsSubmitting] = useState(false);

  async function handleToggle() {
    setError(null);
    setIsSubmitting(true);

    const response = pinId
      ? await fetch(`/api/pins/${pinId}`, { method: "DELETE" })
      : await fetch("/api/pins", {
          method: "POST",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({ targetType, targetId })
        });

    if (!response.ok) {
      setError(pinId ? "Nie udało się odpiąć elementu." : "Nie udało się przypiąć elementu.");
      setIsSubmitting(false);
      return;
    }

    const payload = await response.json();
    setPinId(pinId ? "" : String(payload.pin._id));
    setIsSubmitting(false);
    router.refresh();
  }
}

```

Rys. 28. Fragment kodu przedstawiający obsługę przypinania i odpinania elementów.

W efekcie sekcja przypiętych elementów pełni rolę spersonalizowanego skrótu do najważniejszych danych użytkownika. Implementacja została oparta na jednym wspólnym modelu przypięcia oraz na uniwersalnym komponencie listującym, dzięki czemu dodanie obsługi kolejnego typu elementu wymagałoby głównie rozszerzenia mapowania typów i sposobu pobierania celu przypięcia.

7.3 Implementacja formularza taska

Formularz taska znajduje się w komponencie “TaskForm” w pliku “task-form.tsx”. Ten sam komponent obsługuje tworzenie i edycję, ponieważ tryb działania przekazywany jest w parametrze “mode”. Rozwiązanie ogranicza powielanie kodu: układ pól, podstawowa walidacja i wysyłanie danych do API są wspólne dla obu widoków.

Struktura formularza została osadzona w komponencie “FormShell”, który zapewnia wspólny układ wizualny dla formularzy aplikacji. Wewnątrz formularza umieszczono pola

odpowiadające najważniejszym właściwościom taska: tytuł, opis, priorytet, status, projekt, termin wykonania oraz tagi. Część pól jest prezentowana warunkowo. Jeżeli użytkownik przypisze task do projektu, formularz pobiera dostępne statusy z kolumn kanban danego projektu i pozwala wybrać właściwą kolumnę. Jeżeli task nie należy do projektu, wykorzystywane są podstawowe statusy tasków osobistych.

Komponent przechowuje stan formularza za pomocą hooków “useState”. W stanie znajdują się między innymi komunikaty błędów, informacja o trwającym zapisie, wybrane tagi, powiązane checklisty, powiązane notatki, nowe tytuły checklist oraz identyfikator wybranego projektu. Dodatkowo zastosowano “useMemo”, aby wyliczać aktualnie wybrany projekt i listę jego statusów tylko wtedy, gdy zmienia się dane wejściowe formularza. Funkcja “useEffect” pilnuje natomiast, aby po zmianie projektu wybrany status nadal należał do dostępnych kolumn kanban.

```
const selectedProject = useMemo(
  () => projectOptions.find((project) => project.id === selectedProjectId),
  [projectOptions, selectedProjectId]
);
const projectStatusOptions = useMemo(
  () => selectedProject?.kanbanColumns ?? [],
  [selectedProject]
);
const isProjectTask = Boolean(selectedProjectId);

function addNewChecklist() {
  setNewChecklistTitles((currentTitles) => [...currentTitles, ""]);
}

function updateNewChecklist(index: number, title: string) {
  setNewChecklistTitles((currentTitles) =>
    currentTitles.map((currentTitle, titleIndex) => titleIndex === index ? title : currentTitle)
  );
}

function removeNewChecklist(index: number) {
  setNewChecklistTitles((currentTitles) =>
    currentTitles.filter((_, titleIndex) => titleIndex !== index)
  );
}

useEffect(() => {
  if (
    isProjectTask &&
    projectStatusOptions.length &&
    !projectStatusOptions.some((status) => status.id === selectedStatusId)
  ) {
    setSelectedStatusId(projectStatusOptions[0]?.id ?? "todo");
  }
}, [isProjectTask, projectStatusOptions, selectedStatusId]);
```

Rys. 29. Fragment kodu przedstawiający stan formularza taska oraz dopasowanie statusów do wybranego projektu.

Pokazany fragment kodu obrazuje sposób połączenia pól formularza z konfiguracją projektu. Stan komponentu przechowuje wartości wybierane przez użytkownika, a wyliczane listy statusów pozwalają dopasować formularz taska do kolumn kanban przypisanych do kon-

kretnego projektu. Dzięki temu użytkownik nie wybiera statusu oderwanego od struktury projektu, lecz korzysta z opcji wynikających z jego aktualnej konfiguracji.

Wysyłanie formularza obsługuje funkcja `“handleSubmit”`. Po zatrzymaniu domyślnego działania formularza tworzony jest obiekt `“FormData”`, z którego pobierane są wartości pól. Przed wysłaniem żądania sprawdzane jest, czy tytuł taska nie jest pusty. Następnie komponent buduje obiekt danych zawierający między innymi pola `“title”`, `“description”`, `“priority”`, `“statusId”`, `“projectId”`, `“dueDate”`, `“tags”`, `“checkboxlistIds”`, `“newChecklists”` oraz `“noteIds”`. W zależności od trybu formularz wysyła żądanie `“POST”` do endpointu `“/api/tasks”` albo żądanie `“PATCH”` do endpointu `“/api/tasks/{taskId}”`.

Po poprawnym utworzeniu taska użytkownik jest przenoszony do widoku listy tasków, a router odświeża dane strony. W trybie edycji formularz pozostaje w bieżącym widoku, wyświetla komunikat o zapisaniu zmian i wywołuje funkcję `“onSaved”`, jeżeli została przekazana przez komponent nadrzędny. Takie rozwiązanie pozwala używać formularza zarówno jako pełnej strony tworzenia, jak i elementu edycji w istniejącym przepływie pracy.

7.4 Implementacja formularza projektu

Formularz projektu zapisano w komponencie `“ProjectForm”` w pliku `“project-form.tsx”`. Podobnie jak formularz taska działa on w trybie tworzenia i edycji, ale ma szerszy zakres odpowiedzialności. Poza podstawowymi polami projektu obsługuje konfigurację kolumn kanban oraz dodawanie początkowych tasków, więc już na etapie tworzenia projektu można przygotować jego roboczą strukturę.

Podstawowa część formularza obejmuje tytuł, opis, priorytet, status cyklu życia projektu, termin wykonania oraz tagi. Dane te są wysyłane do backendu jako pola `“title”`, `“description”`, `“priority”`, `“lifecycleStatus”`, `“dueDate”` oraz `“tags”`. Formularz pozwala również powiązać projekt z istniejącymi checklistami i notatkami oraz utworzyć nowe checklisty podczas zapisu projektu. Dzięki temu użytkownik może przygotować projekt wraz z najważniejszymi elementami organizacyjnymi w jednym widoku.

Najważniejszą częścią formularza jest konfiguracja kolumn kanban. Domyślnie formularz posiada zestaw kolumn `“backlog”`, `“todo”`, `“in_progress”`, `“testing”` oraz `“done”`. Każda kolumna ma identyfikator, nazwę, kolor, informację o tym, czy jest kolumną końcową, oraz pozycję. Funkcja `“slugifyColumnName”` tworzy techniczny identyfikator kolumny na podstawie jej nazwy, natomiast funkcja `“normalizeKanbanColumns”` porządkuje tablicę kolumn przed wysłaniem jej do API. Normalizacja usuwa puste nazwy, uzupełnia brakujące kolory, nadaje pozycje oraz zabezpiecza przed powtórzeniem identyfikatorów.

```

function slugifyColumnId(value: string, fallback: string) {
  const normalized = value
    .trim()
    .toLowerCase()
    .replace(/^[^a-z0-9]+/g, "_")
    .replace(/^[^_]+$/g, "");
  return normalized || fallback;
}

function normalizeKanbanColumns(columns: KanbanColumnInput[]) {
  const usedIds = new Set<string>();

  return columns
    .map((column, index) => {
      const title = column.title.trim();
      const baseId = slugifyColumnId(column.id || title, `column_${index + 1}`);
      let id = baseId;
      let duplicateIndex = 2;

      while (usedIds.has(id)) {
        id = `${baseId}_${duplicateIndex}`;
        duplicateIndex += 1;
      }

      usedIds.add(id);

      return {
        id,
        title,
        position: index,
        color: column.color || "#71717a",
        isDone: column.isDone
      };
    })
    .filter((column) => column.title);
}

```

Rys. 30. Fragment kodu przedstawiający tworzenie identyfikatorów oraz normalizację kolumn kanban w formularzu projektu.

Pierwszy fragment kodu pokazuje przygotowanie kolumn kanban przed zapisem projektu. Funkcja tworząca identyfikator techniczny ujednolica nazwy kolumn, natomiast normalizacja porządkuje ich listę, usuwa puste wartości i zabezpiecza formularz przed zapisaniem niespójnej konfiguracji.

```

function updateKanbanColumn(index: number, column: KanbanColumnInput) {
  const previousColumnId = kanbanColumns[index]?.id;

  setKanbanColumns((currentColumns) =>
    currentColumns.map((currentColumn, columnIndex) =>
      columnIndex === index ? column : currentColumn
    )
  );

  if (previousColumnId && previousColumnId !== column.id) {
    setNewTasks((currentTasks) =>
      currentTasks.map((task) =>
        task.statusId === previousColumnId ? { ...task, statusId: column.id } : task
      )
    );
  }
}

function moveKanbanColumn(index: number, direction: -1 | 1) {
  setKanbanColumns((currentColumns) => {
    const nextIndex = index + direction;

    if (nextIndex < 0 || nextIndex >= currentColumns.length) {
      return currentColumns;
    }

    const nextColumns = [...currentColumns];
    const [column] = nextColumns.splice(index, 1);
    nextColumns.splice(nextIndex, 0, column);

    return nextColumns;
  });
}

```

Rys. 31. *Fragment kodu przedstawiający edycję oraz zmianę kolejności kolumn kanban w formularzu projektu.*

Drugi fragment pokazuje funkcje wykorzystywane podczas bieżącej pracy użytkownika z formularzem projektu. Kod pozwala aktualizować właściwości kolumn, usuwać je oraz zmieniać ich kolejność, dzięki czemu końcowa struktura tablicy kanban może zostać dopasowana do sposobu realizacji projektu.

Formularz projektu umożliwia także zarządzanie kolejnością kolumn. Funkcje “moveKanbanColumn”, “updateKanbanColumn”, “removeKanbanColumn” oraz mechanizm dodawania nowej kolumny pozwalają użytkownikowi zbudować strukturę tablicy dopasowaną do sposobu pracy nad projektem. Jeżeli podczas edycji zmieni się identyfikator kolumny, komponent aktualizuje również statusy nowych tasków przypisanych do tej kolumny. Dzięki temu dane przygotowane w formularzu pozostają spójne przed wysłaniem do backendu.

Drugim rozbudowanym elementem jest sekcja początkowych tasków projektu. Każdy nowy task otrzymuje lokalny identyfikator generowany przez “crypto.randomUUID”, tytuł, opis, priorytet, status, termin oraz tagi. Funkcje “addProjectTask”, “updateProjectTask” i “removeProjectTask” pozwalają dodawać, modyfikować oraz usuwać taski jeszcze przed utworzeniem projektu. Podczas zapisu puste tytuły są odfiltrowywane, aby do API trafiły

wyłącznie kompletne elementy.

Zapis projektu odbywa się w funkcji “handleSubmit”. Formularz najpierw sprawdza obecność tytułu oraz poprawność listy kolumn kanban. Następnie przygotowuje obiekt danych zawierający konfigurację projektu, powiązane elementy, nowe checklisty, początkowe taski oraz tablicę “kanbanColumns”. W trybie tworzenia wykonywane jest żądanie “POST” do endpointu “/api/projects”, natomiast w trybie edycji żądanie “PATCH” do endpointu “/api/projects/{projectId}”. Po poprawnym utworzeniu projektu użytkownik jest przenoszony do listy projektów, a po edycji otrzymuje komunikat o zapisaniu zmian.

8. Implementacja kodu po stronie backendu

Rozdział przedstawia wybrane elementy backendu, które dobrze pokazują sposób działania części serwerowej aplikacji. Omówiono obsługę checklist przez API, wspólne mechanizmy pracy z zadaniami oraz autoryzację użytkownika. Nie rozpisano każdego endpointu osobno, ponieważ wiele z nich powtarza ten sam schemat: sprawdzenie sesji, walidację danych, wykonanie operacji na bazie i zwrócenie odpowiedzi do frontendu.

8.1 Implementacja handlera API checklist

Obsługa checklist została zaimplementowana w dwóch routach API: listowym oraz dynamicznym dla pojedynczej checklisty. Pierwszy route odpowiada za operacje wykonywane na całej kolekcji checklist, natomiast drugi za operacje dotyczące pojedynczej checklisty wskazanej przez identyfikator w adresie żądania. Takie rozdzielenie wynika ze sposobu działania mechanizmu routingu w Next.js, w którym struktura katalogów określa ścieżki dostępne w API.

W pliku “route.ts” zdefiniowano metody “GET” oraz “POST”. Metoda “GET” pobiera listę aktywnych checklist należących do aktualnie zalogowanego użytkownika. Przed wykonaniem zapytania wywoływana jest funkcja “getCurrentUserId”, a brak identyfikatora kończy działanie odpowiedzią “401 Unauthorized”.

Po pozytywnej weryfikacji użytkownika wykonywane jest połączenie z bazą danych przez funkcję “connectDatabase”. Następnie model “Checklist” wyszukuje dokumenty spełniające warunki “ownerId” oraz “archivedAt: null”. Oznacza to, że użytkownik otrzymuje tylko własne checklisty, które nie zostały wcześniej zarchiwizowane. Wyniki są sortowane według pola “position”, a następnie według daty aktualizacji, dzięki czemu interfejs może prezentować elementy w ustalonej kolejności.

Metoda “POST” odpowiada za utworzenie nowej checklisty. Treść żądania jest odczytywana za pomocą funkcji “parseJsonBody”, która jednocześnie korzysta ze schematu walidacji “checklistCreateSchema”¹. Schemat waliduje tytuł checklisty, elementy checklisty, typ i identyfikator elementu nadrzędnego oraz pozycję. W przypadku niepoprawnych

¹ Zod, *Defining schemas*, dokumentacja biblioteki Zod dotycząca definiowania schematów walidacji, [b.r.], <https://zod.dev/api?id=property> (dostęp: 16.08.2026).

danych handler zwraca odpowiedź błędu i nie wykonuje zapisu w bazie danych.

Przed utworzeniem dokumentu dane są przekazywane do funkcji “sanitizeMutation”. Jej zadaniem jest ograniczenie zapisywanych wartości do pól dopuszczonych przez logikę aplikacji. Następnie model “Checklist” tworzy dokument z danymi użytkownika oraz identyfikatorem właściciela. Po poprawnym zapisie wywoływana jest funkcja “recordActivityEvent”, która rejestruje zdarzenie utworzenia checklisty w historii aktywności.

```
import { getCurrentUserId, sanitizeMutation, unauthorizedResponse } from "@lib/session";
import { recordActivityEvent } from "@lib/activity-events";
import { checklistCreateSchema } from "@lib/validation-schemas";
import { Checklist } from "@models/checklist";

Tabnine | Edit | Test | Explain | Document
export async function GET() {
  const ownerId = await getCurrentUserId();

  if (!ownerId) {
    return unauthorizedResponse();
  }

  await connectDatabase();
  const checklists = await Checklist.find({ ownerId, archivedAt: null }).sort({
    position: 1,
    updatedAt: -1
  });

  return NextResponse.json({ checklists });
}

Tabnine | Edit | Test | Explain | Document
export async function POST(request: NextRequest) {
  const ownerId = await getCurrentUserId();

  if (!ownerId) {
    return unauthorizedResponse();
  }

  const { data, error } = await parseJsonBody(request, checklistCreateSchema);

  if (!data) {
    return badRequestResponse(error);
  }

  await connectDatabase();
  const payload = sanitizeMutation(data);
  const checklist = await Checklist.create({ ...payload, ownerId });
  await recordActivityEvent({
    ownerId,
    entityType: "checklist",
    entityId: checklist.id,
    action: "created"
  });

  return NextResponse.json({ checklist }, { status: 201 });
}
```

Rys. 32. Fragment kodu przedstawiający obsługę metod GET i POST dla checklist.

Na fragmencie kodu widać wspólny schemat obsługi żądań listowych. Obie metody najpierw sprawdzają użytkownika wykonującego żądanie. Metoda “GET” filtruje checklisty po właścicielu i stanie archiwizacji, natomiast metoda “POST” dodatkowo waliduje dane

wejściowe, oczyszcza przekazywany obiekt i zapisuje zdarzenie aktywności.

Operacje na pojedynczej checkliście znajdują się w dynamicznej ścieżce API, w której identyfikator checklisty jest częścią adresu żądania. W metodach “GET”, “PATCH” i “DELETE” sprawdzany jest użytkownik wykonujący operację oraz poprawność identyfikatora checklisty za pomocą funkcji “isValidObjectId”. Dzięki temu aplikacja nie wykonuje zapytań do bazy dla niepoprawnie zbudowanych identyfikatorów.

Metoda “PATCH” aktualizuje checklistę przy użyciu schematu “checklistUpdate Schema”. Schemat ten dopuszcza częściową aktualizację, ale wymaga przekazania przynajmniej jednego pola. Aktualizacja wykonywana jest przez funkcję “findOneAndUpdate” z warunkami “_id”, “ownerId” oraz “archivedAt: null”. Taki zapis zabezpiecza przed zmianą checklist należących do innych użytkowników oraz przed modyfikacją elementów zarchiwizowanych.

```
export async function PATCH(request: NextRequest, { params }: RouteContext) {
  const ownerId = await getCurrentUserId();

  if (!ownerId) {
    return unauthorizedResponse();
  }

  if (!isValidObjectId(params.checklistId)) {
    return notFoundResponse();
  }

  const { data, error } = await parseJsonBody(request, checklistUpdateSchema);

  if (!data) {
    return badRequestResponse(error);
  }

  await connectDatabase();
  const payload = sanitizeMutation(data);
  const checklist = await Checklist.findOneAndUpdate(
    { _id: params.checklistId, ownerId, archivedAt: null },
    { $set: payload },
    { new: true, runValidators: true }
  );

  if (!checklist) {
    return notFoundResponse();
  }

  await recordActivityEvent({
    ownerId,
    entityType: "checklist",
    entityId: checklist.id,
    action: "updated"
  });

  return NextResponse.json({ checklist });
}
```

Rys. 33. Fragment kodu przedstawiający aktualizację pojedynczej checklisty.

Aktualizacja pojedynczej checklisty łączy kilka zabezpieczeń w jednym przepływie. Najpierw wykonywana jest kontrola dostępu i poprawności identyfikatora, później walido-

wana jest treść żądania, a sama aktualizacja dotyczy wyłącznie dokumentu należącego do aktualnego użytkownika. Warunek “archivedAt: null” uniemożliwia modyfikację checklist, które zostały już usunięte z aktywnych widoków.

Usunięcie checklisty zostało zrealizowane jako archiwizacja. Metoda “DELETE” nie usuwa dokumentu fizycznie z bazy, lecz zapisuje w polu “archivedAt” aktualną datę. Dzięki temu dane nie są widoczne w standardowych zapytaniach, ale w razie potrzeby możliwe byłoby zachowanie historii działania systemu. Po aktualizacji dokumentu rejestrowane jest zdarzenie usunięcia checklisty.

```
export async function DELETE(_request: NextRequest, { params }: RouteContext) {
  const ownerId = await getCurrentUserId();

  if (!ownerId) {
    return unauthorizedResponse();
  }

  if (!isValidObjectId(params.checklistId)) {
    return notFoundResponse();
  }

  await connectDatabase();
  const checklist = await Checklist.findOneAndUpdate(
    { _id: params.checklistId, ownerId, archivedAt: null },
    { $set: { archivedAt: new Date() } },
    { new: true }
  );

  if (!checklist) {
    return notFoundResponse();
  }

  await recordActivityEvent({
    ownerId,
    entityType: "checklist",
    entityId: checklist.id,
    action: "deleted"
  });

  return NextResponse.json({ checklist });
}
```

Rys. 34. Fragment kodu przedstawiający archiwizację checklisty w metodzie DELETE.

Zastosowanie archiwizacji sprawia, że operacja usunięcia jest odwracalna na poziomie danych i nie narusza powiązań z innymi elementami systemu. Kod zachowuje ten sam model bezpieczeństwa co aktualizacja: wymaga zalogowanego użytkownika, poprawnego identyfikatora oraz zgodności pola “ownerId”. Dopiero spełnienie tych warunków pozwala ustawić datę archiwizacji.

8.2 Wspólne mechanizmy obsługi żądań API

W backendzie wydzielono funkcje pomocnicze używane przez wiele endpointów API. Odpowiadają one za powtarzalne czynności: odczyt danych z żądania, walidację, przygotowa-

nie odpowiedzi błędów, pobranie użytkownika z sesji i oczyszczenie danych przed zapisem. Dzięki temu kod handlerów checklist, tasków czy projektów nie musi za każdym razem powtarzać tych samych fragmentów technicznych.

Pierwszym przykładem jest moduł “api-request.ts”, w którym znajdują się funkcje “readJsonBody” oraz “parseJsonBody”. Funkcja “readJsonBody” próbuje odczytać treść żądania jako obiekt JSON i zwraca pustą wartość, gdy format jest niepoprawny. Funkcja “parseJsonBody” dodaje do tego walidację schematem Zod. Handler otrzymuje więc albo poprawne dane, albo ujednolicony komunikat błędu.

```
import { NextRequest } from "next/server";
import { z } from "zod";

export async function readJsonBody(request: NextRequest) {
  try {
    const body: unknown = await request.json();
    return body && typeof body === "object" ? (body as Record<string, unknown>) : null;
  } catch {
    return null;
  }
}

export async function parseJsonBody<TSchema extends z.ZodTypeAny>(
  request: NextRequest,
  schema: TSchema
) {
  const body = await readJsonBody(request);

  if (!body) {
    return {
      data: null,
      error: "Invalid JSON body"
    };
  }

  const result = schema.safeParse(body);

  if (!result.success) {
    return {
      data: null,
      error: result.error.issues[0]?.message ?? "Invalid request body"
    };
  }

  return {
    data: result.data as z.infer<TSchema>,
    error: null
  };
}
```

Rys. 35. Fragment kodu przedstawiający odczyt i walidację ciała żądania API.

Na rysunku widoczny jest wspólny mechanizm, który ogranicza powtarzanie kodu w handlerach API. Funkcja odczytująca ciało żądania zabezpiecza aplikację przed niepoprawnym formatem JSON, a funkcja walidująca zwraca ujednolicony wynik w postaci danych albo komunikatu błędu. Dzięki temu poszczególne endpointy mogą korzystać z tego samego sposobu obsługi danych wejściowych.

Drugim wspólnym obszarem jest obsługa odpowiedzi błędów. W module odpowiedzi API zdefiniowano funkcje zwracające błędy żądania, przekroczenia limitu oraz niedostępności usługi. Każda z nich zwraca odpowiedź “NextResponse.json” z ustalonym kodem statusu HTTP i spójną strukturą komunikatu. Podobną rolę pełnią funkcje “unauthorizedResponse” oraz “notFoundResponse” z modułu “session.ts”. Dzięki temu błędy “400”, “401”, “404”, “429” i “503” są zwracane w przewidywalny sposób, bez ręcznego składania odpowiedzi w każdym handlerze.

Jak opisano w podrozdziale 3.5, mechanizm sesji pozwala powiązać żądanie z zalogowanym użytkownikiem. W backendzie sprowadzono to do funkcji “getCurrentUserId”, która zwraca identyfikator wykorzystywany później jako warunek zapytań do bazy danych, najczęściej w polu “ownerId”.

Wspólna funkcja “sanitizeMutation” odpowiada za oczyszczanie danych przed zapisem w bazie. Usuwa ona pola techniczne, których klient nie powinien ustawiać samodzielnie, takie jak “_id”, “id”, “ownerId”, “createdAt” oraz “updatedAt”. Mechanizm ten stanowi dodatkową warstwę ochrony obok walidacji schematem, ponieważ nawet jeżeli w przesłanym obiekcie pojawią się pola systemowe, nie zostaną one przekazane do operacji modyfikującej dokument.

```
import { getSession } from "next-auth";
import { NextResponse } from "next/server";
import { authOptions } from "@lib/auth";
export { sanitizeMutation } from "@lib/sanitize-mutation";

Tabnine | Edit | Test | Explain | Document
export async function getCurrentUserId() {
  const session = await getSession(authOptions);
  return session?.user?.id ?? null;
}

Tabnine | Edit | Test | Explain | Document
export function unauthorizedResponse() {
  return NextResponse.json({ message: "Unauthorized" }, { status: 401 });
}

Tabnine | Edit | Test | Explain | Document
export function notFoundResponse() {
  return NextResponse.json({ message: "Not found" }, { status: 404 });
}
```

Rys. 36. Fragment kodu przedstawiający pobieranie użytkownika z sesji oraz oczyszczanie danych przed zapisem.

Ten fragment pokazuje dwa zabezpieczenia stosowane w wielu miejscach backendu. Funkcja pobierająca identyfikator użytkownika pozwala powiązać operację z właścicielem danych, natomiast oczyszczanie modyfikacji usuwa pola techniczne, których klient nie powinien nadpisywać. Wspólne użycie tych mechanizmów wzmacnia kontrolę dostępu i zmniejsza ryzyko przypadkowej modyfikacji danych systemowych.

Kolejnym wspólnym mechanizmem jest funkcja “recordActivityEvent”, zapisana w

module “activity-events.ts”. Przyjmuje ona identyfikator użytkownika, typ encji, identyfikator encji, nazwę akcji oraz opcjonalne metadane, a następnie tworzy dokument historii aktywności. W aplikacji ten zapis pojawia się między innymi przy tworzeniu, aktualizowaniu, przenoszeniu i archiwizowaniu elementów.

8.3 Implementacja API odpowiedzialnego za autoryzację

Mechanizm autoryzacji został oparty na bibliotece Auth.js, używanej w projekcie przez pakiet NextAuth. Główny handler autoryzacji znajduje się w dynamicznej ścieżce API przeznaczonej dla NextAuth. Plik ten tworzy handler NextAuth na podstawie konfiguracji “authOptions”, a następnie udostępnia go dla metod “GET” i “POST”. Dzięki temu biblioteka może obsługiwać zarówno rozpoczęcie logowania, jak i odpowiedzi zwrotne od dostawców OAuth. Konfiguracja “authOptions” została umieszczona w pliku “src/lib/auth.ts”. Zdefiniowano w nim listę dostawców logowania OAuth, obejmującą Google oraz GitHub. Każdy dostawca posiada identyfikator, nazwę, dane klienta pobierane ze zmiennych środowiskowych oraz funkcję tworzącą konfigurację providera. Aplikacja uruchamia tylko tych dostawców, dla których dostępne są wymagane dane “clientId” i “clientSecret”.

W przypadku logowania z użyciem zewnętrznych dostawców ważne jest nie tylko samo przekazanie użytkownika do usługi logowania, ale również poprawne obsłużenie odpowiedzi zwrotnej, tokenów oraz powiązania konta z lokalnym użytkownikiem aplikacji. Sun i Beznosov, analizując systemy jednokrotnego logowania oparte na OAuth, zwracają uwagę, że wiele problemów bezpieczeństwa wynika nie z idei protokołu, lecz z decyzji implementacyjnych podejmowanych przez twórców aplikacji². Z tego względu w projekcie autoryzacja została skupiona w jednym module konfiguracyjnym, a pozostałe handlersy API korzystają z jednolitej funkcji zwracającej identyfikator aktualnego użytkownika.

Do przechowywania danych autoryzacyjnych wykorzystano adapter MongoDB. Adapter zapisuje w bazie informacje o użytkownikach i powiązanych kontach zewnętrznych, dzięki czemu aplikacja może rozpoznawać tę samą osobę po kolejnych logowaniach. Dane te są przechowywane w kolekcjach technicznych tworzonych przez mechanizm NextAuth, niezależnie od domenowych kolekcji aplikacji, takich jak taski, checklisty czy projekty.

W konfiguracji zastosowano strategię “jwt” oraz callback “session”. Jak opisano w podrozdziale 3.5, sesja służy do rozpoznania zalogowanego użytkownika; w implementacji callback dopisuje identyfikator użytkownika do obiektu “session.user.id”, aby pozostałe części backendu mogły korzystać z jednolitego sposobu identyfikacji właściciela danych.

² S.-T. Sun, K. Beznosov, *The Devil is in the (Implementation) Details: An Empirical Analysis of OAuth SSO Systems*, [w:] *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, 2012, s. 378–390, DOI: 10.1145/2382196.2382238, <https://doi.org/10.1145/2382196.2382238> (dostęp: 31.07.2026).

```

export const authOptions: NextAuthOptions = {
  adapter: MongoDBAdapter(clientPromise),
  session: {
    strategy: "jwt"
  },
  providers: enabledOAuthProviderConfigs.map((config) =>
    config.createProvider(config.clientId, config.clientSecret)
  ),
  pages: {
    signIn: "/login"
  },
  callbacks: {
    async session({ session, token }) {
      if (session.user) {
        session.user.id = token.sub ?? "";
      }

      return session;
    }
  }
};

```

Rys. 37. Fragment kodu przedstawiający konfigurację autoryzacji w *Auth.js*.

Konfiguracja “authOptions” stanowi centralny punkt mechanizmu autoryzacji. W tym miejscu wskazano adapter MongoDB, strategię sesji, stronę logowania oraz listę aktywnych dostawców OAuth. Callback “session” dopisuje identyfikator użytkownika do obiektu sesji, dzięki czemu pozostałe endpointy mogą korzystać z jednej funkcji pobierającej właściciela danych.

W praktyce autoryzacja jest używana w handlerach API przez funkcję pobierającą identyfikator aktualnego użytkownika. Każda operacja na danych użytkownika rozpoczyna się od pobrania identyfikatora sesji. Następnie identyfikator ten trafia do warunków zapytań jako “ownerId”. Takie rozwiązanie sprawia, że nawet poprawnie zbudowane żądanie nie pozwala odczytać ani zmienić danych należących do innej osoby.

9. Testy jednostkowe

Test jednostkowy sprawdza niewielki fragment programu, na przykład funkcję, schemat walidacji albo logikę handlera, bez uruchamiania całego systemu. W projekcie takie testy potraktowano jako sposób kontroli najważniejszych reguł dotyczących tasków: poprawności danych wejściowych, reakcji API na błędne żądania oraz skutków ubocznych operacji, takich jak powiązanie taska z projektem.

W aplikacji testy zostały uruchamiane za pomocą narzędzia Vitest, które jest dostosowane do testowania kodu JavaScript i TypeScript. Dokumentacja Vitest wskazuje, że test opisuje oczekiwane zachowanie wybranego fragmentu kodu, a funkcje “test” oraz “expect” pozwalają definiować sprawdzane przypadki i porównywać wynik działania programu z oczekiwanym rezultatem¹. W kontekście aplikacji React podobną rolę pełnią narzędzia pozwalające renderować komponenty w środowisku testowym i sprawdzać widoczne dla użytkownika elementy interfejsu; dokumentacja React Testing Library opisuje m.in. funkcję “render”, służącą do osadzania komponentu w kontenerze testowym².

Testy przygotowane w projekcie uruchamiane są poleceniem “npm test”, które wykonuje skrypt “vitest run”. Zakres testów obejmuje wybrane, kluczowe fragmenty związane z taskami: walidację danych oraz obsługę endpointów API. Nie jest to pełny zestaw testów całej aplikacji, ponieważ nie obejmuje wszystkich komponentów interfejsu, scenariuszy end-to-end ani integracji z rzeczywistą bazą danych. Stanowi natomiast podstawowe zabezpieczenie najważniejszych reguł biznesowych dotyczących tworzenia, aktualizacji, pobierania i archiwizowania tasków.

9.1 Testy funkcjonalności tasków

Dla funkcjonalności tasków dodano testy jednostkowe w pliku “task-validation.test.ts”. Testy koncentrują się na schematach “taskCreateSchema” oraz “taskUpdateSchema”, ponieważ to one decydują, jakie dane mogą zostać przyjęte przez API odpowiedzialne za tworzenie i aktualizowanie tasków.

Pierwszy test sprawdza poprawny, pełny zestaw danych taska. Weryfikowane są między innymi tytuł, opis, priorytet, status, identyfikator projektu, termin, tagi, powiązane chc-

¹ Vitest, *Writing Tests*, dokumentacja pisania testów w Vitest, [b.r.], <https://vitest.dev/guide/learn/writing-tests.html> (dostęp: 03.08.2026).

² Testing Library, *React Testing Library API*, dokumentacja API biblioteki React Testing Library, [b.r.], <https://testing-library.com/docs/react-testing-library/api/> (dostęp: 03.08.2026).

klisty, nowe checklisty oraz notatki. Test sprawdza również automatyczne przycinanie białych znaków w polach tekstowych, co zapobiega zapisywaniu w bazie danych wartości zawierających przypadkowe spacje na początku lub końcu tekstu.

```
describe("task validation schemas", () => {
  Run | Debug
  it("accepts a complete task payload and trims text fields", () => {
    const result = taskCreateSchema.safeParse({
      title: "  Prepare implementation chapter  ",
      description: "Describe backend and tests",
      priority: "high",
      statusId: "  in_progress  ",
      projectId: "665f1f77bcf86cd799439013",
      dueDate: "2026-07-26T10:00:00.000Z",
      tags: ["  thesis  ", "backend"],
      checklistIds: ["665f1f77bcf86cd799439014"],
      newChecklists: [{ title: "  Review sources  " }],
      noteIds: ["665f1f77bcf86cd799439015"],
      position: 2,
      completedAt: null
    });

    expect(result.success).toBe(true);
    if (!result.success) {
      return;
    }

    expect(result.data.title).toBe("Prepare implementation chapter");
    expect(result.data.statusId).toBe("in_progress");
    expect(result.data.tags).toEqual(["thesis", "backend"]);
    expect(result.data.newChecklists).toEqual([{ title: "Review sources" }]);
  });
});
```

Rys. 38. Fragment testu przedstawiający walidację pełnego zestawu danych taska.

Widoczny test sprawdza nie tylko przyjęcie poprawnego obiektu taska, ale także działanie reguł porządkujących dane tekstowe. Asercje odnoszą się między innymi do tytułu, statusu i tagów, dzięki czemu potwierdzają, że schemat walidacji zwraca dane w oczekiwanej postaci po przejściu przez parser.

Drugi test kontroluje odrzucanie pól, które nie należą do obsługiwanego formatu danych. Dotyczy to zarówno głównego obiektu taska, jak i obiektu nowej checklisty tworzonej razem z taskiem. Jest to istotne, ponieważ API nie powinno przyjmować dowolnych pól przesłanych z klienta. W przeciwnym razie użytkownik mógłby przekazać dane, których aplikacja nie przewiduje w swoim modelu.

Kolejny test sprawdza reakcję walidacji na niepoprawne wartości. Odrzucany jest między innymi nieznany priorytet, błędny format daty oraz puste identyfikatory powiązanych checklist i notatek. Dzięki temu błędne dane zostają zatrzymane przed wykonaniem operacji zapisu w bazie danych.

Ostatni test dotyczy aktualizacji taska. Schemat "taskUpdateSchema" dopuszcza częściową aktualizację, ale wymaga przekazania przynajmniej jednego pola. Test sprawdza więc, że pusty obiekt zostaje odrzucony, natomiast aktualizacja zawierająca samo pole "statusId"

jest uznawana za poprawną. Ma to znaczenie dla działania widoku kanban, w którym zmiana statusu taska może być wykonana bez modyfikowania pozostałych właściwości zadania.

W testach wykorzystano asercje porównujące oczekiwany wynik walidacji z wynikiem rzeczywistym. Dzięki temu możliwe jest szybkie sprawdzenie, czy zmiany w schematach danych nie osłabiły reguł zabezpieczających API tasków. Ten zestaw testów ogranicza ryzyko regresji w formularzach i endpointach odpowiedzialnych za taski, ponieważ błędne dane powinny zostać odrzucone jeszcze przed próbą zapisu w bazie danych.

9.2 Testy endpointów API tasków

Funkcjonalność tasków została sprawdzona również na poziomie handlerów API. W pliku `“src/app/api/tasks/route.test.ts”` przetestowano operacje wykonywane na kolekcji tasków, natomiast w pliku `“src/app/api/tasks/[taskId]/route.test.ts”` operacje dotyczące pojedynczego taska. Testy te nie uruchamiają prawdziwej bazy danych, ponieważ zależności takie jak `“Task”`, `“Project”`, `“connectDatabase”`, `“getCurrentUserId”` oraz `“recordActivityEvent”` są zastępowane atrapami przygotowanymi przez funkcję `“vi.mock”`.

W testach endpointu listowego sprawdzono między innymi odrzucenie żądania niezalogowanego użytkownika, pobieranie aktywnych tasków aktualnego właściciela, walidację danych przed zapisem oraz poprawne utworzenie taska powiązanego z projektem. Szczególnie istotne jest sprawdzenie, że przy tworzeniu taska z polem `“projectId”` backend dopisuje identyfikator taska do tablicy `“taskIds”` projektu oraz zapisuje zdarzenie aktywności z akcją `“created”`.

```

it("creates a task, links it to a project and records activity", async () => {
  vi.mocked(getCurrentUserId).mockResolvedValue("user-1");
  vi.mocked(Project.exists).mockResolvedValue({ _id: "project-1" } as never);
  vi.mocked(Task.create).mockResolvedValue({
    id: "task-1",
    _id: "task-1",
    title: "Write docs",
    projectId: "project-1"
  } as never);
  vi.mocked(Project.updateOne).mockResolvedValue({} as never);

  const response = await POST(
    createJsonRequest({
      title: "Write docs",
      description: "Prepare release notes",
      priority: "high",
      statusId: "todo",
      projectId: "project-1",
      tags: ["docs"]
    }) as never
  );

  expect(response.status).toBe(201);
  expect(Task.create).toHaveBeenCalledWith({
    title: "Write docs",
    description: "Prepare release notes",
    priority: "high",
    statusId: "todo",
    projectId: "project-1",
    tags: ["docs"],
    ownerId: "user-1"
  });
  expect(Project.updateOne).toHaveBeenCalledWith(
    { _id: "project-1", ownerId: "user-1", archivedAt: null },
    { $addToSet: { taskIds: "task-1" } }
  );
  expect(recordActivityEvent).toHaveBeenCalledWith({
    ownerId: "user-1",
    entityType: "task",
    entityId: "task-1",
    action: "created"
  });
});

```

Rys. 39. Fragment testu przedstawiający utworzenie taska, powiązanie go z projektem oraz zapis aktywności.

Na rzucie pokazano test endpointu odpowiedzialnego za tworzenie taska. Przygotowane atrapy modeli pozwalają sprawdzić, czy handler wywołuje “Task.create”, dopisuje identyfikator taska do projektu przez operator “\$addToSet” oraz rejestruje zdarzenie aktywności z akcją “created”. Dzięki temu test obejmuje zarówno główną operację zapisu, jak i jej skutki uboczne.

Testy dynamicznego endpointu taska obejmują odczyt pojedynczego dokumentu, odrzucenie pustej aktualizacji, zmianę projektu przypisanego do taska oraz miękkie usuwanie. Podczas przeniesienia taska między projektami sprawdzane jest usunięcie identyfikatora ze starego projektu i dopisanie go do nowego. Przy usunięciu weryfikowane jest ustawienie pola “archivedAt”, usunięcie powiązania z projektem oraz zapisanie zdarzenia “deleted”. Dzięki temu testy obejmują nie tylko walidację wejścia, ale także najważniejsze skutki uboczne operacji wykonywanych przez API. Testy endpointów wykorzystują atrapy zależności, dlatego

należy traktować je jako testy jednostkowe logiki handlerów, a nie jako pełne testy integracyjne z rzeczywistą bazą danych.

```
it("updates an owned task and moves project links", async () => {
  vi.mocked(getCurrentUserId).mockResolvedValue("user-1");
  vi.mocked(Project.exists).mockResolvedValue({ _id: "new-project" } as never);
  vi.mocked(Task.findOne).mockResolvedValue({ id: taskId, projectId: "old-project" });
  vi.mocked(Task.findOneAndUpdate).mockResolvedValue({
    id: taskId,
    _id: taskId,
    title: "Updated",
    projectId: "new-project"
  });
  vi.mocked(Project.updateOne).mockResolvedValue({} as never);

  const response = await PATCH(
    createJsonRequest({
      title: "Updated",
      projectId: "new-project",
      statusId: "in_progress"
    }) as never,
    context
  );

  expect(response.status).toBe(200);
  expect(Task.findOneAndUpdate).toHaveBeenCalledWith(
    { _id: taskId, ownerId: "user-1", archivedAt: null },
    { $set: { title: "Updated", projectId: "new-project", statusId: "in_progress" } },
    { new: true, runValidators: true }
  );
  expect(Project.updateOne).toHaveBeenCalledWith(
    { _id: "old-project", ownerId: "user-1" },
    { $pull: { taskIds: taskId } }
  );
  expect(Project.updateOne).toHaveBeenCalledWith(
    { _id: "new-project", ownerId: "user-1", archivedAt: null },
    { $addToSet: { taskIds: taskId } }
  );
  expect(recordActivityEvent).toHaveBeenCalledWith({
    ownerId: "user-1",
    entityType: "task",
    entityId: taskId,
    action: "moved"
  });
});
```

Rys. 40. Fragment testu przedstawiający aktualizację taska oraz przeniesienie powiązań między projektami.

Ostatni z dodanych screenów przedstawia test dynamicznego endpointu taska. Weryfikowane jest wywołanie aktualizacji dokumentu, usunięcie identyfikatora taska ze starego projektu przez “\$pull”, dopisanie go do nowego projektu przez “\$addToSet” oraz zapis zdarzenia aktywności. Ten przypadek jest ważny, ponieważ zmiana projektu wpływa nie tylko na sam dokument taska, ale również na powiązania zapisane w dokumentach projektów.

Podsumowanie i zakończenie

Celem pracy było zaprojektowanie oraz zaimplementowanie aplikacji internetowej wspierającej indywidualną organizację zadań, checklist, notatek i projektów. Cel ten został zrealizowany: powstał system, w którym użytkownik może porządkować różne typy informacji, łączyć je z projektami, przypinać najważniejsze elementy do dashboardu i korzystać z aplikacji z poziomu przeglądarki internetowej.

Wykonana aplikacja działa w architekturze klient-serwer i wykorzystuje Next.js, React, TypeScript oraz MongoDB. System umożliwia tworzenie i organizowanie tasków, checklist, notatek oraz projektów, a projekty mogą być prezentowane w formie listy albo tablicy kanban. Wdrożono także przypinanie najważniejszych elementów do dashboardu, logowanie przez zewnętrznych dostawców OAuth, walidację danych wejściowych oraz testy obejmujące wybrane mechanizmy tasków.

Podział na frontend, endpointy API i bazę danych uporządkował odpowiedzialności poszczególnych części systemu. Frontend odpowiada za wygodę pracy i prezentację informacji, backend sprawdza sesję użytkownika oraz poprawność żądań, a MongoDB przechowuje dane w modelu dokumentowym. Takie rozwiązanie dobrze pasuje do projektów z własnymi kolumnami kanban, tagami i powiązаныmi checklistami.

Samodzielny wkład autora obejmował analizę wymagań, przygotowanie koncepcji aplikacji, zaprojektowanie struktury bazy danych, wykonanie interfejsu użytkownika, implementację logiki frontendowej i backendowej, konfigurację autoryzacji, przygotowanie scenariuszy przypadków użycia oraz opracowanie testów jednostkowych dla funkcjonalności tasków. W ramach pracy przygotowano również opis architektury, diagramy, dokumentację implementacji i materiały graficzne wykorzystane w rozdziałach projektowych.

Projekt można dalej rozwijać w kilku kierunkach. Najbardziej naturalne byłoby rozszerzenie testów o scenariusze integracyjne, dodanie powiadomień, dopracowanie wyszukiwania oraz przygotowanie wersji wdrożeniowej aplikacji. W przyszłości możliwe byłoby także współdzielenie projektów między użytkownikami, ale taka funkcja wymagałaby osobnego modelu uprawnień i dokładniejszej kontroli dostępu. Obecna wersja spełnia główne założenie pracy, ponieważ dostarcza przejrzyste narzędzie do indywidualnego zarządzania zadaniami i projektami.

Bibliografia

Ahmad M. O., Dennehy D., Conboy K., Oivo M., *Kanban in Software Engineering: A Systematic Mapping Study*, “Journal of Systems and Software” 2018, Vol. 137, s. 96–113, dostęp: <https://doi.org/10.1016/j.jss.2017.11.045> (31.07.2026).

Albiorix Technology, *Web Application Architecture Layers*, grafika przedstawiająca warstwy architektury aplikacji internetowej, [b.r.], dostęp: <https://www.albiorixtech.com/wp-content/uploads/2025/12/web-application-architecture-layers.jpg> (03.02.2026).

Asana, Inc., *Asana*, strona internetowa aplikacji do zarządzania pracą, [b.r.], dostęp: <https://asana.com> (17.07.2026).

Atlassian, *Trello*, strona internetowa aplikacji kanban do zarządzania zadaniami, [b.r.], dostęp: <https://trello.com> (17.07.2026).

Auth.js, *Auth.js*, dokumentacja biblioteki do obsługi uwierzytelniania, [b.r.], dostęp: <https://authjs.dev> (17.07.2026).

Auth.js, *Auth.js Core Reference*, dokumentacja callbacków biblioteki Auth.js, [b.r.], dostęp: <https://authjs.dev/reference/core#callbacks> (16.08.2026).

Canva, *Creating a Data Flow Diagram: How-tos, Templates, and Tips*, artykuł dotyczący tworzenia diagramów przepływu danych, [b.r.], dostęp: <https://www.canva.com/online-whiteboard/data-flow-diagrams/> (03.02.2026).

ClickUp, *ClickUp*, strona internetowa aplikacji do zarządzania projektami, [b.r.], dostęp: <https://clickup.com> (17.07.2026).

Dean J., *Web Programming with HTML5, CSS, and JavaScript*, Jones & Bartlett Learning, Burlington 2017.

Frontend vs Backend, grafika przedstawiająca porównanie warstwy frontendowej i backendowej, [b.r.], dostęp: <https://d1zruf9db62p8s.cloudfront.net/2025/08/Frontend-vs-Backend.webp> (02.02.2026).

Microsoft, *TypeScript Documentation*, dokumentacja języka TypeScript, [b.r.], dostęp: <http://www.typescriptlang.org/docs/> (25.07.2026).

MongoDB, Inc., *Welcome to the MongoDB Docs*, dokumentacja systemu MongoDB, [b.r.], dostęp: <https://www.mongodb.com/docs/> (02.02.2026).

Mongoose, *Schemas*, dokumentacja biblioteki Mongoose dotycząca schematów danych, [b.r.], dostęp: <https://mongoosejs.com/docs/guide.html> (16.08.2026).

Oracle, *MySQL Documentation*, dokumentacja systemu zarządzania bazą danych MySQL, [b.r.], dostęp: <https://dev.mysql.com/doc/> (25.07.2026).

React, *Built-in React Hooks*, dokumentacja hooków biblioteki React, [b.r.], dostęp: <https://react.dev/reference/react/hooks> (16.08.2026).

React, *React: The Library for Web and Native Interfaces*, dokumentacja biblioteki React, [b.r.], dostęp: <https://react.dev> (28.01.2026).

Sun S.-T., Beznosov K., *The Devil is in the (Implementation) Details: An Empirical Analysis of OAuth SSO Systems*, [w:] *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, 2012, s. 378–390, DOI: 10.1145/2382196.2382238, dostęp: <https://doi.org/10.1145/2382196.2382238> (31.07.2026).

Tailwind Labs, *Tailwind CSS*, dokumentacja frameworka CSS, [b.r.], dostęp: <https://tailwindcss.com> (28.01.2026).

Testing Library, *React Testing Library API*, dokumentacja API biblioteki React Testing Library, [b.r.], dostęp: <https://testing-library.com/docs/react-testing-library/api/> (03.08.2026).

Vercel, *Next.js Docs*, dokumentacja frameworka Next.js, [b.r.], dostęp: <https://nextjs.org/docs> (29.01.2026).

Vercel, *Route Handlers*, dokumentacja Next.js dotycząca obsługi endpointów, [b.r.], dostęp: <https://nextjs.org/docs/app/getting-started/route-handlers> (16.08.2026).

Vitest, *Writing Tests*, dokumentacja pisania testów w Vitest, [b.r.], dostęp: <https://vitest.dev/guide/learn/writing-tests.html> (03.08.2026).

Zod, *Defining schemas*, dokumentacja biblioteki Zod dotycząca definiowania schematów walidacji, [b.r.], dostęp: <https://zod.dev/api?id=property> (16.08.2026).

Spis rysunków

1	Tabela, porównująca Trello, Asanę oraz ClickUp.	7
2	Fragment kodu przedstawiający definiowanie komponentu “Button” w czystym JavaScript, bez nakładki JSX	13
3	Fragment kodu przedstawiający deklaratywne definiowanie komponentu “Button” dzięki JSX	13
4	Fragment kodu przedstawiający klasowy komponent “Button”	14
5	Fragment kodu przedstawiający funkcyjny komponent “Button”	15
6	Przykład funkcji zarządzającej stanem komponentu “Counter”	16
7	Przykładowe użycie Tailwind CSS, pochodzące z oficjalnej strony frameworka (dostęp: 28.01.2026)	17
8	Przedstawienie procesu wyświetlania danych użytkownikowi aplikacji internetowej	25
9	Diagram warstw aplikacji internetowej	29
10	Prosty diagram komponentów internetowej aplikacji do zarządzania zadaniami.	30
11	Diagram przepływu danych poziomu 0, przedstawiający system jako jeden proces wraz z relacjami z otoczeniem.	31
12	Diagram przepływu danych poziomu 1	32
13	Schemat struktury bazy danych aplikacji.	33
14	Diagram UML, wizualizujący scenariusz przypięcia taska	39
15	Widok, przedstawiający checklistę, z możliwością przypięcia jej do ekranu głównego.	43
16	Widok przedstawiający zachowanie przycisku “Przypnij” po przypięciu checklisty do ekranu głównego.	44
17	Widok przedstawiający elementy przypięte do ekranu głównego.	45
18	Widok, przedstawiający stronę z taskami użytkownika.	46
19	Widok, przedstawiający formularz kreacji taska.	47
20	Widok przedstawiający pierwszą część formularza kreacji projektu.	48
21	Widok przedstawiający drugą część formularza kreacji projektu.	48
22	Widok, przedstawiający opcję dodawania tasków do projektu.	49
23	Widok, przedstawiający tablicę kanban projektu.	49

24	Widok, przedstawiający taski projektów w formie listy.	50
25	Fragment kodu przedstawiający wyliczanie kolumn widoku kanban.	52
26	Fragment kodu przedstawiający zmianę statusu taska w widoku kanban. . .	53
27	Fragment kodu przedstawiający filtrowanie i sortowanie przypiętych elementów.	55
28	Fragment kodu przedstawiający obsługę przypinania i odpinania elementów.	57
29	Fragment kodu przedstawiający stan formularza taska oraz dopasowanie statusów do wybranego projektu.	58
30	Fragment kodu przedstawiający tworzenie identyfikatorów oraz normalizację kolumn kanban w formularzu projektu.	60
31	Fragment kodu przedstawiający edycję oraz zmianę kolejności kolumn kanban w formularzu projektu.	61
32	Fragment kodu przedstawiający obsługę metod GET i POST dla checklist. .	64
33	Fragment kodu przedstawiający aktualizację pojedynczej checklisty.	65
34	Fragment kodu przedstawiający archiwizację checklisty w metodzie DELETE.	66
35	Fragment kodu przedstawiający odczyt i walidację ciała żądania API. . . .	67
36	Fragment kodu przedstawiający pobieranie użytkownika z sesji oraz oczyszczanie danych przed zapisem.	68
37	Fragment kodu przedstawiający konfigurację autoryzacji w Auth.js.	70
38	Fragment testu przedstawiający walidację pełnego zestawu danych taska. .	72
39	Fragment testu przedstawiający utworzenie taska, powiązanie go z projektem oraz zapis aktywności.	74
40	Fragment testu przedstawiający aktualizację taska oraz przeniesienie powiązań między projektami.	75

Spis tabel

1	Przykładowa tablica kanban	9
---	--------------------------------------	---